

# DSA Patterns

Java — Interview Reference

---

25 Patterns · 250+ Problems · Production-Quality Java Templates

Recognition Guide · Templates · Dry Runs · Complexity · Cheat Sheets

# DSA Patterns Master Index — Java + LC

## Interview Preparation — Complete 25-Pattern Reference

### Complete File Index (All Gaps Filled)

| #  | File                              | Pattern                                             | Category         |
|----|-----------------------------------|-----------------------------------------------------|------------------|
| 00 | 00_master_index.md                | This file                                           | Navigation       |
| 01 | 01_two_pointers.md                | Two Pointers                                        | Arrays & Strings |
| 02 | 02_prefix_sum.md                  | Prefix Sum                                          | Arrays & Strings |
| 03 | 03_sliding_window.md              | Sliding Window                                      | Arrays & Strings |
| 04 | 04_kadanes_algorithm.md           | Kadane's Algorithm                                  | Arrays & Strings |
| 05 | 05_merge_intervals.md             | Merge Intervals                                     | Arrays & Strings |
| 06 | 06_tree_dfs_recursive.md          | Tree DFS (Recursive)                                | Trees            |
| 07 | 07_tree_bfs.md                    | Tree BFS / Level Order                              | Trees            |
| 08 | 08_tree_dfs_iterative.md          | Tree DFS (Iterative)                                | Trees            |
| 09 | 09_fast_slow_pointers.md          | Fast & Slow Pointers                                | Linked List      |
| 10 | 10_linked_list_reversal_dummy.md  | In-Place Reversal + Dummy Nodes                     | Linked List      |
| 11 | 11_graph_dfs_bfs.md               | Graph DFS & BFS                                     | Graphs           |
| 12 | 12_multi_source_bfs_matrix.md     | Multi-Source BFS + Matrix                           | Graphs           |
| 13 | 13_topological_sort.md            | Topological Sort                                    | Advanced Graphs  |
| 14 | 14_union_find.md                  | Union Find (DSU)                                    | Advanced Graphs  |
| 15 | 15_dijkstra_mst.md                | Dijkstra + Kruskal/Prim's                           | Advanced Graphs  |
| 16 | 16_heaps.md                       | Top K, Two Heaps, Merge K Sorted                    | Heaps            |
| 17 | 17_binary_search.md               | Binary Search (All Variants)                        | Binary Search    |
| 18 | 18_dynamic_programming.md         | Dynamic Programming (Complete)                      | DP               |
| 19 | 19_backtracking.md                | Backtracking (Permutations/Subsets/Combos/N-Queens) | Backtracking     |
| 20 | 20_monotonic_stack_parentheses.md | Monotonic Stack + Parentheses                       | Stacks & Queues  |
| 21 | 21_hashing.md                     | Hashing (All Variants)                              | Hashing          |
| 22 | 22_subarray_count.md              | Subarray Count Problems                             | Arrays & Strings |
| 23 | 23_implicit_graphs.md             | Implicit Graphs / BFS on State Spaces               | Graphs           |
| 24 | 24_heap_greedy_minimum.md         | Heap + Greedy Minimum Problems                      | Heaps            |
| 25 | 25_tree_maze_grid_backtracking.md | Tree Maze & Grid Backtracking                       | Backtracking     |

## Master Pattern Decision Tree

## WHAT KIND OF PROBLEM?

■

### ■■■■ ARRAY / STRING

- ■■■■ Pair/triplet with sum → Two Pointers (sorted) OR Hashing (unsorted)
- ■■■■ Max/min CONTIGUOUS subarray sum → Kadane's
- ■■■■ Longest/shortest subarray with constraint → Sliding Window
- ■■■■ Range sum query → Prefix Sum
- ■■■■ Count subarrays sum=k → Prefix Sum + HashMap
- ■■■■ Count subarrays exactly k distinct → AtMost(k) - AtMost(k-1)
- ■■■■ Overlapping intervals → Merge Intervals

■

### ■■■■ LINKED LIST

- ■■■■ Cycle / middle / nth from end → Fast & Slow Pointers
- ■■■■ Reverse / merge / reorder → In-Place Reversal + Dummy Node

■

### ■■■■ TREE

- ■■■■ Level-by-level / min depth → Tree BFS
- ■■■■ Path sum / height / LCA / max path → Tree DFS Recursive
- ■■■■ BST iterator / no recursion allowed → Tree DFS Iterative
- ■■■■ All root-to-leaf paths → Tree Maze Backtracking (25)
- ■■■■ Path sum any node→node → Prefix Sum on Tree (25)

■

### ■■■■ GRAPH (EXPLICIT)

- ■■■■ Connectivity / flood fill / components → Graph DFS or BFS
- ■■■■ Shortest path UNWEIGHTED → BFS
- ■■■■ Distance from multiple sources simultaneously → Multi-Source BFS
- ■■■■ Shortest path WEIGHTED (non-negative) → Dijkstra
- ■■■■ Shortest path WEIGHTED (negative) → Bellman-Ford
- ■■■■ Minimum spanning tree → Kruskal (sparse) / Prim (dense)
- ■■■■ Task ordering / dependency / cycle detection → Topological Sort
- ■■■■ Dynamic connectivity / merge groups → Union Find

■

### ■■■■ GRAPH (IMPLICIT / STATE SPACE)

- ■■■■ "Min steps to transform X to Y" → Implicit Graph BFS (23)
- ■■■■ "Min moves to reach target state" → Implicit Graph BFS (23)
- ■■■■ Word Ladder / Lock combinations / Sliding puzzle → BFS on States (23)

■

### ■■■■ ORDER STATISTICS / SORTED ACCESS

- ■■■■ Kth largest / top K → Min-Heap of size k
- ■■■■ Median of stream → Two Heaps (16)
- ■■■■ Merge K sorted lists → K-size Min-Heap (16)
- ■■■■ "Always pick best available" greedily → Heap + Greedy (24)
- ■■■■ Items unlock at threshold → Two Heap unlock pattern (24)
- ■■■■ Binary search on sorted / monotonic → Binary Search (17)

■

### ■■■■ ENUMERATION

- ■■■■ ALL permutations / combinations / subsets → Backtracking (19)
- ■■■■ ALL paths in grid visiting all cells → Grid Backtracking (25)
- ■■■■ Exact sequence matching in grid → Word Search Backtracking (19)

■

### ■■■■ OPTIMIZATION WITH CHOICES

- ■■■■ Overlapping subproblems + optimal substructure → DP (18)
- ■■■■ Always one greedy locally optimal choice → Greedy
- ■■■■ Count distinct paths (no must-visit-all) → DP, not backtracking

■

### ■■■■ SPECIAL STRUCTURES

- Next greater/smaller element → Monotonic Stack (20)
- Bracket matching / longest valid → Stack (20)
- Frequency / grouping / O(1) lookup → Hashing (21)
- Pair complement lookup → Hashing (21)

## Time Complexity Reference

O(1) HashMap/Set lookup, Heap peek, Array index  
 O(log n) Binary Search, Heap push/pop, BST ops  
 O(n) Single-pass: Two Pointers, Sliding Window, Kadane's, Prefix Sum, Hashing, Monotonic Stack  
 O(n log n) Sort-first: Merge Intervals, Kruskal's Top K from n elements ( $n \log k \approx n \log n$  when  $k \sim n$ )  
 O(n log k) Heap with k-size: Top K, Merge K sorted  
 O(V + E) Graph traversal: DFS, BFS, Topological Sort  
 O((V+E)logV) Dijkstra with binary heap  
 O(E log E) Kruskal's MST  
 O(n × W) Knapsack DP (W = weight/capacity)  
 O(n × m) 2D DP: LCS, Edit Distance, Grid DP  
 O(n<sup>2</sup>) LIS (naïve), some DP pair states  
 O(S × T) Implicit Graph BFS: S = states, T = transitions per state  
 O(2<sup>n</sup>) Backtracking subsets, some DP bit-mask  
 O(n!) Backtracking permutations

## Space Complexity Reference

O(1) Two Pointers, Kadane's, Binary Search (iterative)  
 O(h) Tree DFS recursion (h = height; O(n) skewed, O(log n) balanced)  
 O(n) Prefix array, HashMap/Set, Stack/Queue, Heap  
 O(W) 1D DP (rolling array optimization)  
 O(V) Graph visited array  
 O(V + E) Adjacency list representation  
 O(n × m) 2D DP table, grid visited  
 O(S) Implicit graph visited states

## Pattern Confusion Guide

| Confused Pair                              | Distinguisher                                                                    |
|--------------------------------------------|----------------------------------------------------------------------------------|
| Sliding Window vs Kadane's                 | Kadane's: negatives OK, decide extend-or-restart. SW: shrink window when invalid |
| Sliding Window vs Prefix Sum               | SW: O(1) state update while sweeping. PS: precompute O(n), then O(1) per query   |
| Prefix+HashMap vs Sliding Window for count | Has negatives → Prefix+HashMap. Non-negative + constraint → Sliding Window       |
| Graph BFS vs Dijkstra                      | Unweighted edges → BFS. Weighted non-negative → Dijkstra                         |
| Topological Sort vs DFS                    | Topo Sort = DFS + post-order collection + cycle check                            |

| Confused Pair                                   | Distinguisher                                                                                                 |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Union Find vs BFS/DFS</b>                    | UF: dynamic connectivity, merge, MST. BFS/DFS: traversal, shortest path                                       |
| <b>0/1 Knapsack vs Unbounded</b>                | 0/1 = inner loop right→left (prevent reuse). Unbounded = left→right (allow reuse)                             |
| <b>Backtracking vs DP</b>                       | Backtracking: enumerate ALL solutions. DP: find OPTIMAL with memoization                                      |
| <b>Tree Maze vs Grid Word Search</b>            | Tree Maze: path satisfies structural property (sum, visit all). Word Search: must spell specific word         |
| <b>Implicit Graph BFS vs Explicit Graph BFS</b> | Implicit: generate neighbors on the fly. Explicit: read from adjacency list                                   |
| <b>Heap+Greedy vs Pure Greedy</b>               | Heap+Greedy: best choice changes dynamically (need $O(\log n)$ access). Pure Greedy: sort once, pick linearly |
| <b>Binary Search on Values vs Two Pointers</b>  | BS: monotonic function, any sorted structure. TP: sorted array, need pairs                                    |

## Interview Signal Phrases → Pattern

| If you hear / read...                       | Pattern                   |
|---------------------------------------------|---------------------------|
| "sorted array, find pair with sum"          | Two Pointers              |
| "maximum sum contiguous subarray"           | Kadane's                  |
| "longest/shortest subarray where..."        | Sliding Window            |
| "range sum / range query"                   | Prefix Sum                |
| "count subarrays with sum = k"              | Prefix + HashMap          |
| "count subarrays with exactly k distinct"   | AtMost(k) - AtMost(k-1)   |
| "merge / combine overlapping intervals"     | Merge Intervals           |
| "linked list cycle / find middle"           | Fast & Slow Pointers      |
| "reverse / reorder linked list"             | In-Place Reversal + Dummy |
| "level order / minimum depth in tree"       | Tree BFS                  |
| "all root-to-leaf paths / path sum"         | Tree DFS + Backtracking   |
| "path sum from any node to any node"        | Prefix Sum on Tree        |
| "BST iterator / in-order without recursion" | Tree DFS Iterative        |
| "connected components / flood fill"         | Graph DFS or BFS          |
| "shortest path (unweighted graph)"          | BFS                       |
| "distance from ALL sources simultaneously"  | Multi-Source BFS          |
| "minimum steps to transform X into Y"       | Implicit Graph BFS        |
| "word ladder / lock dial / sliding puzzle"  | Implicit Graph BFS        |
| "shortest path (weighted, non-negative)"    | Dijkstra                  |

| If you hear / read...                           | Pattern                 |
|-------------------------------------------------|-------------------------|
| "minimum cost to connect all nodes"             | Kruskal's / Prim's      |
| "course prerequisites / task ordering"          | Topological Sort        |
| "dynamic connectivity / merge groups"           | Union Find              |
| "kth largest / top K elements"                  | Min-Heap of size K      |
| "median of a stream"                            | Two Heaps               |
| "merge K sorted lists"                          | K-size Min-Heap         |
| "always pick best available, changes over time" | Heap + Greedy           |
| "find in sorted array in $O(\log n)$ "          | Binary Search           |
| "minimum X such that condition holds"           | Binary Search on Answer |
| "all permutations / all subsets / all combos"   | Backtracking            |
| "all paths visiting all cells"                  | Grid Backtracking       |
| "N-Queens / Sudoku / constraint placement"      | Backtracking            |
| "maximum/minimum with repeated subproblems"     | Dynamic Programming     |
| "number of ways to reach / count paths"         | DP                      |
| "next greater element / stock span"             | Monotonic Stack         |
| "valid / longest valid parentheses"             | Stack                   |
| "two sum / group anagrams / duplicates"         | Hashing                 |

## 6-Week Prep Roadmap

### Week 1 — Array Foundations

- Day 1-2: Two Pointers, Hashing → LC 1, 15, 11, 49, 128
- Day 3-4: Sliding Window, Kadane's → LC 3, 76, 53, 152
- Day 5-6: Prefix Sum, Subarray Count → LC 560, 974, 992
- Day 7: Merge Intervals → LC 56, 57, 253

### Week 2 — Linear Structures

- Day 1-2: Linked List → LC 141, 142, 206, 92, 23
- Day 3-4: Tree DFS (all orders, all variants) → LC 104, 543, 124, 236
- Day 5-6: Tree BFS → LC 102, 199, 103
- Day 7: Tree DFS Iterative + BST → LC 94, 173, 230

### Week 3 — Graphs

- Day 1-2: Graph DFS/BFS → LC 200, 133, 547, 797
- Day 3: Multi-Source BFS → LC 994, 542, 417
- Day 4: Implicit Graph BFS → LC 127, 752, 773
- Day 5: Topological Sort → LC 207, 210, 269

- Day 6-7: Union Find + Dijkstra → LC 684, 721, 743, 1584

## **Week 4 — Sorted Structures**

- Day 1-2: Binary Search (all variants) → LC 704, 34, 33, 875, 410
- Day 3-4: Heaps (Top K, Two Heaps, Merge K) → LC 215, 295, 23, 347
- Day 5-6: Heap + Greedy → LC 621, 767, 871, 502
- Day 7: Monotonic Stack → LC 739, 84, 32, 907

## **Week 5 — DP & Backtracking**

- Day 1-2: Linear DP, Knapsack → LC 322, 416, 198, 300
- Day 3-4: 2D DP (LCS, Edit Distance, Grid) → LC 1143, 72, 62, 516
- Day 5-6: Backtracking → LC 78, 46, 39, 51, 131
- Day 7: Grid Backtracking + Tree Maze → LC 113, 437, 980

## **Week 6 — Mock Interviews**

- 2 timed problems per day (45 min each)
- One easy/medium warm-up + one medium/hard
- Practice talking through pattern recognition out loud
- Review all cheat sheets (bottom of each file)

---

## **Edge Cases Always Test**

- Empty input (n=0, null root, empty string)
- Single element
- All same elements
- All negative values (Kadane's, sums)
- Target not reachable (BFS, backtracking)
- Integer overflow (use long for sums)
- Cycle in graph (mark visited!)
- Skewed tree (DFS stack depth)
- Duplicate elements (sort + skip)
- $k > n$  (top K when fewer elements exist)

# Pattern 01 — Two Pointers

---

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "pair", "triplet", "two elements that sum to X"
- "sorted array", "palindrome", "reverse"
- "remove duplicates", "merge sorted arrays"
- "container with most water", "minimum difference between pairs"

**When to reach for Two Pointers:**

- Array/string is **sorted** (or can be sorted)
- You need to find **pairs/triplets** with a condition
- You need to **avoid  $O(n^2)$**  brute force
- Problem involves comparing elements from **both ends** or **two sequences**

**Signal phrase:** `"Find two elements such that..."` → Two Pointers

---

## 2. Core Intuition

**Why brute force fails:**

Checking every pair =  $O(n^2)$ . For  $n=10^4$  that's  $10^8$  operations — TLE.

**The key observation:**

In a sorted array, if `arr[left] + arr[right] > target`, moving `right` left gives a smaller sum. If sum is too small, move `left` right. You **converge** in  $O(n)$ .

**Core invariant:**

One pointer starts at the beginning, one at the end. Each step eliminates a possibility, so you never revisit the same pair.

```
[1, 2, 3, 4, 6] target = 6
L R → 1+6=7 > 6 → move R left
L R → 1+4=5 < 6 → move L right
L R → 2+4=6 ✓ FOUND
```

---

## 3. Generic Java Templates

**Template A — Opposite Ends (Pair Sum / Palindrome)**

```

public int[] twoSumSorted(int[] arr, int target) {
    int left = 0;
    int right = arr.length - 1;

    while (left < right) {
        int sum = arr[left] + arr[right];

        if (sum == target) {
            return new int[]{left, right}; // found
        } else if (sum < target) {
            left++; // need larger sum
        } else {
            right--; // need smaller sum
        }
    }
    return new int[]{-1, -1}; // not found
}

```

### Template B — Same Direction (Remove Duplicates / Fast-Slow)

```

public int removeDuplicates(int[] arr) {
    if (arr.length == 0) return 0;

    int slow = 0; // boundary of unique elements

    for (int fast = 1; fast < arr.length; fast++) {
        if (arr[fast] != arr[slow]) {
            slow++;
            arr[slow] = arr[fast];
        }
    }
    return slow + 1; // length of unique portion
}

```

### Template C — Two Arrays (Merge Sorted)

```

public int[] mergeSorted(int[] a, int[] b) {
    int i = 0, j = 0, k = 0;
    int[] result = new int[a.length + b.length];

    while (i < a.length && j < b.length) {
        if (a[i] <= b[j]) result[k++] = a[i++];
        else result[k++] = b[j++];
    }
    while (i < a.length) result[k++] = a[i++];
    while (j < b.length) result[k++] = b[j++];

    return result;
}

```

---

## 4. Complexity Cheatsheet

| Variant        | Time          | Space      | Notes                  |
|----------------|---------------|------------|------------------------|
| Opposite ends  | $O(n)$        | $O(1)$     | Requires sorted input  |
| Same direction | $O(n)$        | $O(1)$     | In-place modification  |
| Two arrays     | $O(n + m)$    | $O(n + m)$ | Extra array for result |
| With sort      | $O(n \log n)$ | $O(1)$     | Sort cost dominates    |

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                             | Difficulty | Why Important              |
|------|-------------------------------------|------------|----------------------------|
| 167  | Two Sum II (sorted array)           | Easy       | Classic template starter   |
| 26   | Remove Duplicates from Sorted Array | Easy       | Same-direction template    |
| 125  | Valid Palindrome                    | Easy       | Char comparison + skip     |
| 15   | 3Sum                                | Medium     | Two pointers inside a loop |
| 11   | Container With Most Water           | Medium     | Greedy pointer movement    |

### Advanced Variations

| LC # | Problem                   | Difficulty | Why Important                 |
|------|---------------------------|------------|-------------------------------|
| 16   | 3Sum Closest              | Medium     | Track closest sum             |
| 18   | 4Sum                      | Medium     | Nested two pointers           |
| 42   | Trapping Rain Water       | Hard       | Two-pointer with max tracking |
| 75   | Sort Colors (Dutch Flag)  | Medium     | Three pointers                |
| 977  | Squares of a Sorted Array | Easy       | Merge from outside in         |

## 6. Solve Step-by-Step — LC 15: 3Sum

**Problem:** Find all unique triplets in array that sum to zero.

### Step 1 — Brute Force $O(n^3)$

```
// Check every combination of 3 elements → TLE for large input
for (int i = 0; i < n; i++)
    for (int j = i+1; j < n; j++)
        for (int k = j+1; k < n; k++)
            if (arr[i]+arr[j]+arr[k]==0) add triplet
```

### Step 2 — Better $O(n^2)$ with HashSet

Use a set for the third element. Still  $O(n^2)$  time but  $O(n)$  space.

### Step 3 — Optimal $O(n^2)$ Two Pointers

```
public List<List<Integer>> threeSum(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    Arrays.sort(nums); // CRITICAL: sort first

    for (int i = 0; i < nums.length - 2; i++) {
        // Skip duplicates for i
        if (i > 0 && nums[i] == nums[i - 1]) continue;

        // Early exit: smallest triplet already > 0
        if (nums[i] > 0) break;

        int left = i + 1;
        int right = nums.length - 1;

        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];

            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                // Skip duplicates for left and right
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;
                left++;
                right--;
            } else if (sum < 0) {
                left++;
            } else {
                right--;
            }
        }
        return result;
    }
}
```

### Dry Run

Input: [-4, -1, -1, 0, 1, 2]  
i=0: nums[i]=-4, L=-1, R=2 → sum=-3 < 0 → L++  
L=0, R=2 → sum=-2 < 0 → L++  
L=1, R=2 → sum=-1 < 0 → L++  
L=R → stop  
i=1: nums[i]=-1, L=0, R=2 → sum=1 > 0 → R--  
L=0, R=1 → sum=0 ✓ → add [-1,0,1], skip dupes  
i=2: skip (nums[2]==nums[1])  
...  
Result: [[-1,0,1],[-1,-1,2]]

### Edge Cases

- All zeros: [0,0,0,0] → [[0,0,0]] (must deduplicate)
- All positive: [1,2,3] → []

- Length < 3: return empty

## 7. Pattern Variations

| Variation              | Approach                           |
|------------------------|------------------------------------|
| Pair sum (sorted)      | Classic opposite ends              |
| Pair sum (unsorted)    | Sort first, then two pointers      |
| 3Sum / 4Sum            | Outer loop(s) + inner two pointers |
| Remove elements        | Slow/fast same-direction           |
| Palindrome check       | Compare chars from both ends       |
| Merge two arrays       | Two pointers on both arrays        |
| Partition (Dutch Flag) | Three pointers (low/mid/high)      |

## 8. Common Interview Mistakes

1. **Forgetting to sort** before applying opposite-ends template — results are wrong
2. **Not skipping duplicates** in 3Sum — duplicate triplets in output
3. `left < right` **vs** `left <= right` — use `<` for pairs (need 2 distinct elements)
4. **Infinite loop** when not advancing both pointers after finding a match
5. **Integer overflow** in sum — use `long` when values are large
6. **Modifying array** when problem says not to — check constraints

## 9. Interview Cheat Sheet

### TWO POINTERS – MENTAL CHECKLIST

=====

- Is array sorted? (or should I sort it?)
- Am I looking for a pair/triplet satisfying a condition?
- Can I eliminate possibilities with each pointer move?
- Opposite ends OR same direction?
- Handle duplicates (sort + skip)
- Watch for `i < j` vs `i <= j` boundary

### FORMULAS

=====

Opposite ends: `sum < target → left++` | `sum > target → right--`

Same direction: `arr[fast] != arr[slow] → slow++`, copy

### TRICKS

=====

- Sort first:  $O(n \log n)$  but enables  $O(n)$  scan
- Nested loop + two pointers =  $O(n^2)$  for kSum
- Three pointers for Dutch Flag / partition problems

## 10. Practice Roadmap

### Beginner (warm up):

- LC 167 — Two Sum II
- LC 26 — Remove Duplicates
- LC 125 — Valid Palindrome
- LC 977 — Squares of Sorted Array

### Intermediate (pattern fluency):

- LC 15 — 3Sum
- LC 11 — Container With Most Water
- LC 16 — 3Sum Closest
- LC 75 — Sort Colors

### Hard:

- LC 42 — Trapping Rain Water
- LC 18 — 4Sum
- LC 259 — 3Sum Smaller (premium)
- LC 632 — Smallest Range Covering Elements from K Lists

# Pattern 02 — Prefix Sum

---

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "sum of subarray", "range sum query"
- "number of subarrays with sum equal to K"
- "running total", "cumulative sum"
- "between index i and j"

**When to reach for Prefix Sum:**

- Multiple range-sum queries on the same array
- Subarray sum equals a target
- Count of subarrays satisfying a sum condition
- 2D matrix rectangle sum queries

**Signal phrase:** `"Sum of elements between index i and j"` → Prefix Sum

**Signal phrase:** `"Number of subarrays with sum = k"` → Prefix Sum + HashMap

---

## 2. Core Intuition

**Why brute force fails:**

Recomputing the sum from scratch for each query =  $O(n)$  per query. For  $Q$  queries:  $O(n \times Q)$ .

**The key observation:**

If you precompute `prefix[i] = arr[0] + arr[1] + ... + arr[i]`, then:

`sum(i, j) = prefix[j] - prefix[i - 1]`

Answer any range sum in  $O(1)$  after  $O(n)$  preprocessing.

**Core invariant:**

`prefix[0] = 0` (empty prefix – critical for edge cases)

`prefix[i] = prefix[i-1] + arr[i-1]` (1-indexed prefix)

**Visual:**

`arr = [3, 1, 4, 1, 5]`

`prefix = [0, 3, 4, 8, 9, 14]`

^

sentinel zero

`sum(2,4) = prefix[4] - prefix[1] = 9 - 3 = 6 ✓ (1+4+1=6)`

---

## 3. Generic Java Templates

### Template A — Basic Prefix Sum (Range Query)

```
// Preprocessing: O(n)
public int[] buildPrefix(int[] arr) {
    int n = arr.length;
    int[] prefix = new int[n + 1]; // size n+1; prefix[0] = 0 sentinel
    for (int i = 1; i <= n; i++) {
        prefix[i] = prefix[i - 1] + arr[i - 1];
    }
    return prefix;
}

// Query: O(1) [l, r] are 0-indexed inclusive
public int rangeSum(int[] prefix, int l, int r) {
    return prefix[r + 1] - prefix[l];
}
```

## Template B — Subarray Sum Equals K (HashMap)

```
// Count subarrays with sum exactly equal to k
public int subarraySum(int[] nums, int k) {
    // Map: prefixSum → count of times seen
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1); // empty subarray has sum 0

    int count = 0;
    int prefixSum = 0;

    for (int num : nums) {
        prefixSum += num;

        // If (prefixSum - k) was seen before, those subarrays end here with sum = k
        count += prefixCount.getOrDefault(prefixSum - k, 0);

        // Record current prefix sum
        prefixCount.merge(prefixSum, 1, Integer::sum);
    }
    return count;
}
```

## Template C — 2D Prefix Sum (Matrix Rectangle Query)

```
// Build 2D prefix sum
public int[][] build2DPrefix(int[][] matrix) {
    int rows = matrix.length;
    int cols = matrix[0].length;
    int[][] prefix = new int[rows + 1][cols + 1];

    for (int r = 1; r <= rows; r++) {
        for (int c = 1; c <= cols; c++) {
            prefix[r][c] = matrix[r-1][c-1]
            + prefix[r-1][c]
            + prefix[r][c-1]
            - prefix[r-1][c-1]; // subtract double-counted corner
        }
    }
    return prefix;
}

// Query: sum of rectangle (r1,c1) to (r2,c2) – 0-indexed
public int rectSum(int[][] prefix, int r1, int c1, int r2, int c2) {
    return prefix[r2+1][c2+1]
    - prefix[r1][c2+1]
    - prefix[r2+1][c1]
    + prefix[r1][c1]; // add back double-subtracted corner
}
```

## 4. Complexity Cheatsheet

| Variant                       | Build Time      | Query Time | Space           |
|-------------------------------|-----------------|------------|-----------------|
| 1D prefix sum                 | $O(n)$          | $O(1)$     | $O(n)$          |
| Subarray sum = k<br>(HashMap) | $O(n)$          | —          | $O(n)$          |
| 2D prefix sum                 | $O(n \times m)$ | $O(1)$     | $O(n \times m)$ |

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                      | Difficulty | Why Important                     |
|------|------------------------------|------------|-----------------------------------|
| 303  | Range Sum Query - Immutable  | Easy       | Pure prefix sum template          |
| 724  | Find Pivot Index             | Easy       | Balance left/right sums           |
| 525  | Contiguous Array (0s and 1s) | Medium     | Clever prefix with transformation |
| 560  | Subarray Sum Equals K        | Medium     | HashMap + prefix sum combo        |
| 304  | Range Sum Query 2D           | Medium     | 2D prefix template                |

## Advanced Variations

| LC # | Problem                                  | Difficulty | Why Important                       |
|------|------------------------------------------|------------|-------------------------------------|
| 974  | Subarray Sums Divisible by K             | Medium     | Prefix sum + modulo arithmetic      |
| 1248 | Count Number of Nice Subarrays           | Medium     | Transform odds to 1/0, apply prefix |
| 327  | Count of Range Sum                       | Hard       | Merge sort + prefix sum             |
| 1074 | Number of Submatrices that Sum to Target | Hard       | 2D prefix + HashMap                 |
| 862  | Shortest Subarray with Sum at Least K    | Hard       | Deque + prefix sum                  |

## 6. Solve Step-by-Step — LC 560: Subarray Sum Equals K

**Problem:** Count subarrays with sum exactly equal to k.

### Step 1 — Brute Force $O(n^2)$

```
int count = 0;
for (int i = 0; i < nums.length; i++) {
    int sum = 0;
    for (int j = i; j < nums.length; j++) {
        sum += nums[j];
        if (sum == k) count++;
    }
}
```

### Step 2 — Optimal $O(n)$ — Prefix Sum + HashMap

**Key insight:**  $\text{sum}(i, j) = k$  means  $\text{prefix}[j] - \text{prefix}[i-1] = k$

So we need  $\text{prefix}[i-1] = \text{prefix}[j] - k$

Count how many previous prefix sums equal  $\text{currentPrefix} - k$ .

```

public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> map = new HashMap<>();
    map.put(0, 1); // handle subarrays starting at index 0

    int prefixSum = 0;
    int count = 0;

    for (int num : nums) {
        prefixSum += num;

        // How many previous positions had (prefixSum - k)?
        // Each such position marks the start of a valid subarray ending here
        count += map.getOrDefault(prefixSum - k, 0);

        map.merge(prefixSum, 1, Integer::sum);
    }
    return count;
}

```

## Dry Run

nums = [1, 2, 3], k = 3  
map = {0:1}

i=0: num=1, prefix=1, lookup (1-3=-2)→0, map={0:1, 1:1}  
i=1: num=2, prefix=3, lookup (3-3=0)→1 ✓, count=1, map={0:1, 1:1, 3:1}  
i=2: num=3, prefix=6, lookup (6-3=3)→1 ✓, count=2, map={0:1, 1:1, 3:1, 6:1}

Result: 2 ([1,2] and [3]) ✓

## Edge Cases

- Negative numbers: works! HashMap handles negative prefix sums
- k = 0: map.put(0, 1) sentinel handles subarrays of sum 0
- Single element equals k: caught by prefix[j] - 0 = k with sentinel

## 7. Pattern Variations

| Variation                                    | Technique                         |
|----------------------------------------------|-----------------------------------|
| Range sum (static array)                     | Build prefix, answer in O(1)      |
| Range sum (mutable array)                    | Segment tree or BIT (Fenwick)     |
| Subarray sum = k                             | Prefix + HashMap                  |
| Subarray sum divisible by k                  | Prefix mod k + HashMap            |
| Count 0/1 subarrays                          | Transform 0→-1, apply sum=0 trick |
| 2D rectangle sum                             | 2D prefix                         |
| Minimum length subarray ≥ k (positive)       | Sliding window is better          |
| Minimum length subarray ≥ k (with negatives) | Deque + prefix sum                |

## 8. Common Interview Mistakes

1. **Forgetting the sentinel** `prefix[0] = 0` — misses subarrays starting at index 0
  2. **Off-by-one in indexing** — use `prefix[r+1] - prefix[l]` carefully
  3. **Using prefix sum when array is mutable** — use BIT/segment tree instead
  4. **Not handling negative numbers** — prefix sum still works, but sliding window won't
  5. **2D formula error** — forgetting to add back the doubly-subtracted corner: `+prefix[r1][c1]`
- 

## 9. Interview Cheat Sheet

### PREFIX SUM — MENTAL CHECKLIST

=====

- Range sum query? → Build prefix array
- Subarray sum = k? → prefix + HashMap, sentinel {0:1}
- 2D rectangle sum? → 2D prefix, inclusion-exclusion
- Has negative values? → Still works; sliding window won't
- Array is mutable? → Need Fenwick/Segment Tree instead

### FORMULAS

=====

1D: `sum(l,r) = prefix[r+1] - prefix[l]`

2D: `sum = P[r2+1][c2+1] - P[r1][c2+1] - P[r2+1][c1] + P[r1][c1]`

KEY: `count += map.get(prefixSum - k)` → subarray sum = k trick

### TRICKS

=====

- Always put {0:1} in map before loop (subarrays from index 0)
  - Transform `0→-1` to convert "equal count of 0s and 1s" → "sum=0"
  - Prefix mod k for divisibility problems: handles negative remainder with `+k`
- 

## 10. Practice Roadmap

### Beginner:

- LC 303 — Range Sum Query
- LC 724 — Pivot Index
- LC 1480 — Running Sum of 1D Array

### Intermediate:

- LC 560 — Subarray Sum Equals K
- LC 525 — Contiguous Array
- LC 974 — Subarray Sums Divisible by K
- LC 304 — Range Sum Query 2D

### Hard:

- LC 862 — Shortest Subarray with Sum  $\geq K$
- LC 1074 — Number of Submatrices that Sum to Target
- LC 327 — Count of Range Sum

# Pattern 03 — Sliding Window

---

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "subarray / substring of length k"
- "longest / shortest subarray with condition"
- "contiguous elements"
- "maximum/minimum sum in window"
- "at most k distinct", "exactly k"

**When to reach for Sliding Window:**

- Contiguous subarray/substring (not subsequence)
- Fixed or dynamic window over a sequence
- Optimizing over all subarrays —  $O(n)$  instead of  $O(n^2)$

**Signal phrase:** \_"Longest substring without repeating characters"\_ → Dynamic Sliding Window

**Signal phrase:** \_"Maximum sum subarray of size k"\_ → Fixed Sliding Window

---

## 2. Core Intuition

**Why brute force fails:**

Checking every subarray:  $O(n^2)$  for 2 nested loops — TLE.

**The key observation:**

Instead of recomputing the entire window each time, slide it:

- Add the new element entering the window (right pointer advances)
- Remove the old element leaving the window (left pointer advances when invalid)

**Core invariant:**

The window `[left, right]` always satisfies the problem's constraint.

When the constraint is violated, shrink from the left.

Fixed Window ( $k=3$ ):

`[1, 3, -1, -3, 5, 3, 6]`

`L R` → add right, remove left when window  $> k$

Dynamic Window:

expand right until invalid → shrink left until valid again  
track max/min at each valid state

---

## 3. Generic Java Templates

### Template A — Fixed Window Size k

```

public int maxSumFixed(int[] arr, int k) {
    int n = arr.length;
    if (n < k) return -1;

    // Build initial window
    int windowSum = 0;
    for (int i = 0; i < k; i++) {
        windowSum += arr[i];
    }

    int maxSum = windowSum;

    // Slide: add right element, remove left element
    for (int right = k; right < n; right++) {
        windowSum += arr[right];
        windowSum -= arr[right - k]; // element leaving window
        maxSum = Math.max(maxSum, windowSum);
    }

    return maxSum;
}

```

## Template B — Dynamic Window (Longest Valid Subarray)

```

// Longest subarray/substring satisfying a condition
public int longestValid(int[] arr) {
    int left = 0;
    int maxLen = 0;
    // state: whatever you need to track the window's validity

    for (int right = 0; right < arr.length; right++) {
        // 1. Expand: include arr[right] in window state

        // 2. Shrink: while window is invalid, move left
        while (/* window is INVALID */) {
            // remove arr[left] from window state
            left++;
        }

        // 3. Window [left, right] is now valid – update answer
        maxLen = Math.max(maxLen, right - left + 1);
    }

    return maxLen;
}

```

## Template C — Dynamic Window (Shortest Valid Subarray)

```

public int shortestValid(int[] arr, int target) {
    int left = 0;
    int minLen = Integer.MAX_VALUE;
    int windowSum = 0;

    for (int right = 0; right < arr.length; right++) {
        windowSum += arr[right]; // expand

        // Shrink as long as window remains valid
        while (windowSum >= target) {
            minLen = Math.min(minLen, right - left + 1);
            windowSum -= arr[left];
            left++;
        }
    }

    return minLen == Integer.MAX_VALUE ? 0 : minLen;
}

```

### Template D — At Most K (Count subarrays with at most k distinct)

```

// Trick: "exactly k" = atMost(k) - atMost(k-1)
public int atMostK(int[] arr, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    int left = 0, count = 0;

    for (int right = 0; right < arr.length; right++) {
        freq.merge(arr[right], 1, Integer::sum);

        while (freq.size() > k) {
            freq.merge(arr[left], -1, Integer::sum);
            if (freq.get(arr[left]) == 0) freq.remove(arr[left]);
            left++;
        }

        count += right - left + 1; // all subarrays ending at right
    }
    return count;
}

```

## 4. Complexity Cheatsheet

| Variant                           | Time   | Space  |
|-----------------------------------|--------|--------|
| Fixed window                      | $O(n)$ | $O(1)$ |
| Dynamic window (array tracking)   | $O(n)$ | $O(1)$ |
| Dynamic window (HashMap tracking) | $O(n)$ | $O(k)$ |
| At most K distinct                | $O(n)$ | $O(k)$ |

The `left` pointer never goes backward — each element enters/leaves the window once  $\rightarrow O(n)$ .

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                                        | Difficulty | Why Important           |
|------|------------------------------------------------|------------|-------------------------|
| 643  | Maximum Average Subarray I                     | Easy       | Fixed window intro      |
| 3    | Longest Substring Without Repeating Characters | Medium     | Dynamic window with set |
| 76   | Minimum Window Substring                       | Hard       | Classic shrink template |
| 209  | Minimum Size Subarray Sum                      | Medium     | Shortest valid window   |
| 424  | Longest Repeating Character Replacement        | Medium     | Window with constraint  |

### Advanced Variations

| LC # | Problem                             | Difficulty | Why Important             |
|------|-------------------------------------|------------|---------------------------|
| 567  | Permutation in String               | Medium     | Fixed window + char freq  |
| 438  | Find All Anagrams in a String       | Medium     | Fixed window with map     |
| 1004 | Max Consecutive Ones III            | Medium     | At most k flips           |
| 992  | Subarrays with K Different Integers | Hard       | AtMost trick              |
| 480  | Sliding Window Median               | Hard       | Window + sorted structure |

## 6. Solve Step-by-Step — LC 3: Longest Substring Without Repeating Characters

**Problem:** Find length of longest substring with all unique characters.

### Step 1 — Brute Force $O(n^3)$

```
// Generate all substrings, check each for uniqueness
//  $O(n^2)$  substrings  $\times O(n)$  uniqueness check =  $O(n^3)$ 
```

### Step 2 — $O(n^2)$ with HashSet

Start each position, extend until duplicate found.

### Step 3 — Optimal $O(n)$ — Dynamic Sliding Window

```

public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> lastSeen = new HashMap<>(); // char → last index
    int left = 0;
    int maxlen = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);

        // If char was seen within current window, jump left past it
        if (lastSeen.containsKey(c) && lastSeen.get(c) >= left) {
            left = lastSeen.get(c) + 1;
        }

        lastSeen.put(c, right);
        maxlen = Math.max(maxlen, right - left + 1);
    }

    return maxlen;
}

```

## Dry Run

s = "abcabcbb"  
 0123456 7

```

right=0 c=a: map={a:0}, left=0, len=1
right=1 c=b: map={a:0,b:1}, left=0, len=2
right=2 c=c: map={a:0,b:1,c:2}, left=0, len=3
right=3 c=a: a seen at 0 ≥ left(0) → left=1, map={a:3,b:1,c:2}, len=3
right=4 c=b: b seen at 1 ≥ left(1) → left=2, map={a:3,b:4,c:2}, len=3
right=5 c=c: c seen at 2 ≥ left(2) → left=3, map={a:3,b:4,c:5}, len=3
right=6 c=b: b seen at 4 ≥ left(3) → left=5, map={a:3,b:6,c:5}, len=3 (6-5+1=2, max=3)
right=7 c=b: b seen at 6 ≥ left(5) → left=7, len=1

```

Answer: 3 ("abc")

## Edge Cases

- Empty string → 0
- All same characters "aaaa" → 1
- All unique "abcd" → 4
- Unicode / spaces → use HashMap (not array of 128)

## 7. Pattern Variations

| Variation                | Template   | Key State        |
|--------------------------|------------|------------------|
| Fixed window sum         | Template A | running sum      |
| Longest unique chars     | Template B | HashMap lastSeen |
| Shortest subarray ≥ sum  | Template C | running sum      |
| At most k distinct chars | Template D | freq map         |

| Variation                     | Template          | Key State      |
|-------------------------------|-------------------|----------------|
| Exactly k distinct            | $D(k) - D(k-1)$   | two passes     |
| Max ones with k flips         | Template B        | count of zeros |
| Anagram/permutation detection | Fixed + char freq | freq array[26] |

## 8. Common Interview Mistakes

1. **Moving left by 1 each time** when you can jump it directly (use `lastSeen` map)
2. **Checking** `lastSeen.get(c) >= left` — without this, you jump left backward!
3. **Forgetting to update answer inside the loop** (not just at the end)
4. **Using shrink-while for longest** and expand-while for shortest — they're opposite!
5. **Fixed window: forgetting** `arr[right - k]` — the element leaving the window
6. **AtMost trick confusion:**  $\text{exactly}(k) = \text{atMost}(k) - \text{atMost}(k-1)$  — remember which side

## 9. Interview Cheat Sheet

### SLIDING WINDOW — MENTAL CHECKLIST

=====

- Contiguous subarray/substring? (not subsequence)
- Fixed size k? → Template A
- Longest valid window? → Template B (shrink while invalid)
- Shortest valid window? → Template C (shrink while valid)
- Count subarrays with exactly k? →  $\text{atMost}(k) - \text{atMost}(k-1)$
- Tracking what in window? (sum / freq map / set)

### FORMULAS

=====

Window length = `right - left + 1`  
Fixed window remove = `arr[right - k]`  
Longest: update answer AFTER shrinking  
Shortest: update answer BEFORE shrinking

### TRICKS

=====

- `HashMap<Char, lastIndex>` for  $O(1)$  jump on duplicate
- Char freq with `int[26]` when only lowercase letters
- "Exactly K" always decomposes into two "at most" calls
- Each element enters window once, leaves once →  $O(n)$  guaranteed

## 10. Practice Roadmap

### Beginner:

- LC 643 — Maximum Average Subarray I
- LC 1876 — Substrings of Size Three with Distinct Characters
- LC 219 — Contains Duplicate II

### Intermediate:

- LC 3 — Longest Substring Without Repeating Characters
- LC 209 — Minimum Size Subarray Sum
- LC 567 — Permutation in String
- LC 1004 — Max Consecutive Ones III
- LC 424 — Longest Repeating Character Replacement

**Hard:**

- LC 76 — Minimum Window Substring
- LC 992 — Subarrays with K Different Integers
- LC 480 — Sliding Window Median
- LC 239 — Sliding Window Maximum (monotonic deque!)

# Pattern 04 — Kadane's Algorithm

---

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "maximum sum subarray"
- "contiguous subarray with largest sum"
- "maximum product subarray"
- "maximum circular subarray sum"

**When to reach for Kadane's:**

- Maximum/minimum of a contiguous subarray
- Can contain negative numbers
- $O(n)$  is required (brute force is  $O(n^2)$  or  $O(n^3)$ )

**Signal phrase:** \_"Find the subarray with the maximum sum"\_ → Kadane's

**Extended signal:** \_"Maximum product"\_ or \_"Circular subarray"\_ → Kadane's variant

---

## 2. Core Intuition

**Why brute force fails:**

All  $O(n^2)$  pairs of  $(i, j) = 10^4$  for  $n=10^4$ .

**The key observation:**

At each position, you make one decision:

1. **Extend** the previous subarray (add current element to it)
2. **Start fresh** (begin a new subarray at current element)

Pick whichever gives a larger sum.

**Core invariant:**

`currentMax = max(arr[i], currentMax + arr[i])`

If `currentMax` ever drops below `arr[i]`, it means the previous subarray hurts more than it helps — restart.

```
arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]

i=0: cur=max(-2, -2) = -2 globalMax=-2
i=1: cur=max( 1, -2+1) = 1 globalMax=1
i=2: cur=max(-3, 1-3) = -2 globalMax=1
i=3: cur=max( 4, -2+4) = 4 globalMax=4
i=4: cur=max(-1, 4-1) = 3 globalMax=4
i=5: cur=max( 2, 3+2) = 5 globalMax=5
i=6: cur=max( 1, 5+1) = 6 globalMax=6 ← answer
i=7: cur=max(-5, 6-5) = 1 globalMax=6
i=8: cur=max( 4, 1+4) = 5 globalMax=6

Max subarray = [4,-1,2,1], sum = 6
```

---

### 3. Generic Java Templates

#### Template A — Basic Kadane (Maximum Subarray Sum)

```
public int maxSubArray(int[] nums) {
    int currentMax = nums[0]; // must start with first element (not 0!)
    int globalMax = nums[0];

    for (int i = 1; i < nums.length; i++) {
        // Either extend or restart
        currentMax = Math.max(nums[i], currentMax + nums[i]);
        globalMax = Math.max(globalMax, currentMax);
    }

    return globalMax;
}
```

#### Template B — With Subarray Indices Tracking

```

public int[] maxSubArrayWithIndices(int[] nums) {
    int currentMax = nums[0];
    int globalMax = nums[0];
    int start = 0, end = 0, tempStart = 0;

    for (int i = 1; i < nums.length; i++) {
        if (nums[i] > currentMax + nums[i]) {
            currentMax = nums[i];
            tempStart = i; // potential new start
        } else {
            currentMax += nums[i];
        }

        if (currentMax > globalMax) {
            globalMax = currentMax;
            start = tempStart;
            end = i;
        }
    }

    return new int[]{globalMax, start, end};
}

```

## Template C — Maximum Product Subarray

```

// Key difference: negatives can become positive when multiplied by another negative
// Track BOTH max and min at each step
public int maxProduct(int[] nums) {
    int maxProd = nums[0];
    int minProd = nums[0]; // tracks most negative product
    int result = nums[0];

    for (int i = 1; i < nums.length; i++) {
        // Multiplying by negative swaps max and min
        if (nums[i] < 0) {
            int temp = maxProd;
            maxProd = minProd;
            minProd = temp;
        }

        maxProd = Math.max(nums[i], maxProd * nums[i]);
        minProd = Math.min(nums[i], minProd * nums[i]);

        result = Math.max(result, maxProd);
    }

    return result;
}

```

## Template D — Maximum Circular Subarray Sum

```
// Circular = either a normal subarray OR total_sum - minimum_subarray
public int maxSubarraySumCircular(int[] nums) {
    int totalSum = 0;
    int currentMax = 0, globalMax = nums[0];
    int currentMin = 0, globalMin = nums[0];

    for (int num : nums) {
        totalSum += num;

        currentMax = Math.max(num, currentMax + num);
        globalMax = Math.max(globalMax, currentMax);

        currentMin = Math.min(num, currentMin + num);
        globalMin = Math.min(globalMin, currentMin);
    }

    // If all numbers negative, globalMin == totalSum, return globalMax
    if (globalMax < 0) return globalMax;

    // Max of: non-circular case, circular case
    return Math.max(globalMax, totalSum - globalMin);
}
```

## 4. Complexity Cheatsheet

| Variant           | Time | Space |
|-------------------|------|-------|
| Basic Kadane      | O(n) | O(1)  |
| With indices      | O(n) | O(1)  |
| Max product       | O(n) | O(1)  |
| Circular subarray | O(n) | O(1)  |

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                              | Difficulty | Why Important        |
|------|--------------------------------------|------------|----------------------|
| 53   | Maximum Subarray                     | Medium     | Pure Kadane's        |
| 918  | Maximum Sum Circular Subarray        | Medium     | Circular variant     |
| 152  | Maximum Product Subarray             | Medium     | Track min and max    |
| 1749 | Maximum Absolute Sum of Any Subarray | Medium     | Max + min subarrays  |
| 121  | Best Time to Buy and Sell Stock      | Easy       | Kadane's in disguise |

## Advanced Variations

| LC # | Problem                                      | Difficulty | Why Important            |
|------|----------------------------------------------|------------|--------------------------|
| 123  | Best Time to Buy and Sell Stock III          | Hard       | Two transactions         |
| 135  | Maximum Sum of Two Non-Overlapping Subarrays | Medium     | Two Kadane passes        |
| 2272 | Substring with Largest Variance              | Hard       | Kadane on character diff |
| 363  | Max Sum of Rectangle No Larger Than K        | Hard       | Kadane + BST             |
| 1191 | K-Concatenation Maximum Sum                  | Medium     | Kadane with repetition   |

## 6. Solve Step-by-Step — LC 53: Maximum Subarray

**Problem:** Find the contiguous subarray with the largest sum.

### Step 1 — Brute Force $O(n^2)$

```
int maxSum = Integer.MIN_VALUE;
for (int i = 0; i < n; i++) {
    int sum = 0;
    for (int j = i; j < n; j++) {
        sum += nums[j];
        maxSum = Math.max(maxSum, sum);
    }
}
```

### Step 2 — Divide and Conquer $O(n \log n)$

Split array in half, find max crossing the midpoint. Recursively solve.

### Step 3 — Optimal $O(n)$ — Kadane's

```
public int maxSubArray(int[] nums) {
    int currentMax = nums[0];
    int globalMax = nums[0];

    for (int i = 1; i < nums.length; i++) {
        currentMax = Math.max(nums[i], currentMax + nums[i]);
        globalMax = Math.max(globalMax, currentMax);
    }

    return globalMax;
}
```

**Dry Run (already shown above in intuition section)**

## Edge Cases

- Single element: `[-5] → -5` (initializing with `Integer.MIN_VALUE` would be wrong if you start loop at 0)
  - All negatives: `[-3, -1, -2] → -1` (largest single element)
  - All positive: sum of entire array
  - Mixed: classic sliding decision
- 

## 7. Pattern Variations

| Variation             | Key Change                                   |
|-----------------------|----------------------------------------------|
| Max sum subarray      | Basic Kadane                                 |
| Min sum subarray      | Flip max/min in Kadane                       |
| Max product subarray  | Track both max and min (negative × negative) |
| Circular max subarray | <code>total - min_subarray</code>            |
| At least k elements   | Track prefix sums with extra constraint      |
| Stock buy/sell        | Kadane on price differences                  |

---

## 8. Common Interview Mistakes

1. **Initializing** `currentMax = 0` — wrong when all numbers are negative; use `nums[0]`
  2. **Not handling single element case** — always initialize with `nums[0]`
  3. **Max product: forgetting to track minimum** — a large negative × negative = large positive
  4. **Circular: missing** `if (globalMax < 0) return globalMax` — infinite loop / wrong answer
  5. **Confusing** `currentMax` **with** `globalMax` — `currentMax` resets, `globalMax` never goes down
- 

## 9. Interview Cheat Sheet

## KADANE'S — MENTAL CHECKLIST

=====

- Maximum/minimum CONTIGUOUS subarray?
- All negative values possible? → init with nums[0], not 0
- Product instead of sum? → track min AND max
- Circular array? → max(normal, totalSum - minSubarray)
- Stock prices? → Kadane on differences arr[i]-arr[i-1]

## FORMULA

=====

```
currentMax = max(nums[i], currentMax + nums[i])
globalMax = max(globalMax, currentMax)
```

## TRICKS

=====

- Kadane = DP where dp[i] = max subarray ending at i
- Product variant: swap max/min when current num < 0
- "Best time to buy/sell" → convert to diff array → Kadane
- Circular: total\_sum - min\_subarray (complement trick)

---

## 10. Practice Roadmap

### Beginner:

- LC 53 — Maximum Subarray
- LC 121 — Best Time to Buy and Sell Stock

### Intermediate:

- LC 152 — Maximum Product Subarray
- LC 918 — Maximum Sum Circular Subarray
- LC 1749 — Maximum Absolute Sum of Any Subarray

### Hard:

- LC 123 — Best Time to Buy and Sell Stock III
- LC 363 — Max Sum of Rectangle No Larger Than K
- LC 2272 — Substring with Largest Variance

# Pattern 05 — Merge Intervals

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "overlapping intervals", "merge intervals"
- "meeting rooms", "calendar conflicts"
- "insert interval", "find gaps"
- "minimum number of platforms/rooms"
- "non-overlapping intervals"

**When to reach for Merge Intervals:**

- Array of [start, end] pairs
- Need to combine, count, or find conflicts between ranges
- Schedule or resource allocation problems

**Signal phrase:** \_"Given a list of intervals, merge overlapping ones"\_ → Sort + Merge

**Signal phrase:** \_"Minimum rooms for meetings"\_ → Event-based sweep or sort by start/end

## 2. Core Intuition

**Why brute force fails:**

Comparing every pair of intervals =  $O(n^2)$ . For 10<sup>4</sup> intervals → TLE.

**The key observation:**

After **sorting by start time**, overlapping intervals are always adjacent.

Two intervals [a, b] and [c, d] overlap if and only if  $c \leq b$  (next starts before current ends).

Before sort: [1,3], [2,6], [8,10], [15,18]

After sort by start (already sorted here):

[1,3] and [2,6]:  $2 \leq 3 \rightarrow \text{OVERLAP} \rightarrow \text{merge to } [1,6]$

[1,6] and [8,10]:  $8 > 6 \rightarrow \text{NO OVERLAP} \rightarrow \text{add new}$

[8,10] and [15,18]:  $15 > 10 \rightarrow \text{NO OVERLAP} \rightarrow \text{add new}$

Result: [[1,6], [8,10], [15,18]]

**Core invariant:**

At each step, the last interval in result either:

- Gets extended if current interval overlaps (update its end)
- Stays as-is and current interval is added as new

## 3. Generic Java Templates

### Template A — Merge Overlapping Intervals

```

public int[][] merge(int[][] intervals) {
    if (intervals.length <= 1) return intervals;

    // Sort by start time
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

    List<int[]> merged = new ArrayList<>();
    merged.add(intervals[0]); // add first interval

    for (int i = 1; i < intervals.length; i++) {
        int[] last = merged.get(merged.size() - 1);
        int[] curr = intervals[i];

        if (curr[0] <= last[1]) {
            // Overlap: extend end if needed
            last[1] = Math.max(last[1], curr[1]);
        } else {
            // No overlap: start new interval
            merged.add(curr);
        }
    }

    return merged.toArray(new int[merged.size()][]);
}

```

### Template B — Insert Interval (into sorted non-overlapping list)

```

public int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> result = new ArrayList<>();
    int i = 0;
    int n = intervals.length;

    // 1. Add all intervals that end before newInterval starts
    while (i < n && intervals[i][1] < newInterval[0]) {
        result.add(intervals[i++]);
    }

    // 2. Merge all overlapping intervals with newInterval
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
        i++;
    }
    result.add(newInterval);

    // 3. Add remaining intervals
    while (i < n) {
        result.add(intervals[i++]);
    }

    return result.toArray(new int[result.size()][]);
}

```

### Template C — Minimum Rooms / Platforms (Event Sweep)

```
// Meeting Rooms II: minimum rooms needed for all meetings
public int minMeetingRooms(int[][] intervals) {
    int n = intervals.length;
    int[] starts = new int[n];
    int[] ends = new int[n];

    for (int i = 0; i < n; i++) {
        starts[i] = intervals[i][0];
        ends[i] = intervals[i][1];
    }

    Arrays.sort(starts);
    Arrays.sort(ends);

    int rooms = 0;
    int maxRooms = 0;
    int endPtr = 0;

    for (int startPtr = 0; startPtr < n; startPtr++) {
        if (starts[startPtr] < ends[endPtr]) {
            rooms++; // new meeting starts before any ends → need new room
        } else {
            endPtr++; // one meeting ended, reuse its room
        }
        maxRooms = Math.max(maxRooms, rooms);
    }

    return maxRooms;
}
```

## Template D — Non-Overlapping Intervals (Greedy — Minimum Removals)

```
// Minimum intervals to remove to make rest non-overlapping
public int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[1] - b[1]); // sort by END time

    int count = 0; // intervals kept
    int lastEnd = Integer.MIN_VALUE;

    for (int[] interval : intervals) {
        if (interval[0] >= lastEnd) {
            // No overlap: keep this interval
            lastEnd = interval[1];
            count++;
        }
        // else: overlap → skip (remove) this interval
    }

    return intervals.length - count; // number removed
}
```

---

## 4. Complexity Cheatsheet

| Variant                     | Time          | Space  |
|-----------------------------|---------------|--------|
| Merge intervals             | $O(n \log n)$ | $O(n)$ |
| Insert interval             | $O(n)$        | $O(n)$ |
| Minimum rooms (event sweep) | $O(n \log n)$ | $O(n)$ |
| Non-overlapping (greedy)    | $O(n \log n)$ | $O(1)$ |

The sort dominates. After sorting, all variants are  $O(n)$ .

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                   | Difficulty | Why Important             |
|------|---------------------------|------------|---------------------------|
| 56   | Merge Intervals           | Medium     | Core template             |
| 57   | Insert Interval           | Medium     | Insert without re-sorting |
| 252  | Meeting Rooms             | Easy       | Basic overlap check       |
| 253  | Meeting Rooms II          | Medium     | Min rooms / sweep line    |
| 435  | Non-Overlapping Intervals | Medium     | Greedy removal            |

### Advanced Variations

| LC # | Problem                                    | Difficulty | Why Important                   |
|------|--------------------------------------------|------------|---------------------------------|
| 452  | Minimum Number of Arrows to Burst Balloons | Medium     | Greedy end-sorted               |
| 986  | Interval List Intersections                | Medium     | Two-pointer on two lists        |
| 1235 | Maximum Profit in Job Scheduling           | Hard       | DP + binary search on intervals |
| 759  | Employee Free Time                         | Hard       | Merge across multiple lists     |
| 732  | My Calendar III                            | Hard       | Sweep line with TreeMap         |

## 6. Solve Step-by-Step — LC 56: Merge Intervals

**Problem:** Merge all overlapping intervals.

### Step 1 — Brute Force $O(n^2)$

For each interval, check all others for overlap. Repeat until no merges possible.

### Step 2 — Sort + Linear Merge $O(n \log n)$

```

public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

    List<int[]> merged = new ArrayList<>();
    merged.add(intervals[0]);

    for (int i = 1; i < intervals.length; i++) {
        int[] last = merged.get(merged.size() - 1);
        int[] curr = intervals[i];

        if (curr[0] <= last[1]) {
            last[1] = Math.max(last[1], curr[1]); // extend
        } else {
            merged.add(curr);
        }
    }

    return merged.toArray(new int[merged.size()][]);
}

```

## Dry Run

Input: `[[1,3],[2,6],[8,10],[15,18]]`

After sort (already sorted): same

`merged = [[1,3]]`

`i=1: curr=[2,6], last=[1,3]`

`2 <= 3 → overlap → last[1] = max(3,6) = 6`

`merged = [[1,6]]`

`i=2: curr=[8,10], last=[1,6]`

`8 > 6 → no overlap → add`

`merged = [[1,6],[8,10]]`

`i=3: curr=[15,18], last=[8,10]`

`15 > 10 → no overlap → add`

`merged = [[1,6],[8,10],[15,18]]`

Output: `[[1,6],[8,10],[15,18]] ✓`

## Edge Cases

- Single interval → return as-is
- All overlapping: `[[1,4],[2,3]] → [[1,4]]` (contained interval)
- Touch at boundary: `[[1,4],[4,5]] → [[1,5]]` (`4 <= 4 → merge`)

## 7. Pattern Variations

| Variation       | Sort By                | Key Logic                |
|-----------------|------------------------|--------------------------|
| Merge intervals | start                  | extend end if overlap    |
| Minimum rooms   | start & end separately | sweep — count concurrent |

| Variation              | Sort By | Key Logic                   |
|------------------------|---------|-----------------------------|
| Remove minimum         | end     | keep greedy earliest-ending |
| Burst balloons         | end     | arrow covers until end      |
| Interval intersections | —       | two pointer on two lists    |

## 8. Common Interview Mistakes

- 1. Forgetting to sort** — without sorting, overlapping intervals aren't adjacent
- 2. Wrong overlap condition:** `curr[0] <= last[1]` not `<`; `last[1]` — touching intervals merge
- 3. Modifying last in result list:** since Java lists store references, `last[1] = ...` modifies in-place — that's correct and intended
- 4. Non-overlapping: sorting by start** — you must sort by END for the greedy to work
- 5. Assuming input is sorted** — LC 56 does NOT guarantee sorted input; always sort
- 6. Off-by-one in insert interval:** careful with `<`; `newInterval[0]` VS `<=` `newInterval[0]`

## 9. Interview Cheat Sheet

### MERGE INTERVALS – MENTAL CHECKLIST

=====

- Sort by START time (most cases)
- Sort by END time for greedy removal / arrow problems
- Overlap condition: `curr.start <= prev.end`
- Merge = extend `prev.end = max(prev.end, curr.end)`
- Rooms = max simultaneous overlapping at any point

### FORMULAS

=====

Overlap: `intervals[i][0] <= intervals[i-1][1]`

Merge: `last[1] = Math.max(last[1], curr[1])`

Rooms: sort starts/ends separately, sweep with two pointers

### TRICKS

=====

- "Meeting Rooms II" = sorting starts+ends separately (clever!)
- Greedy for non-overlapping: sort by end, greedily keep earliest-ending
- Interval intersection: while both lists active, move the one with smaller end
- TreeMap for dynamic calendar/stream problems

## 10. Practice Roadmap

### Beginner:

- LC 252 — Meeting Rooms
- LC 56 — Merge Intervals

### Intermediate:

- LC 57 — Insert Interval
- LC 253 — Meeting Rooms II
- LC 435 — Non-Overlapping Intervals
- LC 452 — Minimum Number of Arrows to Burst Balloons

**Hard:**

- LC 986 — Interval List Intersections
- LC 759 — Employee Free Time
- LC 1235 — Maximum Profit in Job Scheduling
- LC 732 — My Calendar III

# Pattern 06 — Tree Depth-First Search (Recursive)

---

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "height / depth of tree"
- "path sum", "root to leaf"
- "lowest common ancestor"
- "validate BST", "symmetric tree"
- "serialize / deserialize"
- "diameter", "maximum path sum"

**When to reach for Tree DFS:**

- Need to visit every node
- Problem involves path from root to leaf
- Need information from children to compute parent's value (post-order)
- Need to propagate information from root downward (pre-order)

**Signal phrase:** `"Find all paths from root to leaf with sum = k"` → DFS pre-order

**Signal phrase:** `"Height of tree"` → DFS post-order (need children first)

---

## 2. Core Intuition

**Three traversal orders — know when to use each:**

```
1
 / \
2   3
 / \
4   5
```

Pre-order (Root → Left → Right): 1, 2, 4, 5, 3 ← process before children

In-order (Left → Root → Right): 4, 2, 5, 1, 3 ← BST sorted order

Post-order (Left → Right → Root): 4, 5, 2, 3, 1 ← process after children

**When to use which:**

- **Pre-order:** copy tree, serialize, pass state DOWN (root → leaves)
- **In-order:** BST problems (gives sorted sequence)
- **Post-order:** compute height/size, delete tree, pass info UP (leaves → root)

**Core invariant:**

At each node, trust the recursion — your function returns the correct answer for any subtree. Just handle the current node + combine children's results.

---

## 3. Generic Java Templates

## TreeNode Definition

```
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int val) { this.val = val; }
}
```

## Template A — Pre-order (pass state down)

```
public void preorder(TreeNode node, int runningState) {
    if (node == null) return;

    // Process current node FIRST
    runningState += node.val; // example: accumulate path sum

    // Base case: leaf node
    if (node.left == null && node.right == null) {
        // check/store result
        return;
    }

    preorder(node.left, runningState);
    preorder(node.right, runningState);
}
```

## Template B — Post-order (return info up)

```
public int postorder(TreeNode node) {
    if (node == null) return 0; // base case

    int leftResult = postorder(node.left); // get from left child
    int rightResult = postorder(node.right); // get from right child

    // Combine children results + current node
    return 1 + Math.max(leftResult, rightResult); // example: height
}
```

## Template C — Global Max with Local Computation (Diameter / Max Path Sum)

```

private int globalMax = 0; // or Integer.MIN_VALUE for sum problems

public int solve(TreeNode root) {
    dfs(root);
    return globalMax;
}

private int dfs(TreeNode node) {
    if (node == null) return 0;

    int left = dfs(node.left);
    int right = dfs(node.right);

    // Update global answer (may "use" current node as root of path)
    globalMax = Math.max(globalMax, left + right + node.val);

    // Return to parent: best single path through this node
    return node.val + Math.max(left, right);
}

```

### Template D — In-order BST Validation / Operations

```

private Integer prev = null; // track previous value in in-order

public boolean isValidBST(TreeNode node) {
    if (node == null) return true;

    if (!isValidBST(node.left)) return false; // left subtree

    // In-order: process current
    if (prev != null && node.val <= prev) return false;
    prev = node.val;

    return isValidBST(node.right); // right subtree
}

```

### Template E — Return Multiple Values (avoid global with int[])

```

// Return [height, isDiameter] or use a custom class
private int[] dfs(TreeNode node) {
    // int[0] = height, int[1] = diameter through subtree
    if (node == null) return new int[]{0, 0};

    int[] left = dfs(node.left);
    int[] right = dfs(node.right);

    int height = 1 + Math.max(left[0], right[0]);
    int diameter = Math.max(left[0] + right[0], Math.max(left[1], right[1]));

    return new int[]{height, diameter};
}

```

---

## 4. Complexity Cheatsheet

| Operation            | Time        | Space (call stack)                |
|----------------------|-------------|-----------------------------------|
| Any full traversal   | $O(n)$      | $O(h)$ — $h$ = height             |
| Balanced tree height | $O(\log n)$ | $O(\log n)$                       |
| Skewed tree height   | $O(n)$      | $O(n)$ ← watch for stack overflow |
| BST search           | $O(h)$      | $O(h)$                            |

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                      | Difficulty | Why Important              |
|------|------------------------------|------------|----------------------------|
| 104  | Maximum Depth of Binary Tree | Easy       | Basic post-order template  |
| 112  | Path Sum                     | Easy       | Pre-order with running sum |
| 226  | Invert Binary Tree           | Easy       | Pre-order swap             |
| 543  | Diameter of Binary Tree      | Medium     | Global max + post-order    |
| 98   | Validate Binary Search Tree  | Medium     | In-order with bounds       |

### Advanced Variations

| LC # | Problem                               | Difficulty | Why Important              |
|------|---------------------------------------|------------|----------------------------|
| 124  | Binary Tree Maximum Path Sum          | Hard       | Global max with negatives  |
| 236  | Lowest Common Ancestor                | Medium     | Post-order return signal   |
| 297  | Serialize and Deserialize Binary Tree | Hard       | Pre-order + reconstruction |
| 105  | Construct Tree from Preorder+Inorder  | Medium     | Divide and conquer         |
| 1448 | Count Good Nodes in Binary Tree       | Medium     | Pre-order with max-so-far  |

## 6. Solve Step-by-Step — LC 124: Binary Tree Maximum Path Sum

**Problem:** Find the maximum path sum where path can start and end at any node.

### Key Insight

A path can go: left subtree → current node → right subtree (doesn't have to go through root).

At each node we decide: can including this node's subtree improve the path?

```

class Solution {
    private int maxSum = Integer.MIN_VALUE;

    public int maxPathSum(TreeNode root) {
        dfs(root);
        return maxSum;
    }

    // Returns: max path sum starting from this node going DOWN (one direction only)
    private int dfs(TreeNode node) {
        if (node == null) return 0;

        // Only take positive contributions from children
        int leftGain = Math.max(0, dfs(node.left));
        int rightGain = Math.max(0, dfs(node.right));

        // This node AS the "top" of the path: can use both children
        int pathThroughNode = node.val + leftGain + rightGain;
        maxSum = Math.max(maxSum, pathThroughNode);

        // Return to parent: can only extend in ONE direction
        return node.val + Math.max(leftGain, rightGain);
    }
}

```

## Dry Run

Tree: -10

```

/ \
9 20
/ \
15 7

```

```

dfs(9) → leftGain=0, rightGain=0, path=9, maxSum=9, return 9
dfs(15) → leftGain=0, rightGain=0, path=15, maxSum=15, return 15
dfs(7) → leftGain=0, rightGain=0, path=7, maxSum=15, return 7
dfs(20) → leftGain=15, rightGain=7, path=20+15+7=42, maxSum=42, return 20+15=35
dfs(-10) → leftGain=max(0,9)=9, rightGain=max(0,35)=35
          path=-10+9+35=34, maxSum=42, return -10+35=25

```

Answer: 42 ✓

## Edge Cases

- All negative: [-3, -1, -2] → -1 (single node, Math.max(0, child) handles this)
- Single node: return that node's value
- Path doesn't go through root at all

## 7. Pattern Variations

| Problem Type      | Order      | Key Trick                |
|-------------------|------------|--------------------------|
| Height, size, sum | Post-order | Combine children results |

| Problem Type               | Order            | Key Trick                       |
|----------------------------|------------------|---------------------------------|
| Path sum root→leaf         | Pre-order        | Pass running sum down           |
| Max path any node          | Post-order       | Global max + take max(0, child) |
| LCA                        | Post-order       | Return signal when found        |
| BST validate               | In-order         | Track previous node             |
| Level sum / zigzag         | BFS              | Use queue instead               |
| Build tree from traversals | Divide & conquer | Split by root index             |

## 8. Common Interview Mistakes

1. **Forgetting null check** at the top of every recursive call
2. **Not clamping negatives**: `Math.max(0, dfs(child))` — negative subtree hurts the path
3. **Confusing "return to parent" vs "update global"** — they're different values in Template C
4. **Stack overflow for skewed trees** — mention iterative DFS as follow-up
5. **In-order BST: using a global prev** — works but be careful in Java with `Integer prev = null` (not `int` )
6. **LCA: returning non-null from both children** is the signal — don't keep recursing

## 9. Interview Cheat Sheet

### TREE DFS — MENTAL CHECKLIST

=====

- What order? Pre/In/Post?
- Passing data DOWN? → Pre-order + parameter
- Collecting data UP? → Post-order + return value
- Global max? → Instance variable + post-order
- BST? → In-order, or use min/max bounds
- Null base case always first!

### ORDER CHEATSHEET

=====

Pre (R,L,R): top-down, copy, serialize, path from root  
 In (L,R,R): BST sorted order, validate, kth smallest  
 Post (L,R,R): height, size, LCA, max path sum, delete

### TRICKS

=====

- `max(0, child)` → don't take negative subtrees
- Return `int[]` or small class to avoid global variables
- LCA: if both left/right non-null → current node is LCA
- BST validate: pass (min, max) bounds down instead of tracking prev

## 10. Practice Roadmap

### Beginner:

- LC 104 — Max Depth
- LC 226 — Invert Binary Tree
- LC 112 — Path Sum
- LC 144/94/145 — Pre/In/Post-order traversal

**Intermediate:**

- LC 543 — Diameter of Binary Tree
- LC 98 — Validate BST
- LC 236 — Lowest Common Ancestor
- LC 1448 — Count Good Nodes
- LC 113 — Path Sum II (all paths)

**Hard:**

- LC 124 — Binary Tree Maximum Path Sum
- LC 297 — Serialize and Deserialize Binary Tree
- LC 105 — Construct from Preorder + Inorder
- LC 968 — Binary Tree Cameras

# Pattern 07 — Tree Breadth-First Search (BFS / Level Order)

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "level order traversal"
- "minimum depth", "shortest path in tree"
- "right side view", "left side view"
- "average of each level", "sum per level"
- "zigzag traversal"
- "connect next right pointers"

**When to reach for Tree BFS:**

- Process nodes level by level
- Find minimum depth (DFS finds any path; BFS finds shortest)
- Need nodes at the same depth grouped together
- Right/left view problems (last/first node per level)

**Signal phrase:** `"Level order"` → BFS immediately

**Signal phrase:** `"Minimum depth"` → BFS (DFS would be inefficient — finds deepest first)

## 2. Core Intuition

**Why BFS for levels?**

DFS goes deep before wide — it can't naturally group nodes by level.

BFS uses a queue: process all nodes at distance  $d$  before any at distance  $d+1$ .

```
1 ← Level 0
/ \
2 3 ← Level 1
/ \ \
4 5 6 ← Level 2
```

Queue progression:

Start: [1]

Pop 1, push 2,3 → [2,3] level 0 done: [1]

Pop 2, push 4,5 → [3,4,5]

Pop 3, push 6 → [4,5,6] level 1 done: [2,3]

Pop 4,5,6 (no children) level 2 done: [4,5,6]

**Core invariant:**

At the start of each outer loop iteration, the queue contains **exactly all nodes at the current level**.

Process them all, adding their children for the next level.

### 3. Generic Java Templates

#### Template A — Level Order (Groups by Level)

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int levelSize = queue.size(); // snapshot: nodes at this level
        List<Integer> level = new ArrayList<>();

        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);

            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }

        result.add(level);
    }

    return result;
}
```

#### Template B — Right Side View (Last Node Per Level)

```
public List<Integer> rightSideView(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int levelSize = queue.size();

        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();

            // Last node in the level = rightmost visible
            if (i == levelSize - 1) result.add(node.val);

            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
    }

    return result;
}
```

### Template C — Minimum Depth

```
public int minDepth(TreeNode root) {
    if (root == null) return 0;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    int depth = 1;

    while (!queue.isEmpty()) {
        int levelSize = queue.size();

        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();

            // First leaf found = minimum depth (BFS guarantee)
            if (node.left == null && node.right == null) return depth;

            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }

        depth++;
    }

    return depth;
}
```

### Template D — Zigzag Level Order

```

public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    boolean leftToRight = true;

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        LinkedList<Integer> level = new LinkedList<>(); // LinkedList for addFirst

        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();

            if (leftToRight) level.addLast(node.val);
            else level.addFirst(node.val); // reverse order

            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }

        result.add(level);
        leftToRight = !leftToRight;
    }

    return result;
}

```

## 4. Complexity Cheatsheet

| Operation                      | Time   | Space                    |
|--------------------------------|--------|--------------------------|
| Level order traversal          | $O(n)$ | $O(w)$ — $w$ = max width |
| Complete binary tree max width | $O(n)$ | $O(n/2) \approx O(n)$    |
| Skewed tree max width          | $O(n)$ | $O(1)$                   |

BFS space = width of widest level (worst case: last level of complete tree =  $n/2$  nodes).

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                              | Difficulty | Why Important       |
|------|--------------------------------------|------------|---------------------|
| 102  | Binary Tree Level Order Traversal    | Medium     | Core template       |
| 107  | Level Order Traversal II (bottom-up) | Medium     | Reverse result      |
| 199  | Binary Tree Right Side View          | Medium     | Last node per level |

| LC # | Problem                          | Difficulty | Why Important          |
|------|----------------------------------|------------|------------------------|
| 111  | Minimum Depth of Binary Tree     | Easy       | BFS vs DFS distinction |
| 637  | Average of Levels in Binary Tree | Easy       | Aggregate per level    |

## Advanced Variations

| LC # | Problem                             | Difficulty | Why Important             |
|------|-------------------------------------|------------|---------------------------|
| 103  | Binary Tree Zigzag Level Order      | Medium     | Direction toggle          |
| 116  | Populating Next Right Pointers      | Medium     | BFS without extra space   |
| 515  | Find Largest Value in Each Row      | Medium     | Max per level             |
| 1161 | Maximum Level Sum of a Binary Tree  | Medium     | Sum per level             |
| 958  | Check Completeness of a Binary Tree | Medium     | Null-gap detection in BFS |

## 6. Solve Step-by-Step — LC 199: Binary Tree Right Side View

**Problem:** Return the values visible from the right side of the tree (one per level).

### Intuition

Standing to the right, you see the last node at each level. BFS naturally processes level by level — take the last node of each iteration.

```

public List<Integer> rightSideView(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        TreeNode lastNode = null;

        for (int i = 0; i < levelSize; i++) {
            lastNode = queue.poll();
            if (lastNode.left != null) queue.offer(lastNode.left);
            if (lastNode.right != null) queue.offer(lastNode.right);
        }

        result.add(lastNode.val); // last node processed at this level
    }

    return result;
}

```

## Dry Run

Tree: 1

```

/ \
2 3
\ \
5 4

```

Level 0: queue=[1], process 1, push 2,3 → last=1 → result=[1]

Level 1: queue=[2,3], process 2(push 5), 3(push 4) → last=3 → result=[1,3]

Level 2: queue=[5,4], process 5, 4 → last=4 → result=[1,3,4]

## Edge Cases

- Left-skewed tree: right view = all left-side nodes (only one per level)
- Single node: [root.val]
- Root with only left child: both root and left child are visible

## 7. Pattern Variations

| Variation        | Change from Template A         |
|------------------|--------------------------------|
| Bottom-up levels | Reverse result list at end     |
| Right side view  | Keep only last node per level  |
| Left side view   | Keep only first node per level |
| Level averages   | Sum / levelSize                |
| Zigzag           | Toggle addFirst / addLast      |
| Max per level    | Track max during inner loop    |

| Variation          | Change from Template A      |
|--------------------|-----------------------------|
| Check completeness | Track null-seen flag in BFS |

## 8. Common Interview Mistakes

- 1. Forgetting** `levelSize = queue.size()` **snapshot** — queue grows during inner loop, processing would continue into next level
- 2. Using stack instead of queue** — stack gives DFS order, not BFS
- 3. Not handling null root** — always check at the start
- 4. Minimum depth pitfall:** DFS returns depth of first leaf found, not shallowest. BFS is correct here.
- 5. Zigzag: using `Collections.reverse()`** —  $O(n)$  extra per level; `LinkedList.addFirst()` is cleaner and  $O(1)$

## 9. Interview Cheat Sheet

### TREE BFS — MENTAL CHECKLIST

=====

- Level-by-level processing? → BFS with queue
- Minimum depth / shortest path? → BFS (not DFS!)
- Right/Left view? → last/first node per level
- Always snapshot: `levelSize = queue.size()` before inner loop
- Offer children: only if not null

### STRUCTURE

=====

```
Queue<TreeNode> queue = new LinkedList<>();
queue.offer(root);
while (!queue.isEmpty()) {
    int size = queue.size(); ← SNAPSHOT
    for (int i = 0; i < size; i++) {
        TreeNode node = queue.poll();
        // process node
        if (node.left != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }
    // level complete
}
```

### TRICKS

=====

- Right view = last node; Left view = first node ( $i==0$ )
- Zigzag: use `LinkedList`, toggle `addFirst/addLast`
- Bottom-up: `Collections.reverse(result)` at the end
- Check completeness: after first null in BFS, no more non-nulls allowed

## 10. Practice Roadmap

**Beginner:**

- LC 102 — Level Order Traversal
- LC 111 — Minimum Depth
- LC 637 — Average of Levels

**Intermediate:**

- LC 199 — Right Side View
- LC 103 — Zigzag Level Order
- LC 515 — Find Largest Value in Each Row
- LC 958 — Check Completeness

**Hard:**

- LC 116 — Populating Next Right Pointers ( $O(1)$  space variant)
- LC 117 — Populating Next Right Pointers II
- LC 1161 — Maximum Level Sum

# Pattern 08 — Tree DFS Iterative

## 1. Pattern Recognition Guide

**When to use Iterative DFS over Recursive:**

- Interviewer asks "can you do this without recursion?"
- Tree is highly skewed ( $n=10^4$  depth) → stack overflow risk
- Need explicit control over the call stack
- In-order iterative is common in BST iterator problems

**Same problems as recursive DFS** — iterative is an implementation choice, not a different pattern.

**Signal phrase:** \_"Implement BST iterator"\_ → In-order iterative with explicit stack

**Signal phrase:** \_"Without recursion"\_ → Iterative DFS

## 2. Core Intuition

**Recursion = implicit call stack. Iterative = explicit stack (or queue for BFS).**

For DFS, you simulate what the call stack does:

- Push node
- Pop and process
- Push children (right first so left is processed first — stack is LIFO)

Pre-order iterative logic:

Push root → pop → process → push right then left → pop left → ...

Stack: [1]

Pop 1, process 1, push 3, push 2 → Stack: [3,2]

Pop 2, process 2, push 5, push 4 → Stack: [3,5,4]

Pop 4, process 4 → Stack: [3,5]

Pop 5, process 5 → Stack: [3]

Pop 3, process 3 → Stack: []

Result: 1,2,4,5,3 ✓ (pre-order)

**In-order is trickier** — you must go all the way left first, then process, then go right.

## 3. Generic Java Templates

### Template A — Pre-order Iterative

```

public List<Integer> preorderTraversal(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Deque<TreeNode> stack = new ArrayDeque<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        TreeNode node = stack.pop();
        result.add(node.val); // process BEFORE pushing children

        // Push right first so left is processed first (LIFO)
        if (node.right != null) stack.push(node.right);
        if (node.left != null) stack.push(node.left);
    }

    return result;
}

```

### Template B — In-order Iterative (Most Important)

```

public List<Integer> inorderTraversal(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;

    while (curr != null || !stack.isEmpty()) {
        // Go as far left as possible
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }

        // Process node (leftmost unprocessed)
        curr = stack.pop();
        result.add(curr.val);

        // Move to right subtree
        curr = curr.right;
    }

    return result;
}

```

### Template C — Post-order Iterative (Two-Stack Trick)

```

public List<Integer> postorderTraversal(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Deque<TreeNode> stack = new ArrayDeque<>();
    stack.push(root);

    // Trick: do "reverse pre-order" (Root → Right → Left), then reverse result
    while (!stack.isEmpty()) {
        TreeNode node = stack.pop();
        result.add(node.val); // add to result

        if (node.left != null) stack.push(node.left); // push left first
        if (node.right != null) stack.push(node.right); // right processed next
    }

    Collections.reverse(result); // reverse to get Left → Right → Root
    return result;
}

```

### Template D — BST Iterator (In-order, Lazy)

// LC 173: Implement BST iterator – next() returns in-order next, hasNext() check

```

class BSTIterator {
    private Deque<TreeNode> stack = new ArrayDeque<>();

    public BSTIterator(TreeNode root) {
        pushLeft(root); // push all leftmost nodes
    }

    public int next() {
        TreeNode node = stack.pop();
        pushLeft(node.right); // push left spine of right subtree
        return node.val;
    }

    public boolean hasNext() {
        return !stack.isEmpty();
    }

    private void pushLeft(TreeNode node) {
        while (node != null) {
            stack.push(node);
            node = node.left;
        }
    }
}

```

### Template E — General DFS with State (Iterative Path Sum)

```

public boolean hasPathSum(TreeNode root, int targetSum) {
    if (root == null) return false;

    // Stack stores [node, remainingSum]
    Deque<Object[]> stack = new ArrayDeque<>();
    stack.push(new Object[]{root, targetSum});

    while (!stack.isEmpty()) {
        Object[] top = stack.pop();
        TreeNode node = (TreeNode) top[0];
        int remaining = (int) top[1];
        remaining -= node.val;

        if (node.left == null && node.right == null && remaining == 0) return true;

        if (node.right != null) stack.push(new Object[]{node.right, remaining});
        if (node.left != null) stack.push(new Object[]{node.left, remaining});
    }

    return false;
}

```

## 4. Complexity Cheatsheet

| Traversal            | Time                    | Space  |
|----------------------|-------------------------|--------|
| Pre-order iterative  | $O(n)$                  | $O(h)$ |
| In-order iterative   | $O(n)$                  | $O(h)$ |
| Post-order iterative | $O(n)$                  | $O(h)$ |
| BST Iterator next()  | $O(h)$ amortized $O(1)$ | $O(h)$ |

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                         | Difficulty | Why Important           |
|------|---------------------------------|------------|-------------------------|
| 144  | Binary Tree Preorder Traversal  | Easy       | Template A              |
| 94   | Binary Tree Inorder Traversal   | Easy       | Template B (critical)   |
| 145  | Binary Tree Postorder Traversal | Easy       | Template C              |
| 173  | Binary Search Tree Iterator     | Medium     | Template D (BST + lazy) |
| 230  | Kth Smallest in BST             | Medium     | In-order iterative      |

### Advanced Variations

| LC # | Problem                            | Difficulty | Why Important                |
|------|------------------------------------|------------|------------------------------|
| 285  | Inorder Successor in BST           | Medium     | Modified in-order            |
| 426  | Convert BST to Sorted Doubly LL    | Medium     | In-order with prev pointer   |
| 114  | Flatten Binary Tree to Linked List | Medium     | Pre-order iterative          |
| 1028 | Recover a Tree From Preorder       | Hard       | Stack + depth tracking       |
| 255  | Verify Preorder Sequence in BST    | Medium     | Monotonic stack on traversal |

## 6. Solve Step-by-Step — LC 173: BST Iterator

**Problem:** Implement `next()` and `hasNext()` with  $O(h)$  space.

### Why This Is Clever

We don't traverse all at once. Instead, we lazily maintain a stack of "pending" nodes — always the left spine of the remaining tree.

```
class BSTIterator {
    private Deque<TreeNode> stack = new ArrayDeque<>();

    public BSTIterator(TreeNode root) {
        pushLeft(root);
    }

    public int next() {
        TreeNode node = stack.pop(); // smallest remaining
        pushLeft(node.right); // prepare right subtree's in-order
        return node.val;
    }

    public boolean hasNext() {
        return !stack.isEmpty();
    }

    private void pushLeft(TreeNode node) {
        while (node != null) {
            stack.push(node);
            node = node.left;
        }
    }
}
```

### Dry Run

```

BST: 7
  / \
 3  15
 / \
9  20

```

Init: pushLeft(7) → stack: [7, 3] (3 on top)

```

next() → pop 3, pushLeft(3.right=null) → stack: [7], return 3
next() → pop 7, pushLeft(7.right=15) → pushLeft pushes 15,9 → stack: [15,9], return 7
next() → pop 9, pushLeft(null) → stack: [15], return 9
next() → pop 15, pushLeft(20) → stack: [20], return 15
next() → pop 20, pushLeft(null) → stack: [], return 20
hasNext() → false

```

## Amortized Analysis

Each node is pushed and popped exactly once across all next() calls → O(n) total → O(1) amortized per call. Space = O(h) for left spine.

## 7. Pattern Variations

| Variation        | Key Difference                       |
|------------------|--------------------------------------|
| Pre-order        | Push right first, then left          |
| In-order         | Explicit "go left" loop + stack      |
| Post-order       | Reverse of modified pre-order        |
| BST Iterator     | Lazy in-order with pushLeft helper   |
| Path tracking    | Push (node, state) pairs onto stack  |
| Morris Traversal | O(1) space — modify tree temporarily |

## 8. Common Interview Mistakes

- 1. Pre-order: push left first** — wrong! Push right first so left is popped first (LIFO)
- 2. In-order: checking** stack.isEmpty() **only** — also need curr != null in while condition
- 3. Post-order two-stack:** forgetting to reverse at the end
- 4. BST Iterator: not calling** pushLeft(node.right) after popping — misses right subtree
- 5. Using** stack.push **vs** stack.add — Stick to ArrayDeque with push/pop for stack semantics

## 9. Interview Cheat Sheet

## ITERATIVE DFS — MENTAL CHECKLIST

=====

- Pre-order? → push root, loop: pop→process→push right→push left
- In-order? → curr pointer + stack: go left, pop+process, go right
- Post-order? → reverse-pre-order trick, then reverse result
- BST Iterator? → pushLeft helper, lazy evaluation

## IN-ORDER PATTERN (MEMORIZE)

=====

```
curr = root
while (curr != null || !stack.isEmpty()):
    while (curr != null): stack.push(curr); curr = curr.left
    curr = stack.pop()
    process(curr)
    curr = curr.right
```

## TRICKS

=====

- Always use ArrayDeque, never Stack<> class (synchronized, slow)
- Post-order = reverse of (Root → Right → Left) traversal
- BST Iterator amortized  $O(1)$ : each node pushed/popped once total
- Morris Traversal:  $O(1)$  space, modifies tree temporarily (mention as bonus)

---

## 10. Practice Roadmap

### Beginner:

- LC 144 — Preorder Traversal
- LC 94 — Inorder Traversal
- LC 145 — Postorder Traversal

### Intermediate:

- LC 173 — BST Iterator
- LC 230 — Kth Smallest in BST
- LC 114 — Flatten Binary Tree to Linked List

### Hard:

- LC 1028 — Recover Tree From Preorder
- LC 426 — Convert BST to Sorted Doubly Linked List
- LC 255 — Verify Preorder Sequence in BST

# Pattern 09 — Fast and Slow Pointers (Floyd's Algorithm)

---

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "detect cycle in linked list"
- "find middle of linked list"
- "find start of cycle"
- "palindrome linked list"
- "nth node from end" (also two pointers, different gap)
- "happy number" (implied cycle in sequence)

**When to reach for Fast/Slow:**

- Cycle detection in any sequence (linked list, array, number sequences)
- Finding the middle of a linked list in one pass
- Problems where you need a pointer at 1/2 speed

**Signal phrase:** `"Does this linked list have a cycle?"` → Fast/Slow immediately

**Signal phrase:** `"Find the middle"` → Slow stops at middle when fast reaches end

---

## 2. Core Intuition

**The key observation:**

If two pointers move at different speeds in a circular loop, the faster one **MUST** eventually lap the slower one (like runners on a circular track).

**Why it works:**

- No cycle: fast pointer reaches null, no meeting
- With cycle: fast enters cycle, slow enters cycle. Relative speed = 1 step/iteration. Eventually fast catches slow.

**Finding the middle:**

When fast (2 steps) reaches end, slow (1 step) is at the middle.

List: 1 → 2 → 3 → 4 → 5 → null

Slow: 1 → 2 → 3

Fast: 1 → 3 → 5 → null

When fast hits null or `fast.next` is null → slow is at middle (3)

**Cycle detection:**

1 → 2 → 3 → 4 → 5  
↑ ↓  
8 ← 6 ← (cycle)

Slow and fast will meet inside the cycle. Then:  
Move one pointer to head, keep other at meeting point.  
Advance both one step at a time → they meet at cycle start!

---

### 3. Generic Java Templates

#### ListNode Definition

```
public class ListNode {  
    int val;  
    ListNode next;  
    ListNode(int val) { this.val = val; }  
}
```

#### Template A — Detect Cycle

```
public boolean hasCycle(ListNode head) {  
    ListNode slow = head;  
    ListNode fast = head;  
  
    while (fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
  
        if (slow == fast) return true; // they met → cycle exists  
    }  
  
    return false; // fast hit null → no cycle  
}
```

#### Template B — Find Start of Cycle

```

public ListNode detectCycle(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    // Phase 1: Detect meeting point
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) break;
    }

    // No cycle
    if (fast == null || fast.next == null) return null;

    // Phase 2: Find cycle start
    // Move one pointer to head, advance both one step at a time
    slow = head;
    while (slow != fast) {
        slow = slow.next;
        fast = fast.next;
    }

    return slow; // cycle start
}

```

### Template C — Find Middle of Linked List

```

public ListNode findMiddle(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    // When fast reaches end, slow is at middle
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    return slow; // middle node
    // For even length: returns second middle (e.g., [1,2,3,4] → 3)
    // Use fast.next != null only to get first middle → 2
}

```

### Template D — Nth From End (Two Pointers with Gap)

```

public ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;

    ListNode fast = dummy;
    ListNode slow = dummy;

    // Advance fast by n+1 steps (creates gap of n)
    for (int i = 0; i <= n; i++) {
        fast = fast.next;
    }

    // Move both until fast hits null
    while (fast != null) {
        slow = slow.next;
        fast = fast.next;
    }

    // slow.next is the nth from end – remove it
    slow.next = slow.next.next;

    return dummy.next;
}

```

## Template E — Palindrome Linked List

```

public boolean isPalindrome(ListNode head) {
    // Step 1: Find middle
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    // Step 2: Reverse second half
    ListNode prev = null, curr = slow;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }

    // Step 3: Compare first half and reversed second half
    ListNode left = head, right = prev;
    while (right != null) {
        if (left.val != right.val) return false;
        left = left.next;
        right = right.next;
    }

    return true;
}

```

## 4. Complexity Cheatsheet

| Problem          | Time   | Space  |
|------------------|--------|--------|
| Detect cycle     | $O(n)$ | $O(1)$ |
| Find cycle start | $O(n)$ | $O(1)$ |
| Find middle      | $O(n)$ | $O(1)$ |
| Nth from end     | $O(n)$ | $O(1)$ |
| Palindrome check | $O(n)$ | $O(1)$ |

All  $O(1)$  space — that's the beauty of fast/slow pointers.

---

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                   | Difficulty | Why Important              |
|------|---------------------------|------------|----------------------------|
| 141  | Linked List Cycle         | Easy       | Core template              |
| 142  | Linked List Cycle II      | Medium     | Find cycle start           |
| 876  | Middle of the Linked List | Easy       | Middle template            |
| 234  | Palindrome Linked List    | Easy       | Middle + reverse + compare |
| 19   | Remove Nth Node From End  | Medium     | Gap technique              |

### Advanced Variations

| LC # | Problem                          | Difficulty | Why Important            |
|------|----------------------------------|------------|--------------------------|
| 202  | Happy Number                     | Easy       | Cycle in number sequence |
| 287  | Find the Duplicate Number        | Medium     | Array as linked list     |
| 457  | Circular Array Loop              | Medium     | Cycle in circular array  |
| 160  | Intersection of Two Linked Lists | Easy       | Equalize lengths trick   |
| 61   | Rotate List                      | Medium     | Find tail + reconnect    |

---

## 6. Solve Step-by-Step — LC 142: Linked List Cycle II

**Problem:** Find the node where the cycle begins.

### The Math (Why Phase 2 Works)

Let:

- $F$  = distance from head to cycle start

- $C$  = cycle length
- $K$  = distance from cycle start to meeting point

When slow and fast meet:

- slow traveled:  $F + K$
- fast traveled:  $F + K + C$  (one full cycle extra)
- $\text{fast} = 2 \times \text{slow} \rightarrow F + K + C = 2(F + K) \rightarrow C = F + K \rightarrow \mathbf{F = C - K}$

So distance from head to cycle start ( $F$ ) = distance from meeting point to cycle start ( $C - K$ ).

Moving one pointer from head, one from meeting point, both one step  $\rightarrow$  they meet at cycle start!

```
public ListNode detectCycle(ListNode head) {
    ListNode slow = head, fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
            // Phase 2: find cycle start
            slow = head;
            while (slow != fast) {
                slow = slow.next;
                fast = fast.next;
            }
            return slow;
        }
    }
    return null;
}
```

## Dry Run

List:  $3 \rightarrow 1 \rightarrow 2 \rightarrow 0 \rightarrow (\text{back to } 2)$

Indices: 0 1 2 3

Phase 1:

Start:  $\text{slow}=3, \text{fast}=3$

Step1:  $\text{slow}=1, \text{fast}=2$

Step2:  $\text{slow}=2, \text{fast}=0$

Step3:  $\text{slow}=0, \text{fast}=0 \leftarrow \text{MEET at index 3}$

Phase 2:  $\text{slow}=\text{head}(3), \text{fast}=0(\text{index } 3)$

Step1:  $\text{slow}=1, \text{fast}=2 (\text{index } 2)$

Step2:  $\text{slow}=2, \text{fast}=2 \leftarrow \text{MEET at index 2} = \text{cycle start} \checkmark$

## 7. Pattern Variations

| Problem         | Variation                                     |
|-----------------|-----------------------------------------------|
| Cycle detection | fast/slow, meet = cycle exists                |
| Cycle start     | Phase 2: head pointer + meeting point         |
| Middle          | fast reaches end $\rightarrow$ slow at middle |

| Problem        | Variation                                   |
|----------------|---------------------------------------------|
| Nth from end   | Gap of n between fast and slow              |
| Palindrome     | Middle + reverse second half                |
| Happy number   | Treat sequence as linked list, detect cycle |
| Find duplicate | Array indices as next pointers (LC 287)     |

## 8. Common Interview Mistakes

1. `fast.next != null` **check missing** — `fast.next.next` will NPE without it
2. **Meeting check inside loop vs. before advancing** — check AFTER advancing both
3. **Cycle start: moving slow to head but keeping fast at meeting point** — correct; many forget to do this
4. **Middle for even length:** `[1, 2, 3, 4]` → returns 3 (second middle). If you need first middle, use `fast.next != null` instead of `fast != null && fast.next != null`
5. **Palindrome: not restoring the list** — some interviewers expect you to restore the reversed half

## 9. Interview Cheat Sheet

### FAST/SLOW POINTERS – MENTAL CHECKLIST

=====

- Cycle detection? → fast 2 steps, slow 1 step; meet = cycle
- Cycle start? → Phase 2: reset slow to head, both move 1 step
- Middle? → fast reaches null → slow is at middle
- Nth from end? → gap of n between fast and slow using dummy node
- Always check: `fast != null && fast.next != null`

### TEMPLATES

=====

Cycle detect: `while(fast!=null && fast.next!=null) { slow++; fast+=2; if(==) cycle }`

Middle: same loop, when exits: `slow = middle`

Nth from end: fast leads by `n+1`, then move together

### TRICKS

=====

- Dummy node for nth-from-end removes edge case (removing head)
- Happy number: model as linked list, fast/slow on `next()` function
- LC 287 (find duplicate): treat array as linked list, indices as values
- Palindrome = find middle + reverse second half + compare

## 10. Practice Roadmap

### Beginner:

- LC 141 — Linked List Cycle
- LC 876 — Middle of Linked List
- LC 202 — Happy Number

**Intermediate:**

- LC 142 — Linked List Cycle II
- LC 19 — Remove Nth from End
- LC 234 — Palindrome Linked List
- LC 160 — Intersection of Two Lists

**Hard:**

- LC 287 — Find the Duplicate Number
- LC 457 — Circular Array Loop
- LC 61 — Rotate List

# Pattern 10 — In-Place Linked List Reversal + Dummy Nodes

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "reverse a linked list"
- "reverse between positions i and j"
- "reverse in groups of k"
- "rotate linked list"
- "reorder list"

**Dummy Node triggers:**

- "delete node", "merge lists", "add node at head"
- Whenever the head itself might change
- Result list built incrementally

**Signal phrase:** `"Reverse the linked list in place"` → Three-pointer reversal

**Signal phrase:** `"Reverse nodes in k-group"` → Reversal with counting + reconnection

## 2. Core Intuition

**Why you need three pointers:**

You can't reverse a link without knowing where to go back. You need:

- `prev` — the node that `curr` should now point to
- `curr` — the node being processed
- `next` — saved before overwriting `curr.next`

Before: `prev=null, curr=1→2→3→null`

Step 1: save `next=2`, `curr.next=prev(null)`, `prev=curr(1)`, `curr=next(2)`  
`null ← 1 2→3→null`

Step 2: save `next=3`, `curr.next=prev(1)`, `prev=curr(2)`, `curr=next(3)`  
`null ← 1 ← 2 3→null`

Step 3: save `next=null`, `curr.next=prev(2)`, `prev=curr(3)`, `curr=next(null)`  
`null ← 1 ← 2 ← 3`

Loop ends (`curr=null`), return `prev (3) = new head`

**Dummy Node intuition:**

When the head might change (deleting head, inserting before head, merging lists), add a fake node before head. You always return `dummy.next` at the end — no special-case logic.

## 3. Generic Java Templates

### Template A — Full List Reversal

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;

    while (curr != null) {
        ListNode next = curr.next; // save next
        curr.next = prev; // reverse the link
        prev = curr; // advance prev
        curr = next; // advance curr
    }

    return prev; // new head
}
```

### Template B — Reverse Between Positions (1-indexed)

```
public ListNode reverseBetween(ListNode head, int left, int right) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode prev = dummy;

    // Move prev to node just before position 'left'
    for (int i = 1; i < left; i++) {
        prev = prev.next;
    }

    ListNode curr = prev.next; // first node to reverse
    ListNode next = null;

    // Reverse (right - left) times
    for (int i = 0; i < right - left; i++) {
        next = curr.next;
        curr.next = next.next;
        next.next = prev.next;
        prev.next = next;
    }

    return dummy.next;
}
```

### Template C — Reverse in K-Groups

```

public ListNode reverseKGroup(ListNode head, int k) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode groupPrev = dummy;

    while (true) {
        // Check if k nodes remain
        ListNode kth = getKth(groupPrev, k);
        if (kth == null) break;

        ListNode groupNext = kth.next;

        // Reverse the group
        ListNode prev = groupNext;
        ListNode curr = groupPrev.next;
        while (curr != groupNext) {
            ListNode next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }

        // Reconnect: groupPrev.next was group start, now points to old tail
        ListNode tmp = groupPrev.next;
        groupPrev.next = kth;
        groupPrev = tmp; // advance groupPrev to end of reversed group
    }

    return dummy.next;
}

private ListNode getKth(ListNode curr, int k) {
    while (curr != null && k > 0) {
        curr = curr.next;
        k--;
    }
    return curr;
}

```

## Template D — Dummy Node Pattern (Merge Two Sorted Lists)

```

public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0); // sentinel head
    ListNode curr = dummy;

    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) {
            curr.next = l1;
            l1 = l1.next;
        } else {
            curr.next = l2;
            l2 = l2.next;
        }
        curr = curr.next;
    }

    // Attach remaining
    curr.next = (l1 != null) ? l1 : l2;

    return dummy.next; // skip sentinel
}

```

### Template E — Reorder List (Mid + Reverse + Merge)

```

public void reorderList(ListNode head) {
    if (head == null || head.next == null) return;

    // Step 1: Find middle
    ListNode slow = head, fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    // Step 2: Reverse second half
    ListNode second = reverseList(slow.next);
    slow.next = null; // cut first half

    // Step 3: Merge alternating
    ListNode first = head;
    while (second != null) {
        ListNode tmp1 = first.next;
        ListNode tmp2 = second.next;
        first.next = second;
        second.next = tmp1;
        first = tmp1;
        second = tmp2;
    }
}

```

## 4. Complexity Cheatsheet

| Problem       | Time | Space |
|---------------|------|-------|
| Full reversal | O(n) | O(1)  |

| Problem                | Time   | Space |
|------------------------|--------|-------|
| Reverse between i,j    | O(n)   | O(1)  |
| Reverse k-groups       | O(n)   | O(1)  |
| Merge two sorted lists | O(n+m) | O(1)  |
| Reorder list           | O(n)   | O(1)  |

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                  | Difficulty | Why Important         |
|------|--------------------------|------------|-----------------------|
| 206  | Reverse Linked List      | Easy       | Core template         |
| 92   | Reverse Linked List II   | Medium     | Partial reversal      |
| 21   | Merge Two Sorted Lists   | Easy       | Dummy node template   |
| 143  | Reorder List             | Medium     | Mid + Reverse + Merge |
| 25   | Reverse Nodes in k-Group | Hard       | Group reversal        |

### Advanced Variations

| LC # | Problem                               | Difficulty | Why Important      |
|------|---------------------------------------|------------|--------------------|
| 2    | Add Two Numbers                       | Medium     | Dummy node + carry |
| 82   | Remove Duplicates from Sorted List II | Medium     | Dummy + skip       |
| 86   | Partition List                        | Medium     | Two dummy lists    |
| 148  | Sort List                             | Medium     | Merge sort on list |
| 23   | Merge K Sorted Lists                  | Hard       | Heap + dummy       |

## 6. Solve Step-by-Step — LC 92: Reverse Linked List II

**Problem:** Reverse nodes from position `left` to `right` (1-indexed).

### Approach: "Head Insertion" Trick

Instead of traditional 3-pointer reversal (which requires tracking 4 pointers across segment boundaries), use head insertion:

- Keep `prev` at position just before `left`
- Repeatedly take `curr.next` and insert it right after `prev`

```

public ListNode reverseBetween(ListNode head, int left, int right) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode prev = dummy;

    // Reach position before 'left'
    for (int i = 1; i < left; i++) prev = prev.next;

    ListNode curr = prev.next;

    for (int i = 0; i < right - left; i++) {
        ListNode next = curr.next; // node to insert at front
        curr.next = next.next; // skip next
        next.next = prev.next; // next points to current front
        prev.next = next; // prev points to next (new front)
    }

    return dummy.next;
}

```

### Dry Run: [1,2,3,4,5] , left=2, right=4

dummy → 1 → 2 → 3 → 4 → 5  
 prev stops at 1 (position before left=2)  
 curr = 2

i=0: next=3; curr(2).next=4; next(3).next=2; prev(1).next=3

dummy → 1 → 3 → 2 → 4 → 5

i=1: next=4; curr(2).next=5; next(4).next=3; prev(1).next=4

dummy → 1 → 4 → 3 → 2 → 5

Result: [1,4,3,2,5] ✓

## 7. Pattern Variations

| Problem          | Key Technique                       |
|------------------|-------------------------------------|
| Full reversal    | 3-pointer: prev/curr/next           |
| Partial reversal | Dummy + head insertion              |
| K-group reversal | Count k, reverse segment, reconnect |
| Merge two lists  | Dummy node + two pointers           |
| Reorder          | Find mid + reverse + interleave     |
| Partition        | Two dummy lists for < and >= pivot  |
| Add two numbers  | Dummy + carry variable              |

## 8. Common Interview Mistakes

**1. Losing track of next pointer** — always `ListNode next = curr.next` BEFORE modifying `curr.next`

2. **Not using dummy node when head might change** — causes null pointer or incorrect head
  3. **Reverse between: off-by-one in for loop** — loop runs right - left times (not right - left + 1 )
  4. **K-group: not checking remaining nodes** — must verify k nodes exist before reversing
  5. **Reorder list: not cutting first half** — slow.next = null is mandatory before merging
  6. **Returning prev vs dummy.next** — when using dummy, always return dummy.next
- 

## 9. Interview Cheat Sheet

### LINKED LIST REVERSAL – MENTAL CHECKLIST

=====

- Head might change? → Use dummy node, return dummy.next
- Full reversal? → prev=null, curr=head, 3-pointer loop
- Partial reversal? → dummy + head insertion trick
- K-group? → count k, reverse segment, advance groupPrev
- Build result list? → dummy node + curr pointer

### 3-POINTER REVERSAL (MEMORIZE)

=====

```
prev = null; curr = head
while (curr != null):
    next = curr.next
    curr.next = prev
    prev = curr
    curr = next
return prev ← new head
```

### DUMMY NODE PATTERN

=====

```
dummy = new ListNode(0); dummy.next = head
// ... build/modify list using dummy.next as anchor
return dummy.next
```

### TRICKS

=====

- Head insertion for partial reverse: take node from curr.next, put at prev.next
  - Reorder = find middle (fast/slow) + reverse second half + interleave
  - Two dummy lists for partition problems (one for each condition)
  - Sort list: merge sort with find-middle + recursive merge
- 

## 10. Practice Roadmap

### Beginner:

- LC 206 — Reverse Linked List
- LC 21 — Merge Two Sorted Lists
- LC 83 — Remove Duplicates from Sorted List

### Intermediate:

- LC 92 — Reverse Linked List II
- LC 143 — Reorder List

- LC 86 — Partition List
- LC 2 — Add Two Numbers

**Hard:**

- LC 25 — Reverse Nodes in k-Group
- LC 23 — Merge K Sorted Lists
- LC 148 — Sort List

# Pattern 11 — Graph DFS & BFS (Adjacency List)

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "connected components", "number of islands"
- "can you reach from A to B", "path exists"
- "clone graph", "all paths from source"
- "flood fill", "surrounded regions"
- "friend circles", "provinces"

**DFS vs BFS for graphs:**

- **DFS:** explore as deep as possible, backtrack. Good for: connectivity, cycle detection, path existence, flood fill
- **BFS:** explore level by level. Good for: shortest path (unweighted), minimum steps, multi-source spread

**Signal phrase:** \_"Number of connected components"\_ → DFS or Union Find

**Signal phrase:** \_"Shortest path / minimum steps"\_ → BFS

## 2. Core Intuition

**The critical difference from trees: graphs have cycles.**

You MUST track visited nodes, or you'll loop forever.

**DFS mental model:** Go deep down one path, backtrack when stuck, try another path.

**BFS mental model:** Ripple outward from source — like dropping a stone in water.

Graph: 0-1-3

```
| |  
2 4
```

DFS from 0: visit 0 → 1 → 3 → 4 (backtrack) → (backtrack) → 2

BFS from 0: visit 0 → [1,2] → [3] → [4]

## 3. Generic Java Templates

### Graph Representation

```
// Build adjacency list from edge list
Map<Integer, List<Integer>> buildGraph(int n, int[][] edges) {
    Map<Integer, List<Integer>> adj = new HashMap<>();
    for (int i = 0; i < n; i++) adj.put(i, new ArrayList<>());
    for (int[] edge : edges) {
        adj.get(edge[0]).add(edge[1]);
        adj.get(edge[1]).add(edge[0]); // undirected
    }
    return adj;
}
```

## Template A — Graph DFS (Recursive)

```
public void dfs(int node, Map<Integer, List<Integer>> adj, boolean[] visited) {
    visited[node] = true;
    // Process node

    for (int neighbor : adj.get(node)) {
        if (!visited[neighbor]) {
            dfs(neighbor, adj, visited);
        }
    }
}

// Count connected components
public int countComponents(int n, int[][] edges) {
    Map<Integer, List<Integer>> adj = buildGraph(n, edges);
    boolean[] visited = new boolean[n];
    int components = 0;

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            dfs(i, adj, visited);
            components++;
        }
    }
    return components;
}
```

## Template B — Graph DFS (Iterative)

```

public void dfsIterative(int start, Map<Integer, List<Integer>> adj, boolean[] visited) {
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(start);

    while (!stack.isEmpty()) {
        int node = stack.pop();
        if (visited[node]) continue; // guard for multiple pushes
        visited[node] = true;

        // Process node

        for (int neighbor : adj.get(node)) {
            if (!visited[neighbor]) {
                stack.push(neighbor);
            }
        }
    }
}

```

### Template C — Graph BFS (Shortest Path)

```

public int bfsShortestPath(int src, int dst, Map<Integer, List<Integer>> adj) {
    Queue<Integer> queue = new LinkedList<>();
    Set<Integer> visited = new HashSet<>();

    queue.offer(src);
    visited.add(src);
    int distance = 0;

    while (!queue.isEmpty()) {
        int levelSize = queue.size();

        for (int i = 0; i < levelSize; i++) {
            int node = queue.poll();

            if (node == dst) return distance;

            for (int neighbor : adj.get(node)) {
                if (!visited.contains(neighbor)) {
                    visited.add(neighbor);
                    queue.offer(neighbor);
                }
            }
        }
        distance++;
    }

    return -1; // unreachable
}

```

### Template D — Valid Path (src → dst)

```

public boolean validPath(int n, int[][] edges, int source, int destination) {
    if (source == destination) return true;

    Map<Integer, List<Integer>> adj = buildGraph(n, edges);
    boolean[] visited = new boolean[n];

    return dfsPath(source, destination, adj, visited);
}

private boolean dfsPath(int node, int dst, Map<Integer, List<Integer>> adj, boolean[] visited) {
    if (node == dst) return true;
    visited[node] = true;

    for (int neighbor : adj.get(node)) {
        if (!visited[neighbor]) {
            if (dfsPath(neighbor, dst, adj, visited)) return true;
        }
    }
    return false;
}

```

### Template E — All Paths from Source to Target (DAG)

```

public List<List<Integer>> allPathsSourceTarget(int[][] graph) {
    List<List<Integer>> result = new ArrayList<>();
    List<Integer> path = new ArrayList<>();
    path.add(0);
    dfsAllPaths(graph, 0, graph.length - 1, path, result);
    return result;
}

private void dfsAllPaths(int[][] graph, int node, int target,
    List<Integer> path, List<List<Integer>> result) {
    if (node == target) {
        result.add(new ArrayList<>(path));
        return;
    }
    for (int neighbor : graph[node]) {
        path.add(neighbor);
        dfsAllPaths(graph, neighbor, target, path, result);
        path.remove(path.size() - 1); // backtrack
    }
}

```

## 4. Complexity Cheatsheet

| Operation               | Time       | Space      |
|-------------------------|------------|------------|
| DFS / BFS traversal     | $O(V + E)$ | $O(V)$     |
| Adjacency list build    | $O(V + E)$ | $O(V + E)$ |
| Adjacency matrix lookup | $O(1)$     | $O(V^2)$   |

$V$  = vertices,  $E$  = edges. Always  $O(V + E)$  for graph traversal.

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                         | Difficulty | Why Important        |
|------|---------------------------------|------------|----------------------|
| 200  | Number of Islands               | Medium     | Grid DFS / BFS       |
| 133  | Clone Graph                     | Medium     | DFS with HashMap     |
| 547  | Number of Provinces             | Medium     | Adjacency matrix DFS |
| 1971 | Find if Path Exists in Graph    | Easy       | Basic DFS/BFS        |
| 797  | All Paths From Source to Target | Medium     | DFS + backtrack      |

### Advanced Variations

| LC # | Problem                     | Difficulty | Why Important                 |
|------|-----------------------------|------------|-------------------------------|
| 695  | Max Area of Island          | Medium     | DFS returning size            |
| 417  | Pacific Atlantic Water Flow | Medium     | Reverse BFS from edges        |
| 130  | Surrounded Regions          | Medium     | BFS from border               |
| 127  | Word Ladder                 | Hard       | BFS on implicit graph         |
| 207  | Course Schedule             | Medium     | Cycle detection (topological) |

## 6. Solve Step-by-Step — LC 200: Number of Islands

**Problem:** Count number of islands (connected groups of '1's) in a grid.

### Approach: DFS Flood Fill

Whenever we encounter a '1', we DFS to mark the entire island as visited ('0'), then increment count.

```

public int numIslands(char[][] grid) {
    if (grid == null || grid.length == 0) return 0;

    int rows = grid.length, cols = grid[0].length;
    int islands = 0;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == '1') {
                islands++;
                dfs(grid, r, c); // sink the island
            }
        }
    }
    return islands;
}

private void dfs(char[][] grid, int r, int c) {
    // Bounds check + water check
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length || grid[r][c] != '1') {
        return;
    }

    grid[r][c] = '0'; // mark visited (sink it)

    dfs(grid, r + 1, c);
    dfs(grid, r - 1, c);
    dfs(grid, r, c + 1);
    dfs(grid, r, c - 1);
}

```

## Dry Run

Grid:

```

1 1 0 0 0
1 1 0 0 0
0 0 1 0 0
0 0 0 1 1

```

r=0,c=0: '1' found → islands=1, DFS sinks (0,0),(0,1),(1,0),(1,1)

r=2,c=2: '1' found → islands=2, DFS sinks (2,2)

r=3,c=3: '1' found → islands=3, DFS sinks (3,3),(3,4)

Answer: 3 ✓

## Edge Cases

- All water: [['0']] → 0
- All land: one island
- Single cell: 1

## 7. Pattern Variations

| Problem Type     | Approach                                  |
|------------------|-------------------------------------------|
| Count components | Outer loop + DFS/BFS from each unvisited  |
| Path exists      | DFS with early return true                |
| All paths        | DFS + backtracking                        |
| Shortest path    | BFS with distance counter                 |
| Flood fill       | DFS/BFS modifying grid in-place           |
| Multi-source BFS | Add all sources to queue at start         |
| Cycle detection  | DFS with color marking (white/gray/black) |

## 8. Common Interview Mistakes

- 1. Not marking visited before recursing** — infinite loop in cyclic graphs
- 2. Grid bounds check missing** — always check `r >= 0 && r < rows`  
&& `c >= 0 && c < cols` first
- 3. Modifying input grid** — ask interviewer if this is allowed; if not, use separate `visited[][]`
- 4. BFS: adding to visited when polling** instead of when offering — can add duplicates to queue
- 5. Forgetting 4-directional vs 8-directional** — clarify with interviewer
- 6. Using `visited` set for grid** — using grid modification is cleaner and  $O(1)$  space

## 9. Interview Cheat Sheet

## GRAPH DFS/BFS — MENTAL CHECKLIST

=====

- Cycles possible? → MUST track visited
- Shortest path? → BFS (not DFS)
- Connected components? → outer loop + DFS/BFS
- Grid problem? → 4 directions, bounds check, mark visited
- Mark visited WHEN? → BFS: when adding to queue (not polling)

## DFS STRUCTURE

=====

```
visited[node] = true
for neighbor in adj[node]:
    if not visited[neighbor]:
        dfs(neighbor)
```

## BFS STRUCTURE

=====

```
queue.offer(src); visited.add(src)
while queue not empty:
    node = queue.poll()
    for neighbor: if not visited → visited.add + queue.offer
```

## GRID DIRECTIONS

=====

```
int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};
for (int[] d : dirs) dfs(grid, r+d[0], c+d[1]);
```

COMPLEXITY: Always  $O(V+E)$

---

## 10. Practice Roadmap

### Beginner:

- LC 1971 — Find if Path Exists
- LC 200 — Number of Islands
- LC 547 — Number of Provinces

### Intermediate:

- LC 133 — Clone Graph
- LC 695 — Max Area of Island
- LC 797 — All Paths Source to Target
- LC 417 — Pacific Atlantic Water Flow

### Hard:

- LC 127 — Word Ladder
- LC 130 — Surrounded Regions
- LC 207/210 — Course Schedule I/II (Topological)

# Pattern 12 — Multi-Source BFS + Matrix Traversal

---

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "distance to nearest 0/gate/target"
- "rotting oranges", "spread from multiple sources"
- "walls and gates"
- "01 matrix" (distance to nearest 0)
- "Pacific Atlantic water flow" (reverse BFS from two edges)

**Multi-Source BFS triggers:**

- Multiple starting points simultaneously
- "All X spread at once" or "minimum distance from ANY of the X's"
- Expansion from boundaries inward

**Signal phrase:** \_"Minimum distance from each cell to the nearest gate"\_ → Multi-Source BFS

**Signal phrase:** \_"Rotting oranges spread each minute"\_ → Multi-Source BFS, track time (levels)

---

## 2. Core Intuition

**Single-source BFS:** one starting node, expand outward.

**Multi-source BFS:** add ALL starting nodes to the queue at once before the first iteration. The BFS then expands from all of them simultaneously — as if one "super source" connects to all of them with zero-cost edges.

Grid with gates (G) and rooms (R):

```
INF INF INF INF
INF G INF INF
INF INF INF G
INF INF INF INF
```

Single-source: run BFS from each G separately →  $O(k \times m \times n)$

Multi-source: add ALL G's to queue first →  $O(m \times n)$  total

Queue starts: [(1,1), (2,3)] ← both gates at once

BFS expands: distance 1 cells for both simultaneously

---

## 3. Generic Java Templates

### Template A — Multi-Source BFS (Walls and Gates / 01 Matrix)

```

public void wallsAndGates(int[][] rooms) {
    int rows = rooms.length, cols = rooms[0].length;
    Queue<int[]> queue = new LinkedList<>();

    // Step 1: Add ALL sources (gates/zeros) to queue first
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (rooms[r][c] == 0) { // 0 = gate/source
                queue.offer(new int[]{r, c});
            }
        }
    }

    // Step 2: BFS from all sources simultaneously
    int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};
    while (!queue.isEmpty()) {
        int[] cell = queue.poll();
        int r = cell[0], c = cell[1];

        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];

            // Only update INF cells (unvisited rooms)
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols
                && rooms[nr][nc] == Integer.MAX_VALUE) {
                rooms[nr][nc] = rooms[r][c] + 1; // distance propagates
                queue.offer(new int[]{nr, nc});
            }
        }
    }
}

```

## Template B — Rotting Oranges (Track Time as BFS Levels)

```

public int orangesRotting(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    Queue<int[]> queue = new LinkedList<>();
    int fresh = 0;

    // Step 1: Add all rotten oranges, count fresh
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 2) queue.offer(new int[]{r, c});
            else if (grid[r][c] == 1) fresh++;
        }
    }

    if (fresh == 0) return 0; // no fresh oranges

    int minutes = 0;
    int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};

    while (!queue.isEmpty() && fresh > 0) {
        int levelSize = queue.size();
        minutes++;

        for (int i = 0; i < levelSize; i++) {
            int[] cell = queue.poll();
            for (int[] d : dirs) {
                int nr = cell[0] + d[0], nc = cell[1] + d[1];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols
                    && grid[nr][nc] == 1) {
                    grid[nr][nc] = 2; // rot it
                    fresh--;
                    queue.offer(new int[]{nr, nc});
                }
            }
        }
    }

    return fresh == 0 ? minutes : -1; // -1 if unreachable fresh remain
}

```

## Template C — 01 Matrix (Distance to Nearest 0)

```

public int[][] updateMatrix(int[][] mat) {
    int rows = mat.length, cols = mat[0].length;
    int[][] dist = new int[rows][cols];
    Queue<int[]> queue = new LinkedList<>();

    // Add all 0s to queue, mark 1s as unvisited (MAX_VALUE)
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (mat[r][c] == 0) {
                dist[r][c] = 0;
                queue.offer(new int[]{r, c});
            } else {
                dist[r][c] = Integer.MAX_VALUE;
            }
        }
    }

    int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};
    while (!queue.isEmpty()) {
        int[] cell = queue.poll();
        int r = cell[0], c = cell[1];
        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols
                && dist[nr][nc] == Integer.MAX_VALUE) {
                dist[nr][nc] = dist[r][c] + 1;
                queue.offer(new int[]{nr, nc});
            }
        }
    }
    return dist;
}

```

## Template D — Pacific Atlantic Water Flow (Reverse BFS from Borders)

```

public List<List<Integer>> pacificAtlantic(int[][] heights) {
    int rows = heights.length, cols = heights[0].length;
    boolean[][] pacific = new boolean[rows][cols];
    boolean[][] atlantic = new boolean[rows][cols];
    Queue<int[]> pQueue = new LinkedList<>();
    Queue<int[]> aQueue = new LinkedList<>();

    // Add border cells to each ocean's queue
    for (int r = 0; r < rows; r++) {
        pQueue.offer(new int[]{r, 0}); pacific[r][0] = true;
        aQueue.offer(new int[]{r, cols-1}); atlantic[r][cols-1] = true;
    }
    for (int c = 0; c < cols; c++) {
        pQueue.offer(new int[]{0, c}); pacific[0][c] = true;
        aQueue.offer(new int[]{rows-1, c}); atlantic[rows-1][c] = true;
    }

    bfsOcean(heights, pQueue, pacific);
    bfsOcean(heights, aQueue, atlantic);

    List<List<Integer>> result = new ArrayList<>();
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (pacific[r][c] && atlantic[r][c]) {
                result.add(Arrays.asList(r, c));
            }
        }
    }
    return result;
}

private void bfsOcean(int[][] heights, Queue<int[]> queue, boolean[][] visited) {
    int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};
    while (!queue.isEmpty()) {
        int[] cell = queue.poll();
        int r = cell[0], c = cell[1];
        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < heights.length && nc >= 0 && nc < heights[0].length
                && !visited[nr][nc]
                && heights[nr][nc] >= heights[r][c]) { // water flows uphill in reverse
                visited[nr][nc] = true;
                queue.offer(new int[]{nr, nc});
            }
        }
    }
}

```

---

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                     | Difficulty | Why Important                   |
|------|-----------------------------|------------|---------------------------------|
| 994  | Rotting Oranges             | Medium     | Time-based multi-source BFS     |
| 542  | 01 Matrix                   | Medium     | Distance multi-source BFS       |
| 286  | Walls and Gates             | Medium     | Propagate distance from sources |
| 417  | Pacific Atlantic Water Flow | Medium     | Reverse BFS from two borders    |
| 1765 | Map of Highest Peak         | Medium     | Multi-source height BFS         |

## Advanced Variations

| LC # | Problem                        | Difficulty | Why Important             |
|------|--------------------------------|------------|---------------------------|
| 1162 | As Far from Land as Possible   | Medium     | Max distance to nearest 0 |
| 1091 | Shortest Path in Binary Matrix | Medium     | BFS with diagonal moves   |
| 934  | Shortest Bridge                | Medium     | DFS find + BFS expand     |
| 1197 | Minimum Knight Moves           | Medium     | BFS from target           |
| 2146 | K Highest Ranked Items         | Medium     | BFS + sorting             |

## 6. Solve Step-by-Step — LC 994: Rotting Oranges

Already covered in Template B above. Key insight recap:

1. All rotten oranges go in queue simultaneously — that's the "multi-source" start
2. Track `fresh` count — if it hits 0, we're done early
3. Track `minutes` by BFS level
4. If any fresh remain after BFS exhausts → return -1

### Edge Cases

- No fresh oranges: return 0 immediately
- Disconnected fresh: return -1 (fresh can never rot)
- All rotten: return 0
- 1×1 grid with fresh: return -1 (no rotten to spread)

## 7. Pattern Variations

| Variation                    | Key Trick                            |
|------------------------------|--------------------------------------|
| Distance from nearest source | Multi-source BFS, propagate distance |
| Time until all infected      | BFS levels = time units              |

| Variation                          | Key Trick                             |
|------------------------------------|---------------------------------------|
| Can all cells be reached           | Check remaining unvisited after BFS   |
| Flow from border                   | Reverse BFS: from ocean/border inward |
| Shortest path with obstacles       | BFS, skip walls                       |
| Expand from one set toward another | BFS from set A until reaching set B   |

## 8. Common Interview Mistakes

1. **Single-source BFS for multi-source problem** →  $O(k \times n \times m)$  instead of  $O(n \times m)$
2. **Not adding visited check before queueing** → TLE from revisiting cells
3. **Rotting oranges: counting minutes wrong** → use BFS levels, not cell count
4. **Pacific Atlantic: going forward** (water flows down) vs reverse (reachability flows up)
5. **Not handling isolated cells** (fresh orange with no rotten neighbor reachable)

## 9. Interview Cheat Sheet

### MULTI-SOURCE BFS – MENTAL CHECKLIST

=====

- "From any/all X, find distance to Y" → Multi-source BFS
- "Spread simultaneously" → Multi-source BFS (all sources in queue at once)
- Time = BFS level count (use levelSize snapshot)
- Distance = propagate from source cells, not destination
- Reverse BFS: when checking "can X reach Y" – start from Y, go backward

### TEMPLATE SKELETON

=====

```
// 1. Find all sources → add to queue, mark visited
// 2. BFS expand:
while (!queue.isEmpty()):
    [optional: int size = queue.size(); minutes++]
    for each cell in level:
        for each 4-direction neighbor:
            if valid and unvisited:
                mark visited, update dist/state, enqueue
```

### TRICKS

=====

- Use the grid itself as visited marker (set to distance value)
- Separate dist[][] matrix if grid must be preserved
- "Reverse BFS" for flow/reachability problems (Pacific Atlantic)
- Check remaining unvisited count to detect unreachable cells

## 10. Practice Roadmap

### Beginner:

- LC 1765 — Map of Highest Peak

- LC 542 — 01 Matrix

**Intermediate:**

- LC 994 — Rotting Oranges
- LC 286 — Walls and Gates
- LC 1162 — As Far from Land as Possible

**Hard:**

- LC 417 — Pacific Atlantic Water Flow
- LC 934 — Shortest Bridge
- LC 1091 — Shortest Path in Binary Matrix

# Pattern 13 — Topological Sort

---

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "course prerequisites", "task dependencies"
- "build order", "compilation order"
- "can all tasks be completed"
- "order of execution", "scheduling"
- "detect cycle in directed graph"

**When to reach for Topological Sort:**

- Directed Acyclic Graph (DAG) implied
- Need ordering where all dependencies come before dependents
- Cycle detection in directed graph

**Signal phrase:** `"Course A requires course B as prerequisite"` → Topological Sort

**Signal phrase:** `"Is it possible to complete all courses?"` → Cycle detection via Topo Sort

---

## 2. Core Intuition

**What is topological order?**

A linear ordering of nodes such that for every directed edge  $u \rightarrow v$ , node  $u$  comes before  $v$ .

Only possible for DAGs — a cycle means no valid ordering exists.

**Two algorithms:**

**Kahn's Algorithm (BFS-based) — Preferred in interviews**

- Count in-degree (incoming edges) for each node
- Start with all nodes having in-degree 0 (no prerequisites)
- Process each, reduce neighbors' in-degrees; add to queue when they hit 0
- If all nodes processed → valid order. If not → cycle exists.

**DFS-based — Reverse post-order**

- DFS from each unvisited node
- Add to result AFTER all descendants are processed (post-order)
- Reverse the result

Graph:  $0 \rightarrow 1 \rightarrow 3$

↓ ↑  
2 —

In-degrees: 0:0, 1:1, 2:1, 3:2

Queue starts: [0]

Process 0 → reduce 1( $\rightarrow 0$ ), 2( $\rightarrow 0$ ) → queue: [1,2]

Process 1 → reduce 3( $\rightarrow 1$ ) → queue: [2]

Process 2 → reduce 3( $\rightarrow 0$ ) → queue: [3]

Process 3 → queue empty

Order: [0,1,2,3] ✓

---

### 3. Generic Java Templates

#### Template A — Kahn's Algorithm (BFS Topological Sort)

```

public int[] topologicalSort(int n, int[][] prerequisites) {
    // Build adjacency list and compute in-degrees
    List<List<Integer>> adj = new ArrayList<>();
    int[] inDegree = new int[n];

    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());

    for (int[] pre : prerequisites) {
        int course = pre[0], prereq = pre[1];
        adj.get(prereq).add(course); // prereq → course
        inDegree[course]++;
    }

    // Initialize queue with all zero in-degree nodes
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < n; i++) {
        if (inDegree[i] == 0) queue.offer(i);
    }

    int[] order = new int[n];
    int idx = 0;

    while (!queue.isEmpty()) {
        int node = queue.poll();
        order[idx++] = node;

        for (int neighbor : adj.get(node)) {
            inDegree[neighbor]--;
            if (inDegree[neighbor] == 0) {
                queue.offer(neighbor);
            }
        }
    }

    // If idx == n, all nodes processed → no cycle
    return idx == n ? order : new int[]{};
}

```

## Template B — Can Finish (Cycle Detection)

```

public boolean canFinish(int numCourses, int[][] prerequisites) {
    List<List<Integer>> adj = new ArrayList<>();
    int[] inDegree = new int[numCourses];

    for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());

    for (int[] pre : prerequisites) {
        adj.get(pre[1]).add(pre[0]);
        inDegree[pre[0]]++;
    }

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++) {
        if (inDegree[i] == 0) queue.offer(i);
    }

    int processed = 0;
    while (!queue.isEmpty()) {
        int node = queue.poll();
        processed++;
        for (int neighbor : adj.get(node)) {
            if (--inDegree[neighbor] == 0) queue.offer(neighbor);
        }
    }

    return processed == numCourses; // true = no cycle
}

```

## Template C — DFS-Based Topological Sort

```
// 0 = unvisited, 1 = in-progress, 2 = done
public int[] topoSortDFS(int n, int[][] edges) {
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    for (int[] e : edges) adj.get(e[0]).add(e[1]);

    int[] state = new int[n]; // 0/1/2
    Deque<Integer> stack = new ArrayDeque<>();
    boolean[] hasCycle = {false};

    for (int i = 0; i < n; i++) {
        if (state[i] == 0) dfs(i, adj, state, stack, hasCycle);
    }

    if (hasCycle[0]) return new int[]{}; // cycle detected

    int[] result = new int[n];
    int i = 0;
    while (!stack.isEmpty()) result[i++] = stack.pop();
    return result;
}

private void dfs(int node, List<List<Integer>> adj, int[] state,
    Deque<Integer> stack, boolean[] hasCycle) {
    state[node] = 1; // in progress

    for (int neighbor : adj.get(node)) {
        if (state[neighbor] == 1) { hasCycle[0] = true; return; } // back edge = cycle
        if (state[neighbor] == 0) dfs(neighbor, adj, state, stack, hasCycle);
    }

    state[node] = 2; // done
    stack.push(node); // add after all descendants
}

```

## Template D — Course Schedule with Levels (BFS order by level)

```
// Process courses level by level (semester by semester)
public List<List<Integer>> findOrder(int n, int[][] prereqs) {
    // Same Kahn's but group nodes by BFS level
    // ... (same structure as Template A, but collect by level like BFS)
    List<List<Integer>> semesters = new ArrayList<>();
    // [omitted for brevity – same as level-order BFS]
    return semesters;
}

```

## 4. Complexity Cheatsheet

| Algorithm       | Time       | Space      |
|-----------------|------------|------------|
| Kahn's (BFS)    | $O(V + E)$ | $O(V + E)$ |
| DFS-based       | $O(V + E)$ | $O(V + E)$ |
| Cycle detection | $O(V + E)$ | $O(V)$     |

---

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                 | Difficulty | Why Important                 |
|------|-------------------------|------------|-------------------------------|
| 207  | Course Schedule         | Medium     | Cycle detection via topo sort |
| 210  | Course Schedule II      | Medium     | Return actual order           |
| 310  | Minimum Height Trees    | Medium     | Topo sort from leaves inward  |
| 269  | Alien Dictionary        | Hard       | Build graph from word order   |
| 444  | Sequence Reconstruction | Medium     | Unique topo order check       |

### Advanced Variations

| LC # | Problem                           | Difficulty | Why Important                    |
|------|-----------------------------------|------------|----------------------------------|
| 1136 | Parallel Courses                  | Medium     | Min semesters (BFS levels)       |
| 2115 | Find All Possible Recipes         | Medium     | Multi-source topo sort           |
| 329  | Longest Increasing Path in Matrix | Hard       | DFS + memoization (implicit DAG) |
| 1203 | Sort Items by Groups              | Hard       | Two-level topological sort       |
| 802  | Find Eventual Safe States         | Medium     | Reverse graph + topo sort        |

---

## 6. Solve Step-by-Step — LC 210: Course Schedule II

**Problem:** Return the course order to finish all numCourses, or [] if impossible.

```

public int[] findOrder(int numCourses, int[][] prerequisites) {
    List<List<Integer>> adj = new ArrayList<>();
    int[] inDegree = new int[numCourses];

    for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());

    for (int[] pre : prerequisites) {
        // pre[1] must come before pre[0]
        adj.get(pre[1]).add(pre[0]);
        inDegree[pre[0]]++;
    }

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++) {
        if (inDegree[i] == 0) queue.offer(i);
    }

    int[] result = new int[numCourses];
    int idx = 0;

    while (!queue.isEmpty()) {
        int course = queue.poll();
        result[idx++] = course;
        for (int next : adj.get(course)) {
            if (--inDegree[next] == 0) queue.offer(next);
        }
    }

    return idx == numCourses ? result : new int[]{};
}

```

## Dry Run

numCourses=4, prerequisites=[[1,0],[2,0],[3,1],[3,2]]

Meaning: 0→1, 0→2, 1→3, 2→3

adj: 0→[1,2], 1→[3], 2→[3]

inDegree: [0, 1, 1, 2]

Queue starts: [0] (only 0 has no prereqs)

Process 0: result=[0], reduce 1(→0), 2(→0) → queue=[1,2]

Process 1: result=[0,1], reduce 3(→1) → queue=[2]

Process 2: result=[0,1,2], reduce 3(→0) → queue=[3]

Process 3: result=[0,1,2,3]

idx=4 == numCourses=4 → return [0,1,2,3] ✓

## 7. Pattern Variations

| Variation                 | Approach                        |
|---------------------------|---------------------------------|
| Can all courses be taken? | Kahn's — check if all processed |
| Return actual order       | Kahn's — collect in order       |

| Variation                       | Approach                   |
|---------------------------------|----------------------------|
| Minimum time (parallel)         | Kahn's — count BFS levels  |
| Detect back edge                | DFS with 3-color marking   |
| Reconstruct from partial order  | Unique topological sort    |
| Leaves inward (min height tree) | Reverse topo — trim leaves |

## 8. Common Interview Mistakes

- 1. Edge direction:** prerequisites=[course, prereq] → edge is prereq→course. Easy to flip!
- 2. Not checking** `idx == n` at the end — forgetting that incomplete processing = cycle
- 3. DFS: not distinguishing in-progress (1) from done (2)** — both would look "visited"
- 4. Kahn's vs DFS choice:** Kahn's is easier to implement and explain; prefer in interviews
- 5. Forgetting to initialize all adjacency lists** — `adj.get(i)` NPE if not initialized

## 9. Interview Cheat Sheet

### TOPOLOGICAL SORT — MENTAL CHECKLIST

=====

- Dependencies / prerequisites / ordering? → Topological Sort
- Directed graph with possible cycles? → Kahn's detects cycles
- "Can all be completed?" → check processed count == n
- Need actual order? → collect nodes as processed in Kahn's
- Minimum time / parallel processing? → count BFS levels

### KAHN'S ALGORITHM (MEMORIZE)

=====

1. Build adj list + compute `inDegree[]`
2. Queue all nodes with `inDegree[i] == 0`
3. While queue not empty:
  - poll node, add to result
  - for each neighbor: `inDegree[neighbor]--`
  - if `inDegree[neighbor] == 0`: add to queue
4. If `result.size() == n`: valid order. Else: cycle.

### EDGE DIRECTION CHEATSHEET

=====

```
prerequisites = [[course, prereq]]
→ prereq must come BEFORE course
→ Add edge: adj.get(prereq).add(course)
→ inDegree[course]++
```

### TRICKS

=====

- Minimum height trees: repeatedly remove leaf nodes (Kahn's from edges)
- "Safe states": reverse graph, find nodes that reach no cycles
- Two-level topo sort: sort within groups AND between groups

## 10. Practice Roadmap

### Beginner:

- LC 207 — Course Schedule
- LC 210 — Course Schedule II

### Intermediate:

- LC 310 — Minimum Height Trees
- LC 802 — Find Eventual Safe States
- LC 1136 — Parallel Courses

### Hard:

- LC 269 — Alien Dictionary
- LC 329 — Longest Increasing Path in Matrix
- LC 1203 — Sort Items by Groups Respecting Dependencies

# Pattern 14 — Union Find (Disjoint Set Union / DSU)

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "connected components", "number of groups"
- "union two sets", "are these two elements connected"
- "detect cycle in undirected graph"
- "accounts merge", "friend circles"
- "redundant connection", "minimum spanning tree"

**When Union Find beats BFS/DFS:**

- Repeated connectivity queries (online / streaming)
- Dynamic connections (edges added over time)
- Need to merge groups efficiently
- Minimum spanning tree (Kruskal's algorithm)

**Signal phrase:** `"Dynamically connect elements and query connectivity"` → Union Find

**Signal phrase:** `"Detect if adding this edge creates a cycle"` → Union Find

## 2. Core Intuition

**The problem:** We have  $n$  elements. We need to:

1. **Union(x, y):** merge the groups containing  $x$  and  $y$
2. **Find(x):** which group does  $x$  belong to?

**Naive approach:** An array where `group[i] = group ID`. Merging =  $O(n)$  update.

**Union Find:** Each element points to a "parent". Groups form trees. The root = group representative.

**Two optimizations that make it nearly  $O(1)$  amortized:**

1. **Union by Rank:** attach smaller tree under larger tree (keeps trees flat)
2. **Path Compression:** during Find, make every node on the path point directly to root

Initial: each node is its own parent

`[0] [1] [2] [3] [4]`

Union(0,1): 0 becomes parent of 1 (or vice versa by rank)

`[0→0] [1→0] [2] [3] [4]`

Union(2,3): 2 becomes parent of 3

`[0→0] [1→0] [2→2] [3→2] [4]`

Union(1,3): `find(1)=0, find(3)=2` → merge: 2's root under 0's root

`[0→0] [1→0] [2→0] [3→2→0] [4]`

Path compression: when finding 3, `3→2→0`, compress to `3→0`

---

## 3. Generic Java Templates

### Template A — Full Union Find with Rank + Path Compression

```
class UnionFind {
    private int[] parent;
    private int[] rank;
    private int components; // track number of connected components

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        components = n;
        for (int i = 0; i < n; i++) parent[i] = i; // each node is its own parent
    }

    // Find with path compression (iterative – safer for large inputs)
    public int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]]; // path halving (one-pass compression)
            x = parent[x];
        }
        return x;
    }

    // Union by rank – returns true if actually merged (were in different components)
    public boolean union(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);

        if (rootX == rootY) return false; // already connected

        if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
        } else if (rank[rootX] > rank[rootY]) {
            parent[rootY] = rootX;
        } else {
            parent[rootY] = rootX;
            rank[rootX]++;
        }

        components--;
        return true;
    }

    public boolean connected(int x, int y) {
        return find(x) == find(y);
    }

    public int getComponents() {
        return components;
    }
}
```

### Template B — Number of Connected Components

```
public int countComponents(int n, int[][] edges) {
    UnionFind uf = new UnionFind(n);

    for (int[] edge : edges) {
        uf.union(edge[0], edge[1]);
    }

    return uf.getComponents();
}
```

### Template C — Redundant Connection (Cycle Detection)

```
public int[] findRedundantConnection(int[][] edges) {
    int n = edges.length;
    UnionFind uf = new UnionFind(n + 1); // nodes 1-indexed

    for (int[] edge : edges) {
        // If already connected → this edge creates a cycle
        if (!uf.union(edge[0], edge[1])) {
            return edge;
        }
    }

    return new int[]{};
}
```

### Template D — Accounts Merge (Union Find with HashMap)

```

public List<List<String>> accountsMerge(List<List<String>> accounts) {
    // Map email → index
    Map<String, Integer> emailToId = new HashMap<>();
    Map<String, String> emailToName = new HashMap<>();
    int id = 0;

    for (List<String> account : accounts) {
        String name = account.get(0);
        for (int i = 1; i < account.size(); i++) {
            String email = account.get(i);
            if (!emailToId.containsKey(email)) {
                emailToId.put(email, id++);
                emailToName.put(email, name);
            }
        }
    }

    UnionFind uf = new UnionFind(id);

    for (List<String> account : accounts) {
        int firstId = emailToId.get(account.get(1));
        for (int i = 2; i < account.size(); i++) {
            uf.union(firstId, emailToId.get(account.get(i)));
        }
    }

    // Group emails by root
    Map<Integer, List<String>> groups = new HashMap<>();
    for (String email : emailToId.keySet()) {
        int root = uf.find(emailToId.get(email));
        groups.computeIfAbsent(root, k -> new ArrayList<>()).add(email);
    }

    List<List<String>> result = new ArrayList<>();
    for (List<String> emails : groups.values()) {
        Collections.sort(emails);
        emails.add(0, emailToName.get(emails.get(0)));
        result.add(emails);
    }

    return result;
}

```

## 4. Complexity Cheatsheet

| Operation    | Time (with rank + compression)        | Notes                        |
|--------------|---------------------------------------|------------------------------|
| Find         | $O(\alpha(n)) \approx O(1)$ amortized | $\alpha$ = inverse Ackermann |
| Union        | $O(\alpha(n)) \approx O(1)$ amortized |                              |
| n operations | $O(n \times \alpha(n)) \approx O(n)$  | Effectively linear           |
| Space        | $O(n)$                                | parent + rank arrays         |

$\alpha(n)$  is the inverse Ackermann function — grows so slowly it's practically constant for any real input.

---

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                    | Difficulty | Why Important          |
|------|----------------------------|------------|------------------------|
| 547  | Number of Provinces        | Medium     | Basic Union Find usage |
| 684  | Redundant Connection       | Medium     | Cycle detection        |
| 200  | Number of Islands          | Medium     | Also solvable with UF  |
| 721  | Accounts Merge             | Medium     | UF + HashMap           |
| 1202 | Smallest String with Swaps | Medium     | UF for index grouping  |

### Advanced Variations

| LC # | Problem                               | Difficulty | Why Important          |
|------|---------------------------------------|------------|------------------------|
| 685  | Redundant Connection II               | Hard       | Directed graph variant |
| 1061 | Lexicographically Smallest Equivalent | Medium     | UF with custom root    |
| 839  | Similar String Groups                 | Hard       | UF + string comparison |
| 1579 | Remove Max Number of Edges            | Hard       | Two parallel UF        |
| 765  | Couples Holding Hands                 | Hard       | Cyclic swap via UF     |

---

## 6. Solve Step-by-Step — LC 684: Redundant Connection

**Problem:** Find the edge that, when removed, results in a tree (undirected graph with one fewer edge than vertices, no cycle).

### Approach

Process edges one by one. The first edge that connects two already-connected nodes is the answer (it creates the cycle).

```
public int[] findRedundantConnection(int[][] edges) {
    UnionFind uf = new UnionFind(edges.length + 1);

    for (int[] edge : edges) {
        if (!uf.union(edge[0], edge[1])) {
            return edge; // this edge connects already-connected nodes
        }
    }

    return new int[]{};
}
```

## Dry Run

edges = [[1,2],[1,3],[2,3]]

Union(1,2): 1 and 2 not connected → merge → components=2

Union(1,3): 1 and 3 not connected → merge → components=1

Union(2,3): find(2)=1, find(3)=1 → SAME ROOT → cycle! return [2,3]

## 7. Pattern Variations

| Problem                   | Union Find Role                       |
|---------------------------|---------------------------------------|
| Count components          | Count components field                |
| Detect cycle (undirected) | union() returns false = cycle         |
| Minimum spanning tree     | Kruskal's = sort edges + union        |
| Merge accounts/groups     | Union emails/elements together        |
| Percolation               | Top-to-bottom connectivity            |
| Swim in rising water      | Union cells as they become accessible |

## 8. Common Interview Mistakes

1. **Forgetting path compression** — Find without it is  $O(n)$  in worst case
2. **1-indexed vs 0-indexed nodes** — LC often uses 1-indexed; initialize size  $n+1$
3. **Union returning false = important signal** — this is how you detect cycles; don't ignore
4. **Not using rank** — without rank, union by random =  $O(\log n)$  instead of  $O(\alpha(n))$
5. **Accounts merge: unioning index 1 with all others** — index 0 is the name, not an email

## 9. Interview Cheat Sheet

## UNION FIND – MENTAL CHECKLIST

=====

- Dynamic connectivity queries? → Union Find
- Detect cycle in undirected? → union() returns false = cycle
- Count components? → track components field
- Group elements with shared property? → union them
- Kruskal's MST? → sort edges + union

## UNION FIND TEMPLATE (MEMORIZE)

=====

```
parent[i] = i (init)
rank[i] = 0 (init)
```

```
find(x): while parent[x]!=x: parent[x]=parent[parent[x]]; x=parent[x]
return x
```

```
union(x,y): rx=find(x), ry=find(y)
if rx==ry: return false (cycle/already connected)
attach smaller rank under larger
if equal: rank[rx]++
return true
```

## TRICKS

=====

- Path halving (parent[x]=parent[parent[x]]) ≈ full path compression, simpler
- Weighted/ranked union avoids degenerate chains
- UF + HashMap for string/object keys: map to integer IDs first
- Kruskal's: sort all edges by weight, union until n-1 edges added

---

## 10. Practice Roadmap

### Beginner:

- LC 547 — Number of Provinces
- LC 684 — Redundant Connection

### Intermediate:

- LC 200 — Number of Islands (UF approach)
- LC 721 — Accounts Merge
- LC 1202 — Smallest String with Swaps

### Hard:

- LC 685 — Redundant Connection II
- LC 1579 — Remove Max Number of Edges to Keep Graph Fully Traversable
- LC 839 — Similar String Groups

# Pattern 15 — Dijkstra's Algorithm + Kruskal / Prim's MST

## 1. Pattern Recognition Guide

### Dijkstra keywords:

- "shortest path in weighted graph"
- "minimum cost to reach", "cheapest flight"
- "network delay time", "minimum time to reach"

### MST keywords:

- "minimum spanning tree", "minimum cost to connect all nodes"
- "minimum cable length to connect all cities"
- "Kruskal", "Prim"

### Key constraints:

- Dijkstra: **non-negative edge weights only**. Negative weights → Bellman-Ford
- BFS for shortest path: **unweighted graphs only**
- Dijkstra: **single source** → all nodes. For all-pairs → Floyd-Warshall

## 2. Core Intuition

### Dijkstra's Algorithm

**Greedy insight:** Always extend the shortest known path next.

Use a **min-heap** (priority queue): process nodes in order of their current shortest distance from source.

When you pop a node, its distance is finalized (no shorter path can exist — all weights are non-negative).

Graph: 0→1 (4), 0→2 (1), 2→1 (2), 1→3 (1), 2→3 (5)

Source: 0

```
Initial: dist = [0, INF, INF, INF], heap = [(0,0)]
Pop (0,0): relax → 1: min(INF, 4)=4, 2: min(INF, 1)=1
    heap = [(1,2), (4,1)]
Pop (1,2): relax → 1: min(4, 1+2)=3, 3: min(INF, 1+5)=6
    heap = [(3,1), (4,1-stale), (6,3)]
Pop (3,1): relax → 3: min(6, 3+1)=4
    heap = [(4,1-stale), (4,3), (6,3-stale)]
Pop (4,1 - stale): dist[1]=3 ≠ 4 → SKIP
Pop (4,3): done, all processed
```

Result: [0, 3, 1, 4]

**Why skip stale entries?** Heap can have outdated entries. When we pop a node with a distance > its current best, skip it.

---

### 3. Generic Java Templates

#### Template A — Dijkstra's Shortest Path

```
public int[] dijkstra(int n, int[][] edges, int source) {
    // Build adjacency list: node → [(neighbor, weight)]
    List<int[]>[] adj = new List[n];
    for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
    for (int[] edge : edges) {
        adj[edge[0]].add(new int[]{edge[1], edge[2]});
        adj[edge[1]].add(new int[]{edge[0], edge[2]}); // undirected
    }

    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[source] = 0;

    // Min-heap: [distance, node]
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
    pq.offer(new int[]{0, source});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int d = curr[0], node = curr[1];

        // Skip stale entries
        if (d > dist[node]) continue;

        for (int[] neighbor : adj[node]) {
            int nextNode = neighbor[0], weight = neighbor[1];
            int newDist = dist[node] + weight;

            if (newDist < dist[nextNode]) {
                dist[nextNode] = newDist;
                pq.offer(new int[]{newDist, nextNode});
            }
        }
    }

    return dist;
}
```

#### Template B — Dijkstra to Specific Target (Early Exit)

```

public int dijkstraToTarget(int n, int[][] edges, int source, int target) {
    // ... (same setup as above)

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int d = curr[0], node = curr[1];

        if (node == target) return d; // found shortest path to target

        if (d > dist[node]) continue;

        // relax neighbors...
    }

    return -1; // unreachable
}

```

### Template C — Kruskal's MST (Minimum Spanning Tree)

```

public int kruskalMST(int n, int[][] edges) {
    // Sort edges by weight
    Arrays.sort(edges, (a, b) -> a[2] - b[2]);

    UnionFind uf = new UnionFind(n);
    int totalWeight = 0;
    int edgesUsed = 0;

    for (int[] edge : edges) {
        int u = edge[0], v = edge[1], w = edge[2];

        if (uf.union(u, v)) { // connect if not already connected
            totalWeight += w;
            edgesUsed++;
            if (edgesUsed == n - 1) break; // MST complete: n-1 edges
        }
    }

    return edgesUsed == n - 1 ? totalWeight : -1; // -1 if not connected
}

```

### Template D — Prim's MST (Greedy, Heap-based)

```

public int primMST(int n, int[][] edges) {
    List<int[]>[] adj = new List[n];
    for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
    for (int[] e : edges) {
        adj[e[0]].add(new int[]{e[1], e[2]});
        adj[e[1]].add(new int[]{e[0], e[2]});
    }

    boolean[] inMST = new boolean[n];
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
    pq.offer(new int[]{0, 0}); // [weight, node], start at node 0

    int totalWeight = 0, nodesAdded = 0;

    while (!pq.isEmpty() && nodesAdded < n) {
        int[] curr = pq.poll();
        int w = curr[0], node = curr[1];

        if (inMST[node]) continue; // already in MST
        inMST[node] = true;
        totalWeight += w;
        nodesAdded++;

        for (int[] neighbor : adj[node]) {
            if (!inMST[neighbor[0]]) {
                pq.offer(new int[]{neighbor[1], neighbor[0]});
            }
        }
    }

    return nodesAdded == n ? totalWeight : -1;
}

```

## 4. Complexity Cheatsheet

| Algorithm                 | Time                | Space           |
|---------------------------|---------------------|-----------------|
| Dijkstra (binary heap)    | $O((V + E) \log V)$ | $O(V + E)$      |
| Dijkstra (Fibonacci heap) | $O(E + V \log V)$   | — (rarely used) |
| Kruskal's                 | $O(E \log E)$       | $O(V)$          |
| Prim's (binary heap)      | $O((V + E) \log V)$ | $O(V + E)$      |
| Bellman-Ford              | $O(V \times E)$     | $O(V)$          |

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem            | Difficulty | Why Important    |
|------|--------------------|------------|------------------|
| 743  | Network Delay Time | Medium     | Classic Dijkstra |

| LC # | Problem                             | Difficulty | Why Important              |
|------|-------------------------------------|------------|----------------------------|
| 1631 | Path With Minimum Effort            | Medium     | Dijkstra on grid           |
| 787  | Cheapest Flights Within K Stops     | Medium     | Modified Dijkstra (BFS/DP) |
| 1135 | Connecting Cities With Minimum Cost | Medium     | Kruskal's MST              |
| 1584 | Min Cost to Connect All Points      | Medium     | Prim's or Kruskal's        |

## Advanced Variations

| LC # | Problem                                         | Difficulty | Why Important               |
|------|-------------------------------------------------|------------|-----------------------------|
| 778  | Swim in Rising Water                            | Hard       | Dijkstra on grid            |
| 1514 | Path with Maximum Probability                   | Medium     | Dijkstra with probability   |
| 882  | Reachable Nodes in Subdivided Graph             | Hard       | Modified Dijkstra           |
| 1168 | Optimize Water Distribution                     | Hard       | Kruskal's with virtual node |
| 1334 | Find the City with Smallest Number of Neighbors | Medium     | Floyd-Warshall              |

## 6. Solve Step-by-Step — LC 743: Network Delay Time

**Problem:** Find the time for a signal to reach ALL nodes from source k.

```

public int networkDelayTime(int[][] times, int n, int k) {
    List<int[]>[] adj = new List[n + 1];
    for (int i = 1; i <= n; i++) adj[i] = new ArrayList<>();
    for (int[] t : times) adj[t[0]].add(new int[]{t[1], t[2]});

    int[] dist = new int[n + 1];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[k] = 0;

    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
    pq.offer(new int[]{0, k});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int d = curr[0], node = curr[1];

        if (d > dist[node]) continue;

        for (int[] next : adj[node]) {
            int newDist = dist[node] + next[1];
            if (newDist < dist[next[0]]) {
                dist[next[0]] = newDist;
                pq.offer(new int[]{newDist, next[0]});
            }
        }
    }

    int maxDist = 0;
    for (int i = 1; i <= n; i++) {
        if (dist[i] == Integer.MAX_VALUE) return -1; // unreachable
        maxDist = Math.max(maxDist, dist[i]);
    }
    return maxDist; // time for signal to reach ALL nodes = max shortest distance
}

```

## 7. Pattern Variations

| Problem                             | Algorithm Choice                    |
|-------------------------------------|-------------------------------------|
| Shortest path, non-negative weights | Dijkstra                            |
| Shortest path, negative weights     | Bellman-Ford                        |
| Shortest path, unweighted           | BFS                                 |
| All-pairs shortest path             | Floyd-Warshall                      |
| Min cost to connect all nodes       | Kruskal's or Prim's                 |
| Max probability path                | Dijkstra (negate log probabilities) |
| Path with minimum max edge          | Dijkstra (minimize max seen so far) |

## 8. Common Interview Mistakes

1. **Dijkstra with negative weights** — doesn't work; Bellman-Ford handles this
  2. **Not skipping stale entries** in heap — causes incorrect dist updates
  3. **Kruskal's: forgetting MST needs exactly  $n-1$  edges** — graph may be disconnected
  4. **1-indexed nodes** — common in LC; initialize adj list of size  $n+1$
  5. **Prim's vs Kruskal's**: Prim's better for dense graphs ( $E \approx V^2$ ), Kruskal's for sparse
- 

## 9. Interview Cheat Sheet

### DIJKSTRA — MENTAL CHECKLIST

=====

- Weighted graph, non-negative weights? → Dijkstra
- Unweighted graph? → BFS (simpler)
- Negative weights? → Bellman-Ford
- Min spanning tree? → Kruskal (sparse) or Prim (dense)
- Always skip stale heap entries: if ( $d > \text{dist}[\text{node}]$ ) continue

### DIJKSTRA STRUCTURE

=====

```
dist[src] = 0; pq = [(0, src)]
while pq not empty:
    (d, node) = pq.poll()
    if d > dist[node]: continue ← CRITICAL
    for each (neighbor, weight) in adj[node]:
        if dist[node] + weight < dist[neighbor]:
            dist[neighbor] = dist[node] + weight
            pq.offer(dist[neighbor], neighbor)
```

### KRUSKAL'S STRUCTURE

=====

```
sort edges by weight
for each edge (u, v, w):
    if union(u, v): # not yet connected
        add to MST, totalWeight += w
    if edgesUsed == n-1: done
```

### TRICKS

=====

- Network delay = max of all shortest distances from source
  - "K stops" variant: use BFS with (cost, node, stops) tuple
  - Min effort path: Dijkstra where "distance" = max edge weight on path
  - Max probability: Dijkstra with log probabilities (negate to minimize)
- 

## 10. Practice Roadmap

### Beginner:

- LC 743 — Network Delay Time
- LC 1791 — Find Center of Star Graph

### Intermediate:

- LC 1631 — Path with Minimum Effort

- LC 1584 — Min Cost to Connect All Points
- LC 1514 — Path with Maximum Probability

**Hard:**

- LC 787 — Cheapest Flights Within K Stops
- LC 778 — Swim in Rising Water
- LC 1168 — Optimize Water Distribution in a Village

# Pattern 16 — Heaps: Top K, Two Heaps, Merge K Sorted

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "top k largest/smallest/frequent elements"
- "kth largest/smallest element"
- "median of a stream"
- "merge k sorted lists/arrays"
- "task scheduler", "reorganize string"
- "find median from data stream"

**When to reach for a Heap:**

- Need the k-th order statistic efficiently
- Need continuous access to the min or max
- Streaming/online problem — process as data arrives
- K-way merge

**Signal phrase:** `"K largest elements"` → Min-heap of size k

**Signal phrase:** `"Median of a stream"` → Two heaps (max-heap left + min-heap right)

## 2. Core Intuition

### Min-Heap for Top K Largest

Maintain a min-heap of size k. For each new element:

- If heap has fewer than k elements, add it
- If new element > heap's min (root), remove min, add new
- After processing all: heap contains the k largest

**Why min-heap?** The root is the smallest of our "top k" — if a new element beats it, it belongs in the top k.

### Two Heaps for Median

Keep elements in two halves:

- **Max-heap:** lower half (root = median candidate)
- **Min-heap:** upper half (root = median candidate)

Invariant: max-heap size == min-heap size OR max-heap has one more element.

- Odd count: median = max-heap root
- Even count: median = avg(max-heap root, min-heap root)

## 3. Generic Java Templates

## Template A — Top K Largest Elements

```
public int[] topKLargest(int[] nums, int k) {
    // Min-heap of size k: smallest of the k-largest sits at top
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) {
            minHeap.poll(); // remove smallest – doesn't belong in top k
        }
    }

    int[] result = new int[k];
    int i = k - 1;
    while (!minHeap.isEmpty()) result[i--] = minHeap.poll();
    return result;
}

// Kth Largest only
public int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) minHeap.poll();
    }
    return minHeap.peek(); // root = kth largest
}
```

## Template B — Top K Frequent Elements

```
public int[] topKFrequent(int[] nums, int k) {
    // Count frequencies
    Map<Integer, Integer> freq = new HashMap<>();
    for (int num : nums) freq.merge(num, 1, Integer::sum);

    // Min-heap by frequency
    PriorityQueue<int[]> minHeap = new PriorityQueue<>((a, b) -> a[1] - b[1]);

    for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {
        minHeap.offer(new int[]{entry.getKey(), entry.getValue()});
        if (minHeap.size() > k) minHeap.poll();
    }

    int[] result = new int[k];
    for (int i = 0; i < k; i++) result[i] = minHeap.poll()[0];
    return result;
}
```

## Template C — Two Heaps (Median Finder)

```

class MedianFinder {
    private PriorityQueue<Integer> maxHeap; // lower half
    private PriorityQueue<Integer> minHeap; // upper half

    public MedianFinder() {
        maxHeap = new PriorityQueue<>(Collections.reverseOrder()); // max at top
        minHeap = new PriorityQueue<>(); // min at top
    }

    public void addNum(int num) {
        // Always add to maxHeap first
        maxHeap.offer(num);

        // Balance: maxHeap root must be <= minHeap root
        if (!minHeap.isEmpty() && maxHeap.peek() > minHeap.peek()) {
            minHeap.offer(maxHeap.poll());
        }

        // Maintain size invariant: maxHeap has same or one more element
        if (maxHeap.size() > minHeap.size() + 1) {
            minHeap.offer(maxHeap.poll());
        } else if (minHeap.size() > maxHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
    }

    public double findMedian() {
        if (maxHeap.size() == minHeap.size()) {
            return (maxHeap.peek() + minHeap.peek()) / 2.0;
        }
        return maxHeap.peek(); // maxHeap has one extra
    }
}

```

## Template D — Merge K Sorted Lists

```

public ListNode mergeKLists(ListNode[] lists) {
    // Min-heap ordered by node value
    PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val);

    // Add head of each list
    for (ListNode list : lists) {
        if (list != null) pq.offer(list);
    }

    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;

    while (!pq.isEmpty()) {
        ListNode node = pq.poll(); // smallest current node
        curr.next = node;
        curr = curr.next;

        if (node.next != null) pq.offer(node.next); // add next from same list
    }

    return dummy.next;
}

```

## Template E — K Closest Points to Origin

```

public int[][] kClosest(int[][] points, int k) {
    // Max-heap by distance – keep only k closest (remove farthest)
    PriorityQueue<int[]> maxHeap = new PriorityQueue<>((
        (a, b) -> (b[0]*b[0]+b[1]*b[1]) - (a[0]*a[0]+a[1]*a[1])
    ));

    for (int[] point : points) {
        maxHeap.offer(point);
        if (maxHeap.size() > k) maxHeap.poll(); // remove farthest
    }

    return maxHeap.toArray(new int[k][]);
}

```

## 4. Complexity Cheatsheet

| Operation                         | Time          | Notes          |
|-----------------------------------|---------------|----------------|
| Heap push/pop                     | $O(\log n)$   |                |
| Build heap from $n$ elements      | $O(n)$        | heapify        |
| Top $K$ from $n$ elements         | $O(n \log k)$ | $k$ -size heap |
| Merge $K$ lists (total $N$ nodes) | $O(N \log K)$ | $K$ -size heap |
| Median add                        | $O(\log n)$   | two heaps      |
| Median query                      | $O(1)$        | peek tops      |

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                      | Difficulty | Why Important         |
|------|------------------------------|------------|-----------------------|
| 215  | Kth Largest Element in Array | Medium     | Min-heap of size k    |
| 347  | Top K Frequent Elements      | Medium     | Freq map + min-heap   |
| 295  | Find Median from Data Stream | Hard       | Two heaps             |
| 23   | Merge K Sorted Lists         | Hard       | K-way merge with heap |
| 973  | K Closest Points to Origin   | Medium     | Max-heap of size k    |

### Advanced Variations

| LC # | Problem                           | Difficulty | Why Important                |
|------|-----------------------------------|------------|------------------------------|
| 767  | Reorganize String                 | Medium     | Max-heap greedy              |
| 621  | Task Scheduler                    | Medium     | Max-heap + idle slots        |
| 1046 | Last Stone Weight                 | Easy       | Max-heap simulation          |
| 871  | Minimum Number of Refueling Stops | Hard       | Max-heap greedy              |
| 502  | IPO                               | Hard       | Two heaps for profit/capital |

## 6. Solve Step-by-Step — LC 295: Find Median from Data Stream

**Problem:** Implement MedianFinder with addNum() and findMedian().

### Key Insight

Two heaps create a balanced partition. The tops of both heaps give us the median in  $O(1)$ .

(Full implementation already in Template C)

### Dry Run

```

addNum(1): maxHeap=[1], minHeap=[]
findMedian(): maxHeap.size > minHeap.size → return 1.0

addNum(2): add 2 to maxHeap=[2,1] → 2>no minHeap → ok
  sizes: max=2, min=0 → max too big → move max(2) to min
  maxHeap=[1], minHeap=[2]
findMedian(): equal sizes → (1+2)/2.0 = 1.5

addNum(3): add 3 to maxHeap=[3,1] → 3 > minHeap.peek(2) → move 3 to min
  maxHeap=[1], minHeap=[2,3]
  sizes: max=1, min=2 → min too big → move min(2) to max
  maxHeap=[2,1], minHeap=[3]
findMedian(): maxHeap.size > minHeap.size → return 2.0

```

---

## 7. Pattern Variations

| Problem           | Heap Choice                       |
|-------------------|-----------------------------------|
| Top K largest     | Min-heap of size k                |
| Top K smallest    | Max-heap of size k                |
| Kth largest       | Min-heap of size k, peek root     |
| K closest         | Max-heap of size k (by distance)  |
| K most frequent   | Min-heap of size k (by frequency) |
| Median stream     | Two heaps (max-heap + min-heap)   |
| Merge K sorted    | Min-heap with K elements          |
| Greedy scheduling | Max-heap by frequency/priority    |

---

## 8. Common Interview Mistakes

- 1. Min-heap for top K smallest** → actually use max-heap of size k (remove max)
  - 2. Two heaps: forgetting to rebalance after adding** → median computation wrong
  - 3. Two heaps: value vs comparator direction** — Java's PriorityQueue is min by default; max needs `Collections.reverseOrder()`
  - 4. Merge K: not null-checking** before adding to heap
  - 5. Task scheduler: not understanding idle slots** — frequency-based, not just sorting
- 

## 9. Interview Cheat Sheet

## HEAP PATTERNS – MENTAL CHECKLIST

=====

- Top K LARGEST → min-heap size k (removes smallest → keeps largest)
- Top K SMALLEST → max-heap size k (removes largest → keeps smallest)
- Kth LARGEST → min-heap size k, peek()
- Median stream → two heaps (max-heap left, min-heap right)
- Merge K sorted → min-heap with one node from each list

## JAVA HEAP SYNTAX

=====

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>(); // default min
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<int[]> custom = new PriorityQueue<>((a,b) -> a[1]-b[1]); // by index 1
```

## TWO HEAPS INVARIANT

=====

maxHeap (lower half) ≤ all elements in minHeap (upper half)  
|maxHeap.size - minHeap.size| ≤ 1  
median = maxHeap.peek() if odd, avg of both tops if even

## TRICKS

=====

- Always keep k-size heap →  $O(n \log k)$  not  $O(n \log n)$
- For custom objects: sort by one field, break ties by another in comparator
- Greedy scheduling: always pick highest frequency remaining task
- K-way merge: only K elements in heap at once →  $O(N \log K)$

---

## 10. Practice Roadmap

### Beginner:

- LC 1046 — Last Stone Weight
- LC 215 — Kth Largest Element in Array

### Intermediate:

- LC 347 — Top K Frequent Elements
- LC 973 — K Closest Points to Origin
- LC 767 — Reorganize String
- LC 621 — Task Scheduler

### Hard:

- LC 295 — Find Median from Data Stream
- LC 23 — Merge K Sorted Lists
- LC 871 — Minimum Number of Refueling Stops
- LC 502 — IPO

# Pattern 17 — Binary Search (All Variants)

---

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "sorted array", "rotated sorted array"
- "find minimum / maximum that satisfies condition"
- "search in log n", "find target"
- "first/last occurrence", "leftmost/rightmost"
- "capacity", "speed", "days" — binary search on answer space

**Binary Search on Values (Answer Space):**

- "Minimum speed to arrive on time"
- "Minimum capacity to ship in D days"
- "Koko eating bananas" — any "find minimum X such that condition holds"

**Signal phrase:** `"Sorted array, find target"` → Classic Binary Search

**Signal phrase:** `"Find minimum X such that f(X) is true"` → Binary Search on range

---

## 2. Core Intuition

**Core idea:** Eliminate half the search space with each comparison.

The critical insight for all variants: **identify the invariant** — what is always true about where `left` and `right` point?

**Template trap:** Off-by-one errors kill binary search solutions. Pick ONE template and memorize it perfectly.

Classic: find target in `[1,3,5,7,9]`

`0 1 2 3 4`

`left=0, right=4`

`mid=2, arr[2]=5 > 3 → right = mid-1 = 1`

`left=0, right=1`

`mid=0, arr[0]=1 < 3 → left = mid+1 = 1`

`left=1, right=1`

`mid=1, arr[1]=3 == target → return 1 ✓`

---

## 3. Generic Java Templates

**Template A — Classic Binary Search (Find Exact Target)**

```

public int binarySearch(int[] nums, int target) {
    int left = 0, right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2; // avoid overflow vs (left+right)/2

        if (nums[mid] == target) {
            return mid;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1; // not found
}

```

### Template B — Left-Most Occurrence (First True Position)

```

// Returns index of first element >= target (lower bound)
// Returns nums.length if all elements < target
public int lowerBound(int[] nums, int target) {
    int left = 0, right = nums.length; // right = nums.length (open end)

    while (left < right) { // NOTE: < not <=
        int mid = left + (right - left) / 2;

        if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid; // mid could be the answer, don't exclude it
        }
    }

    return left; // left == right = first position where nums[i] >= target
}

// First occurrence of exact target
public int firstOccurrence(int[] nums, int target) {
    int idx = lowerBound(nums, target);
    if (idx < nums.length && nums[idx] == target) return idx;
    return -1;
}

```

### Template C — Right-Most Occurrence (Last True Position)

```
// Returns index of last element <= target (upper bound - 1)
public int lastOccurrence(int[] nums, int target) {
    int left = 0, right = nums.length;

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] <= target) {
            left = mid + 1; // mid is valid but maybe not the last
        } else {
            right = mid;
        }
    }

    // left-1 = last index where nums[i] <= target
    int idx = left - 1;
    if (idx >= 0 && nums[idx] == target) return idx;
    return -1;
}
```

## Template D — Binary Search on Answer Space

```
// Find minimum value in [lo, hi] such that condition(mid) is true
// Requires: once condition is true, it stays true (monotonic)
public int binarySearchOnRange(int lo, int hi) {
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;

        if (condition(mid)) {
            hi = mid; // mid could be the answer, search left half
        } else {
            lo = mid + 1; // mid is too small, search right half
        }
    }

    return lo; // lo == hi = minimum value satisfying condition
}

// Example: Koko eating bananas (LC 875)
private boolean canEat(int[] piles, int speed, int h) {
    int hours = 0;
    for (int pile : piles) hours += (pile + speed - 1) / speed; // ceil division
    return hours <= h;
}

public int minEatingSpeed(int[] piles, int h) {
    int lo = 1, hi = Arrays.stream(piles).max().getAsInt();

    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canEat(piles, mid, h)) hi = mid;
        else lo = mid + 1;
    }

    return lo;
}
```

## Template E — Search in Rotated Sorted Array

```
public int searchRotated(int[] nums, int target) {
    int left = 0, right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) return mid;

        // Determine which half is sorted
        if (nums[left] <= nums[mid]) { // left half is sorted
            if (nums[left] <= target && target < nums[mid]) {
                right = mid - 1; // target in left sorted half
            } else {
                left = mid + 1;
            }
        } else { // right half is sorted
            if (nums[mid] < target && target <= nums[right]) {
                left = mid + 1; // target in right sorted half
            } else {
                right = mid - 1;
            }
        }
    }

    return -1;
}
```

## Template F — Binary Search on 2D Matrix

```
// Matrix where each row and col is sorted
public boolean searchMatrix(int[][] matrix, int target) {
    int rows = matrix.length, cols = matrix[0].length;
    int left = 0, right = rows * cols - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        int val = matrix[mid / cols][mid % cols]; // convert 1D index to 2D

        if (val == target) return true;
        else if (val < target) left = mid + 1;
        else right = mid - 1;
    }

    return false;
}
```

---

## 4. Complexity Cheatsheet

| Variant                           | Time                      | Space  |
|-----------------------------------|---------------------------|--------|
| Classic binary search             | $O(\log n)$               | $O(1)$ |
| Binary search on range $[lo, hi]$ | $O(\log(hi-lo) \times f)$ | $O(1)$ |

| Variant                 | Time                  | Space  |
|-------------------------|-----------------------|--------|
| 2D matrix binary search | $O(\log(m \times n))$ | $O(1)$ |

f = cost of condition check per iteration.

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                        | Difficulty | Why Important           |
|------|--------------------------------|------------|-------------------------|
| 704  | Binary Search                  | Easy       | Classic template        |
| 34   | Find First and Last Position   | Medium     | Left + right bound      |
| 33   | Search in Rotated Sorted Array | Medium     | Rotated variant         |
| 74   | Search a 2D Matrix             | Medium     | 2D flattening           |
| 875  | Koko Eating Bananas            | Medium     | Binary search on answer |

### Advanced Variations

| LC # | Problem                              | Difficulty | Why Important              |
|------|--------------------------------------|------------|----------------------------|
| 153  | Find Minimum in Rotated Sorted Array | Medium     | Rotated min                |
| 162  | Find Peak Element                    | Medium     | Converge to peak           |
| 1011 | Capacity to Ship Packages in D Days  | Medium     | Binary search on capacity  |
| 410  | Split Array Largest Sum              | Hard       | Binary search on answer    |
| 4    | Median of Two Sorted Arrays          | Hard       | Binary search on partition |

## 6. Solve Step-by-Step — LC 34: Find First and Last Position

**Problem:** Find [first occurrence, last occurrence] of target in sorted array.  $O(\log n)$ .

```

public int[] searchRange(int[] nums, int target) {
    return new int[]{firstOccurrence(nums, target), lastOccurrence(nums, target)};
}

private int firstOccurrence(int[] nums, int target) {
    int left = 0, right = nums.length;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] < target) left = mid + 1;
        else right = mid;
    }
    return (left < nums.length && nums[left] == target) ? left : -1;
}

private int lastOccurrence(int[] nums, int target) {
    int left = 0, right = nums.length;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] <= target) left = mid + 1;
        else right = mid;
    }
    int idx = left - 1;
    return (idx >= 0 && nums[idx] == target) ? idx : -1;
}

```

### Dry Run: [5,7,7,8,8,10] , target=8

First occurrence:

```

left=0, right=6
mid=3, nums[3]=8 >= 8 → right=3
mid=1, nums[1]=7 < 8 → left=2
mid=2, nums[2]=7 < 8 → left=3
left=right=3 → nums[3]=8 ✓ → return 3

```

Last occurrence:

```

left=0, right=6
mid=3, nums[3]=8 <= 8 → left=4
mid=5, nums[5]=10 > 8 → right=5
mid=4, nums[4]=8 <= 8 → left=5
left=right=5 → idx=4, nums[4]=8 ✓ → return 4

```

Result: [3, 4] ✓

## 7. Pattern Variations

| Problem          | Binary Search On              | Condition                      |
|------------------|-------------------------------|--------------------------------|
| Classic find     | Array index                   | arr[mid] == target             |
| First occurrence | Array index                   | arr[mid] >= target → go left   |
| Last occurrence  | Array index                   | arr[mid] <= target → go right  |
| Rotated array    | Array index                   | determine which half is sorted |
| 2D matrix        | 1D index (mid/cols, mid%cols) | standard                       |

| Problem              | Binary Search On        | Condition                 |
|----------------------|-------------------------|---------------------------|
| Koko / ship capacity | Answer range [1, max]   | can complete in time?     |
| Split array          | Answer range [max, sum] | can split within limit?   |
| Peak finding         | Array index             | go toward higher neighbor |

## 8. Common Interview Mistakes

1. `left + right) / 2` **overflow** — always use `left + (right - left) / 2`
2. `while (left <= right)` **vs** `while (left < right)` — mixing templates causes infinite loops
3. `right = mid` **vs** `right = mid - 1` — depends on template; don't mix
4. **Rotated array: not handling** `nums[left] == nums[mid]` — with duplicates, need `left++`
5. **Binary search on answer: wrong lo/hi initialization** — lo must be minimum valid answer, hi must be maximum
6. **Forgetting to validate result** after lower bound binary search — `nums[left]` might not equal target

## 9. Interview Cheat Sheet

## BINARY SEARCH – MENTAL CHECKLIST

=====

- Sorted (or monotonic) property? → Binary Search
- Find exact target? → Template A ( $\text{left} \leq \text{right}$ )
- First/last occurrence? → Template B/C ( $\text{left} < \text{right}$ ,  $\text{right} = \text{nums.length}$ )
- Min/max satisfying condition? → Template D on answer range
- Rotated array? → Template E (check which half is sorted)
- 2D matrix? → Flatten to 1D:  $\text{val} = \text{matrix}[\text{mid}/\text{cols}][\text{mid}\% \text{cols}]$

## TWO TEMPLATES TO MEMORIZE

=====

### 1. Exact search ( $\text{left} \leq \text{right}$ ):

```
mid = left + (right-left)/2
if found: return mid
if too small: left = mid+1
if too large: right = mid-1
return -1
```

### 2. Left bound / answer range ( $\text{left} < \text{right}$ ):

```
mid = left + (right-left)/2
if condition(mid): right = mid ← mid could be answer
else: left = mid+1
return left
```

## ANSWER SPACE BINARY SEARCH

=====

```
lo = minimum possible answer
hi = maximum possible answer
condition: is mid a valid answer?
→ find minimum valid answer
```

## TRICKS

=====

- Ceil division:  $(n + d - 1) / d$  used in shipping/banana problems
- Peak finding: move toward higher neighbor (both directions valid)
- Count of elements  $< \text{target} = \text{lowerBound}(\text{target})$  index
- Count of target occurrences =  $\text{upperBound} - \text{lowerBound}$

---

## 10. Practice Roadmap

### Beginner:

- LC 704 — Binary Search
- LC 35 — Search Insert Position
- LC 278 — First Bad Version

### Intermediate:

- LC 34 — Find First and Last Position
- LC 33 — Search in Rotated Sorted Array
- LC 875 — Koko Eating Bananas
- LC 1011 — Capacity to Ship Packages
- LC 74 — Search a 2D Matrix

### Hard:

- LC 4 — Median of Two Sorted Arrays
- LC 410 — Split Array Largest Sum
- LC 154 — Find Minimum in Rotated Sorted Array II (duplicates)
- LC 668 — Kth Smallest Number in Multiplication Table

# Pattern 18 — Dynamic Programming (Complete Guide)

---

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "maximum/minimum" with choices
- "number of ways", "count paths"
- "longest subsequence/substring"
- "can you achieve X", "is it possible"
- "overlapping subproblems" — same calculation needed multiple times

**When NOT to use DP:**

- Greedy works (no "choices" to make — always one obvious next step)
- The problem has no overlapping subproblems (divide and conquer)

**The two DP questions:**

1. What is my **state**? (What info do I need to define a subproblem?)
  2. What is my **recurrence**? (How do I compute state from smaller states?)
- 

## 2. Core Intuition

**Memoization (Top-Down) vs Tabulation (Bottom-Up)**

**Memoization:** Write the recursive solution, cache results.

- Easier to think about
- Only computes needed states
- Recursion overhead; stack overflow risk

**Tabulation:** Fill a table iteratively from base cases to answer.

- Faster in practice (no recursion overhead)
  - Must compute all states (some may be unnecessary)
  - No stack overflow risk
- 

## 3. DP Patterns and Templates

**Pattern 1 — Fibonacci / Linear DP**

```
// Climbing Stairs: n steps, climb 1 or 2 at a time – how many ways?
public int climbStairs(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1; // O(1) space, O(n) time
}
```

## Pattern 2 — 0/1 Knapsack

**State:**  $dp[i][w]$  = max value using first  $i$  items with capacity  $w$

**Recurrence:**

- Don't take item  $i$ :  $dp[i-1][w]$
- Take item  $i$  (if  $w \geq \text{weight}[i]$ ):  $dp[i-1][w-\text{weight}[i]] + \text{value}[i]$
- $dp[i][w] = \max(\text{above two})$

```
public int knapsack(int[] weights, int[] values, int capacity) {
    int n = weights.length;
    int[] dp = new int[capacity + 1]; // space-optimized (1D)

    for (int i = 0; i < n; i++) {
        // Traverse right-to-left to avoid using item i twice
        for (int w = capacity; w >= weights[i]; w--) {
            dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }

    return dp[capacity];
}
```

## Pattern 3 — Unbounded Knapsack (Coin Change)

**State:**  $dp[\text{amount}]$  = min coins to make amount

**Recurrence:** for each coin  $c$ :  $dp[\text{amount}] = \min(dp[\text{amount}], dp[\text{amount}-c] + 1)$

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1); // sentinel "infinity"
    dp[0] = 0;

    for (int a = 1; a <= amount; a++) {
        for (int coin : coins) {
            if (coin <= a) {
                dp[a] = Math.min(dp[a], dp[a - coin] + 1);
            }
        }
    }

    return dp[amount] > amount ? -1 : dp[amount];
}
```

## Pattern 4 — Longest Common Subsequence (LCS)

**State:**  $dp[i][j]$  = LCS length of  $s1[0..i-1]$  and  $s2[0..j-1]$

**Recurrence:**

- If  $s1[i-1] == s2[j-1]$  :  $dp[i][j] = dp[i-1][j-1] + 1$
- Else:  $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

```
public int longestCommonSubsequence(String s1, String s2) {
    int m = s1.length(), n = s2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s1.charAt(i-1) == s2.charAt(j-1)) {
                dp[i][j] = dp[i-1][j-1] + 1;
            } else {
                dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }

    return dp[m][n];
}
```

## Pattern 5 — Longest Increasing Subsequence (LIS)

**State:**  $dp[i]$  = length of LIS ending at index  $i$

**Recurrence:** for  $j < i$ : if  $nums[j] < nums[i]$  :  $dp[i] = \max(dp[i], dp[j] + 1)$

```
// O(n2) approach
public int lengthOfLIS(int[] nums) {
    int n = nums.length;
    int[] dp = new int[n];
    Arrays.fill(dp, 1); // each element is a subsequence of length 1
    int maxLen = 1;

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                dp[i] = Math.max(dp[i], dp[j] + 1);
            }
        }
        maxLen = Math.max(maxLen, dp[i]);
    }
    return maxLen;
}

// O(n log n) with patience sorting + binary search
public int lengthOfLISOptimal(int[] nums) {
    List<Integer> tails = new ArrayList<>();
    for (int num : nums) {
        int pos = Collections.binarySearch(tails, num);
        if (pos < 0) pos = -(pos + 1); // insertion point
        if (pos == tails.size()) tails.add(num);
        else tails.set(pos, num);
    }
    return tails.size();
}
```

## Pattern 6 — Palindrome DP

```
// Longest Palindromic Subsequence
public int longestPalindromeSubseq(String s) {
    int n = s.length();
    int[][] dp = new int[n][n];

    // Base: single chars are palindromes of length 1
    for (int i = 0; i < n; i++) dp[i][i] = 1;

    // Fill by increasing length
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i <= n - len; i++) {
            int j = i + len - 1;
            if (s.charAt(i) == s.charAt(j)) {
                dp[i][j] = dp[i+1][j-1] + 2;
            } else {
                dp[i][j] = Math.max(dp[i+1][j], dp[i][j-1]);
            }
        }
    }

    return dp[0][n-1];
}
```

## Pattern 7 — 2D Grid DP

```
// Unique paths in m×n grid (only right/down moves)
public int uniquePaths(int m, int n) {
    int[] dp = new int[n];
    Arrays.fill(dp, 1); // first row: all 1s

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[j] += dp[j-1]; // dp[j] = paths from above + paths from left
        }
    }

    return dp[n-1];
}
```

## 4. Complexity Cheatsheet

| Pattern               | Time            | Space           | Optimized Space     |
|-----------------------|-----------------|-----------------|---------------------|
| Linear (Fibonacci)    | $O(n)$          | $O(n)$          | $O(1)$ — two vars   |
| 0/1 Knapsack          | $O(n \times W)$ | $O(n \times W)$ | $O(W)$ — 1D array   |
| Unbounded Knapsack    | $O(n \times W)$ | $O(W)$          | Already 1D          |
| LCS                   | $O(m \times n)$ | $O(m \times n)$ | $O(\min(m, n))$     |
| LIS ( $O(n^2)$ )      | $O(n^2)$        | $O(n)$          | —                   |
| LIS ( $O(n \log n)$ ) | $O(n \log n)$   | $O(n)$          | —                   |
| Palindrome DP         | $O(n^2)$        | $O(n^2)$        | $O(n)$ with rolling |
| Grid DP               | $O(m \times n)$ | $O(m \times n)$ | $O(n)$ — 1D rolling |

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                        | Difficulty | Pattern            |
|------|--------------------------------|------------|--------------------|
| 70   | Climbing Stairs                | Easy       | Linear             |
| 322  | Coin Change                    | Medium     | Unbounded Knapsack |
| 1143 | Longest Common Subsequence     | Medium     | LCS                |
| 300  | Longest Increasing Subsequence | Medium     | LIS                |
| 416  | Partition Equal Subset Sum     | Medium     | 0/1 Knapsack       |

### Advanced Variations

| LC # | Problem                         | Difficulty | Pattern             |
|------|---------------------------------|------------|---------------------|
| 72   | Edit Distance                   | Medium     | 2D DP (LCS variant) |
| 516  | Longest Palindromic Subsequence | Medium     | Palindrome DP       |
| 312  | Burst Balloons                  | Hard       | Interval DP         |
| 1143 | Longest Common Subsequence      | Medium     | LCS                 |
| 188  | Best Time to Buy Sell Stock IV  | Hard       | State machine DP    |

## 6. Solve Step-by-Step — LC 322: Coin Change

**Problem:** Minimum coins to make amount. Unlimited coins of each denomination.

### Brute Force — Exponential

Try all combinations of coins recursively.

### Better — Memoized DFS

Cache `memo[amount]` = min coins for each sub-amount.

### Optimal — Tabulation (Bottom-Up)

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1); // anything > amount = impossible sentinel
    dp[0] = 0; // base case: 0 coins to make amount 0

    for (int a = 1; a <= amount; a++) {
        for (int coin : coins) {
            if (coin <= a) {
                dp[a] = Math.min(dp[a], dp[a - coin] + 1);
            }
        }
    }

    return dp[amount] > amount ? -1 : dp[amount];
}
```

**Dry Run:** `coins=[1,2,5]`, `amount=11`

```

dp[0]=0
dp[1]=min(dp[0]+1)=1 (use coin 1)
dp[2]=min(dp[1]+1, dp[0]+1)=1 (use coin 2)
dp[3]=min(dp[2]+1, dp[1]+1)=2 (use 1+2)
...
dp[5]=min(dp[4]+1, dp[3]+1, dp[0]+1)=1 (use coin 5)
dp[6]=min(dp[5]+1, dp[4]+1, dp[1]+1)=2
...
dp[11]=min(dp[10]+1, dp[9]+1, dp[6]+1)=3 (5+5+1)

```

Answer: 3 ✓

---

## 7. DP Problem-Solving Framework

Step 1: Define the STATE clearly

"dp[i] represents..."

"dp[i][j] represents..."

Step 2: Write the RECURRENCE

"dp[i] depends on dp[i-1] because..."

Step 3: Identify BASE CASES

"dp[0] = ..., dp[1] = ..."

Step 4: Determine ITERATION ORDER

"We need dp[i-1] before dp[i], so iterate left to right"

Step 5: EXTRACT ANSWER

"The answer is dp[n] / dp[n-1] / max(dp)"

Step 6: OPTIMIZE SPACE if possible

"We only need previous row → use 1D rolling array"

---

## 8. Common Interview Mistakes

- 1. Wrong state definition** — most DP bugs come from an imprecise state
- 2. Forgetting base cases** — off-by-one errors from uninitialized dp[0]
- 3. 0/1 knapsack: iterating left-to-right** — causes double-counting (use item multiple times); must go right-to-left
- 4. Unbounded knapsack: iterating right-to-left** — prevents reusing coins; must go left-to-right
- 5. LCS: using i, j without +1 offset** —  $dp[i][j]$  = LCS of first i chars, NOT index i
- 6. Palindrome: wrong subproblem order** — must fill by length, not by i or j

---

## 9. Interview Cheat Sheet

## DP — MENTAL CHECKLIST

=====

- "Maximum/minimum with choices" → DP (not greedy)
- "Number of ways" → DP (count paths)
- "Longest subsequence" → LCS or LIS template
- "Can you make sum X" → Knapsack template
- "Palindrome" → Expand from center or  $dp[i][j]$

## STATE DEFINITION TEMPLATES

=====

$dp[i]$  → best answer using first  $i$  elements

$dp[i][j]$  → best answer for substring  $s[i..j]$

$dp[i][j]$  → best answer for first  $i$  items with constraint  $j$

$dp[i][j]$  → LCS/edit distance of  $s1[0..i]$  and  $s2[0..j]$

## KNAPSACK DIRECTION

=====

0/1 Knapsack: inner loop RIGHT TO LEFT (no reuse)

Unbounded: inner loop LEFT TO RIGHT (allow reuse)

## COMMON RECURRENCES

=====

LCS:  $dp[i][j] = dp[i-1][j-1] + 1$  if match, else  $\max(dp[i-1][j], dp[i][j-1])$

LIS:  $dp[i] = \max(dp[j] + 1)$  for all  $j < i$  where  $nums[j] < nums[i]$

Edit:  $dp[i][j] = dp[i-1][j-1]$  if match, else  $1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$

Coin:  $dp[a] = \min(dp[a - \text{coin}] + 1)$  for all valid coins

---

## 10. Practice Roadmap

### Beginner:

- LC 70 — Climbing Stairs
- LC 509 — Fibonacci Number
- LC 198 — House Robber
- LC 746 — Min Cost Climbing Stairs

### Intermediate:

- LC 322 — Coin Change
- LC 300 — LIS
- LC 1143 — LCS
- LC 416 — Partition Equal Subset Sum
- LC 62 — Unique Paths
- LC 72 — Edit Distance

### Hard:

- LC 312 — Burst Balloons (Interval DP)
- LC 188 — Best Time to Buy/Sell Stock IV
- LC 1335 — Minimum Difficulty of a Job Schedule
- LC 32 — Longest Valid Parentheses

# Pattern 19 — Backtracking (Permutations, Subsets, Combinations, N-Queens)

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "all permutations", "all subsets", "all combinations"
- "generate all valid", "find all solutions"
- "word search in grid", "path in maze"
- "N-Queens", "Sudoku solver"

**When to reach for Backtracking:**

- Need ALL solutions, not just one
- Making choices at each step, need to undo choices
- State-space search with pruning

**Signal phrase:** \_"Generate all permutations"\_ → Backtracking

**Signal phrase:** \_"Does a valid arrangement exist?"\_ → Backtracking with early return

## 2. Core Intuition

**Backtracking = DFS + Undo**

At each step:

1. **Choose:** make a decision (add element, place queen, move in maze)
2. **Explore:** recurse with this decision
3. **Unchoose:** undo the decision (backtrack)

**The key insight:** We build a solution incrementally. If we detect the current partial solution can't lead to a valid complete solution, we prune the branch and backtrack.

Permutations of [1,2,3]:

```
[ ]
/ | \
[1] [2] [3]
/ \ / \ / \
[1,2] [1,3] [2,1] [2,3] [3,1] [3,2]
| | | | | |
[1,2,3] [1,3,2] [2,1,3] [2,3,1] [3,1,2] [3,2,1]
```

## 3. Generic Java Templates

**Template A — Subsets (Power Set)**

```

public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] nums, int start, List<Integer> current,
    List<List<Integer>> result) {
    // Add current subset at every node (including empty)
    result.add(new ArrayList<>(current));

    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]); // choose
        backtrack(nums, i + 1, current, result); // explore
        current.remove(current.size() - 1); // unchoose
    }
}

```

### Template B — Combinations (Choose K from N)

```

public List<List<Integer>> combine(int n, int k) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(1, n, k, new ArrayList<>(), result);
    return result;
}

private void backtrack(int start, int n, int k, List<Integer> current,
    List<List<Integer>> result) {
    if (current.size() == k) { // found a valid combination
        result.add(new ArrayList<>(current));
        return;
    }

    // Pruning: remaining elements must be enough to fill k slots
    for (int i = start; i <= n - (k - current.size()) + 1; i++) {
        current.add(i);
        backtrack(i + 1, n, k, current, result);
        current.remove(current.size() - 1);
    }
}

```

### Template C — Permutations (No Duplicates)

```

public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, new boolean[nums.length], new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] nums, boolean[] used, List<Integer> current,
    List<List<Integer>> result) {
    if (current.size() == nums.length) {
        result.add(new ArrayList<>(current));
        return;
    }

    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue; // skip already used elements

        used[i] = true;
        current.add(nums[i]); // choose
        backtrack(nums, used, current, result);
        current.remove(current.size() - 1); // unchoose
        used[i] = false;
    }
}

```

### Template D — Permutations With Duplicates (Sort + Skip)

```

public List<List<Integer>> permuteUnique(int[] nums) {
    Arrays.sort(nums); // CRITICAL: sort to group duplicates
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, new boolean[nums.length], new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] nums, boolean[] used, List<Integer> current,
    List<List<Integer>> result) {
    if (current.size() == nums.length) {
        result.add(new ArrayList<>(current));
        return;
    }

    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;

        // Skip duplicate: same value at same depth level
        // used[i-1] == false means we already explored and backtracked nums[i-1]
        if (i > 0 && nums[i] == nums[i-1] && !used[i-1]) continue;

        used[i] = true;
        current.add(nums[i]);
        backtrack(nums, used, current, result);
        current.remove(current.size() - 1);
        used[i] = false;
    }
}

```

### Template E — Combination Sum (Unlimited Use, With Duplicates Variant)

```

// LC 39: each number can be used unlimited times
public List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> result = new ArrayList<>();
    Arrays.sort(candidates);
    backtrack(candidates, target, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] candidates, int remaining, int start,
    List<Integer> current, List<List<Integer>> result) {
    if (remaining == 0) {
        result.add(new ArrayList<>(current));
        return;
    }

    for (int i = start; i < candidates.length; i++) {
        if (candidates[i] > remaining) break; // pruning: sorted, rest will be too large

        current.add(candidates[i]);
        backtrack(candidates, remaining - candidates[i], i, current, result); // i (not i+1) = reuse
        current.remove(current.size() - 1);
    }
}

```

## Template F — N-Queens

```

public List<List<String>> solveNQueens(int n) {
    List<List<String>> result = new ArrayList<>();
    int[] queens = new int[n]; // queens[row] = col position
    Arrays.fill(queens, -1);

    Set<Integer> cols = new HashSet<>();
    Set<Integer> diag1 = new HashSet<>(); // row - col
    Set<Integer> diag2 = new HashSet<>(); // row + col

    backtrack(queens, n, 0, cols, diag1, diag2, result);
    return result;
}

private void backtrack(int[] queens, int n, int row,
    Set<Integer> cols, Set<Integer> diag1, Set<Integer> diag2,
    List<List<String>> result) {
    if (row == n) {
        result.add(buildBoard(queens, n));
        return;
    }

    for (int col = 0; col < n; col++) {
        if (cols.contains(col) || diag1.contains(row - col) || diag2.contains(row + col)) {
            continue; // queen attacks this position
        }

        queens[row] = col;
        cols.add(col); diag1.add(row - col); diag2.add(row + col);

        backtrack(queens, n, row + 1, cols, diag1, diag2, result);

        queens[row] = -1;
        cols.remove(col); diag1.remove(row - col); diag2.remove(row + col);
    }
}

private List<String> buildBoard(int[] queens, int n) {
    List<String> board = new ArrayList<>();
    for (int row = 0; row < n; row++) {
        char[] line = new char[n];
        Arrays.fill(line, '.');
        line[queens[row]] = 'Q';
        board.add(new String(line));
    }
    return board;
}

```

## Template G — Word Search in Grid

```

public boolean exist(char[][] board, String word) {
    int rows = board.length, cols = board[0].length;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (dfs(board, word, r, c, 0)) return true;
        }
    }
    return false;
}

private boolean dfs(char[][] board, String word, int r, int c, int idx) {
    if (idx == word.length()) return true; // all chars matched
    if (r < 0 || r >= board.length || c < 0 || c >= board[0].length) return false;
    if (board[r][c] != word.charAt(idx)) return false;

    char temp = board[r][c];
    board[r][c] = '#'; // mark visited

    boolean found = dfs(board, word, r+1, c, idx+1) ||
        dfs(board, word, r-1, c, idx+1) ||
        dfs(board, word, r, c+1, idx+1) ||
        dfs(board, word, r, c-1, idx+1);

    board[r][c] = temp; // unmark (backtrack)
    return found;
}

```

## 4. Complexity Cheatsheet

| Problem               | Time                       | Space  | Notes                          |
|-----------------------|----------------------------|--------|--------------------------------|
| Subsets               | $O(2^n \times n)$          | $O(n)$ | $2^n$ subsets, $n$ to copy     |
| Permutations          | $O(n! \times n)$           | $O(n)$ | $n!$ permutations, $n$ to copy |
| Combinations $C(n,k)$ | $O(C(n,k) \times k)$       | $O(k)$ |                                |
| N-Queens              | $O(n!)$                    | $O(n)$ |                                |
| Word Search           | $O(m \times n \times 4^L)$ | $O(L)$ | $L$ = word length              |

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem         | Difficulty | Pattern                 |
|------|-----------------|------------|-------------------------|
| 78   | Subsets         | Medium     | Base backtracking       |
| 46   | Permutations    | Medium     | Used-array backtracking |
| 39   | Combination Sum | Medium     | Unlimited reuse         |
| 51   | N-Queens        | Hard       | Constraint checking     |

| LC # | Problem     | Difficulty | Pattern              |
|------|-------------|------------|----------------------|
| 79   | Word Search | Medium     | Grid DFS + backtrack |

## Advanced Variations

| LC # | Problem                      | Difficulty | Pattern                      |
|------|------------------------------|------------|------------------------------|
| 90   | Subsets II (duplicates)      | Medium     | Sort + skip                  |
| 47   | Permutations II (duplicates) | Medium     | Sort + skip                  |
| 40   | Combination Sum II           | Medium     | Sort + skip                  |
| 131  | Palindrome Partitioning      | Medium     | Backtrack + palindrome check |
| 37   | Sudoku Solver                | Hard       | Constraint propagation       |

## 6. Solve Step-by-Step — LC 39: Combination Sum

**Problem:** Find all combinations that sum to target. Elements can be reused.

(Full template in Template E — dry run below)

**Dry Run:** candidates=[2,3,6,7], target=7

backtrack(remaining=7, start=0, current=[])

```
i=0: add 2, backtrack(remaining=5, start=0, current=[2])
i=0: add 2, backtrack(remaining=3, start=0, current=[2,2])
i=0: add 2, backtrack(remaining=1, start=0, current=[2,2,2])
i=0: add 2 → remaining=-1 → break (pruning!)
i=1: add 3 → remaining=-2 → break
i=1: add 3, backtrack(remaining=0, current=[2,2,3]) → ADD ✓
i=1: add 3, backtrack(remaining=2, start=1, current=[2,3])
i=1: add 3 → remaining=-1 → break
(no more)
i=1: add 3, backtrack(remaining=4, start=1, current=[3])
i=1: add 3, backtrack(remaining=1, start=1, current=[3,3])
pruning...
i=3: add 7, backtrack(remaining=0, current=[7]) → ADD ✓
```

Result: [[2,2,3],[7]] ✓

## 7. Pruning Strategies (Critical for Interviews)

| Technique    | How                                          | Problem         |
|--------------|----------------------------------------------|-----------------|
| Sort + break | Sort first, break when candidate > remaining | Combination Sum |
| Size limit   | Return early when current.size() == k        | Combinations    |

| Technique       | How                                                                               | Problem      |
|-----------------|-----------------------------------------------------------------------------------|--------------|
| Skip duplicates | Sort + if <code>i &gt; start</code> && <code>nums[i] == nums[i-1]</code> continue | Subsets II   |
| Used array      | Track which elements are in current path                                          | Permutations |
| Diagonal sets   | Precompute queen attack positions                                                 | N-Queens     |
| Mark grid       | Set cell to '#', restore after                                                    | Word Search  |

## 8. Common Interview Mistakes

- 1. Not making a copy when adding to result:** `result.add(current)` adds a reference — use `new ArrayList<>(current)`
- 2. Duplicates: sorting but wrong skip condition:** `if (i > start && nums[i] == nums[i-1])` for combinations; `if (i > 0 && nums[i] == nums[i-1] && !used[i-1])` for permutations
- 3. Reuse vs no-reuse:** pass `i` vs `i+1` as start parameter
- 4. Word Search: not restoring the cell** after backtracking — classic bug
- 5. N-Queens: not tracking diagonals** with sets —  $O(1)$  lookup instead of  $O(n)$  scan

## 9. Interview Cheat Sheet

## BACKTRACKING – MENTAL CHECKLIST

=====

- All solutions needed? → Backtracking
- Make copy when adding: `result.add(new ArrayList<>(current))`
- Avoid duplicates? → Sort first, skip same-value siblings
- Can reuse elements? → pass `i` as start (not `i+1`)
- No reuse? → pass `i+1` as start
- Grid problem? → mark visited, restore after

## TEMPLATE SKELETON

=====

```
void backtrack(state, current, result):
    if base_case:
        result.add(copy of current)
        return
    for each choice:
        if invalid: continue ← pruning
        make choice
        backtrack(next_state, current, result)
        undo choice ← THE BACKTRACK
```

## DUPLICATES SKIP PATTERNS

=====

Combinations: if (`i > start` && `nums[i] == nums[i-1]`) continue

Permutations: if (`i > 0` && `nums[i] == nums[i-1]` && `!used[i-1]`) continue

## COMPLEXITY INTUITION

=====

Subsets:  $2^n$  paths in tree

Permutations:  $n!$  leaf nodes

Each result copy:  $O(n)$

Total:  $O(2^n \times n)$  or  $O(n! \times n)$

---

## 10. Practice Roadmap

### Beginner:

- LC 78 — Subsets
- LC 46 — Permutations
- LC 77 — Combinations

### Intermediate:

- LC 39 — Combination Sum
- LC 90 — Subsets II
- LC 47 — Permutations II
- LC 79 — Word Search
- LC 131 — Palindrome Partitioning

### Hard:

- LC 51 — N-Queens
- LC 37 — Sudoku Solver
- LC 291 — Word Pattern II

- LC 489 — Robot Room Cleaner

# Pattern 20 — Monotonic Stack + Matching Parentheses

## 1. Pattern Recognition Guide

### Monotonic Stack keywords:

- "next greater element", "next smaller element"
- "previous greater/smaller"
- "largest rectangle in histogram"
- "daily temperatures", "stock span"
- "trapping rain water" (stack approach)

### Matching Parentheses keywords:

- "valid parentheses", "balanced brackets"
- "minimum add to make valid", "score of parentheses"
- "longest valid parentheses"
- "remove invalid parentheses"

**Signal phrase:** `"For each element, find the next greater element"` → Monotonic Stack

**Signal phrase:** `"Valid/balanced brackets"` → Stack with push/pop

## 2. Core Intuition

### Monotonic Stack

A stack that maintains elements in either increasing or decreasing order.

**When an element enters, it pops all elements that violate the monotonic property.**

Those popped elements have found their "answer" — the current element is their "next greater" (or smaller).

Daily Temperatures: [73, 74, 75, 71, 69, 72, 76, 73]

Find: days until warmer temperature

Stack stores INDICES (not values) – we need indices to compute days

```
i=0: stack=[], push 0 → stack=[0]
i=1: 74>73(idx 0) → pop 0, ans[0]=1-0=1 → push 1 → stack=[1]
i=2: 75>74(idx 1) → pop 1, ans[1]=2-1=1 → push 2 → stack=[2]
i=3: 71<75 → push 3 → stack=[2,3]
i=4: 69<71 → push 4 → stack=[2,3,4]
i=5: 72>69(4)→pop,ans[4]=1; 72>71(3)→pop,ans[3]=2; 72<75→push5 → stack=[2,5]
i=6: 76>72(5)→pop,ans[5]=1; 76>75(2)→pop,ans[2]=4; push6 → stack=[6]
i=7: 73<76 → push 7 → stack=[6,7]
```

No next warmer for indices 6,7 → ans[6]=ans[7]=0

Result: [1,1,4,2,1,1,0,0] ✓

## Matching Parentheses

Push open brackets. When a close bracket is seen, check if top of stack matches.

---

## 3. Generic Java Templates

### Template A — Next Greater Element (Monotonic Decreasing Stack)

```
public int[] nextGreaterElement(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    Arrays.fill(result, -1); // default: no greater element

    Deque<Integer> stack = new ArrayDeque<>(); // stores INDICES

    for (int i = 0; i < n; i++) {
        // Pop all indices whose value is less than current
        while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
            int idx = stack.pop();
            result[idx] = nums[i]; // current element is idx's next greater
        }
        stack.push(i);
    }

    return result;
}
```

### Template B — Daily Temperatures (Next Greater, Return Distance)

```
public int[] dailyTemperatures(int[] temps) {
    int n = temps.length;
    int[] result = new int[n];
    Deque<Integer> stack = new ArrayDeque<>();

    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temps[stack.peek()] < temps[i]) {
            int idx = stack.pop();
            result[idx] = i - idx; // days until warmer
        }
        stack.push(i);
    }

    return result;
}
```

### Template C — Previous Smaller Element

```
// For each element, find the nearest smaller element to its LEFT
public int[] previousSmaller(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    Deque<Integer> stack = new ArrayDeque<>(); // monotonic increasing

    for (int i = 0; i < n; i++) {
        // Maintain increasing stack – pop elements >= current
        while (!stack.isEmpty() && stack.peek() >= nums[i]) {
            stack.pop();
        }
        result[i] = stack.isEmpty() ? -1 : stack.peek(); // top = nearest smaller
        stack.push(nums[i]);
    }

    return result;
}
```

## Template D — Largest Rectangle in Histogram

```
// For each bar, find the area of the largest rectangle using that bar's height
public int largestRectangleArea(int[] heights) {
    int n = heights.length;
    int maxArea = 0;

    // Monotonic increasing stack (stores indices)
    // When we pop, we found the "right boundary" (current) and "left boundary" (new top)
    Deque<Integer> stack = new ArrayDeque<>();

    for (int i = 0; i <= n; i++) {
        int currHeight = (i == n) ? 0 : heights[i]; // sentinel 0 at end to flush stack

        while (!stack.isEmpty() && heights[stack.peek()] > currHeight) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }

        stack.push(i);
    }

    return maxArea;
}
```

## Template E — Valid Parentheses (Matching Brackets)

```

public boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();

    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);
        } else {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if (c == ')' && top != '(') return false;
            if (c == ']' && top != '[') return false;
            if (c == '}' && top != '{') return false;
        }
    }

    return stack.isEmpty(); // all matched
}

```

## Template F — Longest Valid Parentheses

```

public int longestValidParentheses(String s) {
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(-1); // sentinel base index

    int maxLen = 0;

    for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) == '(') {
            stack.push(i);
        } else {
            stack.pop(); // match with top '('
            if (stack.isEmpty()) {
                stack.push(i); // new base: unmatched ')'
            } else {
                maxLen = Math.max(maxLen, i - stack.peek());
            }
        }
    }

    return maxLen;
}

```

## Template G — Next Greater in Circular Array (LC 503)

```

public int[] nextGreaterElements(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    Arrays.fill(result, -1);
    Deque<Integer> stack = new ArrayDeque<>();

    // Iterate twice to simulate circular behavior
    for (int i = 0; i < 2 * n; i++) {
        while (!stack.isEmpty() && nums[stack.peek()] < nums[i % n]) {
            result[stack.pop()] = nums[i % n];
        }
        if (i < n) stack.push(i); // only push in first pass
    }

    return result;
}

```

## 4. Complexity Cheatsheet

| Operation            | Time | Space | Why O(n)?                       |
|----------------------|------|-------|---------------------------------|
| Next Greater Element | O(n) | O(n)  | Each element pushed/popped once |
| Largest Rectangle    | O(n) | O(n)  | Each bar pushed/popped once     |
| Valid Parentheses    | O(n) | O(n)  | Single pass                     |
| Longest Valid Parens | O(n) | O(n)  | Single pass                     |

Despite the nested while loop, total push+pop operations = O(n) — amortized O(1) per element.

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                        | Difficulty | Why Important                |
|------|--------------------------------|------------|------------------------------|
| 739  | Daily Temperatures             | Medium     | Classic monotonic stack      |
| 496  | Next Greater Element I         | Easy       | Basic NGE                    |
| 84   | Largest Rectangle in Histogram | Hard       | Width calculation with stack |
| 20   | Valid Parentheses              | Easy       | Classic bracket matching     |
| 32   | Longest Valid Parentheses      | Hard       | Stack with index tracking    |

### Advanced Variations

| LC # | Problem                            | Difficulty | Why Important               |
|------|------------------------------------|------------|-----------------------------|
| 503  | Next Greater Element II (Circular) | Medium     | Double iteration trick      |
| 42   | Trapping Rain Water                | Hard       | Stack or two-pointer        |
| 85   | Maximal Rectangle                  | Hard       | Row-by-row histogram        |
| 901  | Online Stock Span                  | Medium     | Monotonic stack with counts |
| 1019 | Next Greater Node in Linked List   | Medium     | Stack on linked list        |

## 6. Solve Step-by-Step — LC 84: Largest Rectangle in Histogram

**Problem:** Find the largest rectangular area that can be formed in a histogram.

### Key Insight

For each bar as the "shortest" bar in a rectangle:

- Extend left until a shorter bar is found
- Extend right until a shorter bar is found
- Area = height × (right boundary - left boundary - 1)

The monotonic stack gives us left and right boundaries in O(n) total.

```
public int largestRectangleArea(int[] heights) {
    Deque<Integer> stack = new ArrayDeque<>();
    int maxArea = 0;

    for (int i = 0; i <= heights.length; i++) {
        int h = (i == heights.length) ? 0 : heights[i];

        while (!stack.isEmpty() && heights[stack.peek()] > h) {
            int height = heights[stack.pop()];
            // Left boundary: new stack top (exclusive)
            // Right boundary: i (exclusive)
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }

    return maxArea;
}
```

**Dry Run:** [2,1,5,6,2,3]

```

i=0: h=2, stack=[], push 0 → stack=[0]
i=1: h=1, 2>1 → pop 0, height=2, width=1(stack empty), area=2; push 1 → stack=[1]
i=2: h=5, 1<5 → push 2 → stack=[1,2]
i=3: h=6, 5<6 → push 3 → stack=[1,2,3]
i=4: h=2, 6>2 → pop 3, height=6, width=4-2-1=1, area=6
    5>2 → pop 2, height=5, width=4-1-1=2, area=10 ← max
    1<2 → stop; push 4 → stack=[1,4]
i=5: h=3, 2<3 → push 5 → stack=[1,4,5]
i=6: h=0(sentinel), 3>0 → pop 5, height=3, width=6-4-1=1, area=3
    2>0 → pop 4, height=2, width=6-1-1=4, area=8
    1>0 → pop 1, height=1, width=6(stack empty), area=6

```

maxArea = 10 ✓

## 7. Pattern Variations

| Problem                 | Stack Type           | What Stack Stores               |
|-------------------------|----------------------|---------------------------------|
| Next greater (right)    | Monotonic decreasing | Indices waiting for NGE         |
| Previous smaller (left) | Monotonic increasing | Values seen so far              |
| Largest rectangle       | Monotonic increasing | Indices, pop when smaller seen  |
| Trapping rain water     | Monotonic decreasing | Indices, compute water when pop |
| Valid brackets          | N/A (just pairs)     | Open brackets                   |
| Longest valid parens    | N/A                  | Indices of unmatched chars      |

## 8. Common Interview Mistakes

- 1. Storing values instead of indices** — you need indices to compute widths/distances
- 2. Largest rectangle: forgetting sentinel 0** — stack never fully flushes without it
- 3. Largest rectangle width formula** —  $i - \text{stack.peek()} - 1$ , not  $i - \text{stack.peek()}$
- 4. Valid parentheses: not checking** `stack.isEmpty()` before popping — NPE
- 5. Circular NGE: pushing index in second pass** — only push if  $(i \leq n)$
- 6. Trapping rain water with stack: water = (min(leftMax, rightMax) - current) × width** — complex; two-pointer approach is cleaner

## 9. Interview Cheat Sheet

## MONOTONIC STACK – MENTAL CHECKLIST

=====

- "Next greater/smaller" for each element? → Monotonic stack
- Decreasing stack → finds NEXT GREATER (pop when bigger arrives)
- Increasing stack → finds NEXT SMALLER (pop when smaller arrives)
- Store INDICES (not values) when distance/width matters
- Sentinel value at end to flush remaining stack

## DIRECTION GUIDE

=====

Next Greater Element: iterate LEFT→RIGHT, pop when `nums[i] > nums[stack.top]`

Previous Smaller: iterate LEFT→RIGHT, maintain increasing stack

## MONOTONIC STACK TYPE

=====

Decreasing: each new element pops smaller ones → they found their NGE

Increasing: each new element pops larger ones → they found their previous smaller

## PARENTHESES TRICKS

=====

- Valid: push open, pop+match on close, check empty at end
- Longest valid: stack stores indices; sentinel -1; update on each ')' match
- Score: push 0 for '(', on ')': pop + max(2×top, 1) or just count depth

## LARGEST RECTANGLE

=====

pop when smaller height arrives → current idx = right boundary

new stack top = left boundary

width = `i - stack.peek() - 1` (or `i` if stack empty)

---

## 10. Practice Roadmap

### Beginner:

- LC 20 — Valid Parentheses
- LC 496 — Next Greater Element I
- LC 739 — Daily Temperatures

### Intermediate:

- LC 503 — Next Greater Element II (Circular)
- LC 901 — Online Stock Span
- LC 32 — Longest Valid Parentheses
- LC 1019 — Next Greater Node in Linked List

### Hard:

- LC 84 — Largest Rectangle in Histogram
- LC 85 — Maximal Rectangle
- LC 42 — Trapping Rain Water
- LC 907 — Sum of Subarray Minimums

# Pattern 21 — Hashing (Existence, Counting, Duplicates, Pair Sum)

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "two sum", "pair with sum k"
- "contains duplicate", "find duplicate"
- "frequency of elements", "most frequent"
- "group anagrams", "isomorphic strings"
- "subarray with sum k" (with prefix sum)
- "longest consecutive sequence"

**When to reach for Hashing:**

- $O(1)$  lookup needed (replace nested loops)
- Counting/frequency of elements
- Checking membership quickly
- Grouping elements by some key

**Signal phrase:** `"Two elements that sum to target"` → `HashMap` (complement lookup)

**Signal phrase:** `"Group strings by their character frequency"` → `HashMap` with sorted-string key

## 2. Core Intuition

**HashSet:**  $O(1)$  existence check. Replaces the inner loop in  $O(n^2) \rightarrow O(n)$ .

**HashMap:**  $O(1)$  key-value storage. Replaces recomputation with cached results.

**The Two Sum insight:**

Instead of: "for each pair, check if sum = target" →  $O(n^2)$

Do: "for each element x, look up (target - x) in a map" →  $O(n)$

`nums = [2, 7, 11, 15], target = 9`

Without hash: check (2,7), (2,11), (2,15), (7,11)... →  $O(n^2)$

With hash:

`i=0: need 9-2=7, map={}, 7 not found → map={2:0}`

`i=1: need 9-7=2, 2 is in map at idx 0 → return [0,1] ✓`

## 3. Generic Java Templates

### Template A — Check for Existence (Two Sum)

```

public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>(); // value → index

    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];

        if (seen.containsKey(complement)) {
            return new int[]{seen.get(complement), i};
        }

        seen.put(nums[i], i);
    }

    return new int[]{-1, -1}; // no solution
}

```

## Template B — Counting Elements (Frequency Map)

```

public Map<Integer, Integer> frequencyMap(int[] nums) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int num : nums) {
        freq.merge(num, 1, Integer::sum); // freq.put(num, freq.getOrDefault(num,0)+1)
    }
    return freq;
}

// Top K Frequent
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int num : nums) freq.merge(num, 1, Integer::sum);

    // Bucket sort by frequency
    List<Integer>[] buckets = new List[nums.length + 1];
    for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
        int f = e.getValue();
        if (buckets[f] == null) buckets[f] = new ArrayList<>();
        buckets[f].add(e.getKey());
    }

    int[] result = new int[k];
    int idx = 0;
    for (int i = buckets.length - 1; i >= 0 && idx < k; i--) {
        if (buckets[i] != null) {
            for (int num : buckets[i]) {
                result[idx++] = num;
            }
            if (idx == k) break;
        }
    }
    return result;
}

```

## Template C — Duplicate Detection

```
// Contains Duplicate
public boolean containsDuplicate(int[] nums) {
    Set<Integer> seen = new HashSet<>();
    for (int num : nums) {
        if (!seen.add(num)) return true; // add returns false if already present
    }
    return false;
}

// Contains Duplicate Within K Distance
public boolean containsNearbyDuplicate(int[] nums, int k) {
    Map<Integer, Integer> lastIndex = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        if (lastIndex.containsKey(nums[i]) && i - lastIndex.get(nums[i]) <= k) {
            return true;
        }
        lastIndex.put(nums[i], i);
    }
    return false;
}
```

## Template D — Grouping by Key (Anagrams)

```
public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> groups = new HashMap<>();

    for (String str : strs) {
        // Canonical key: sorted characters
        char[] chars = str.toCharArray();
        Arrays.sort(chars);
        String key = new String(chars);

        groups.computeIfAbsent(key, k -> new ArrayList<>()).add(str);
    }

    return new ArrayList<>(groups.values());
}

// Alternative key: char count array as string
private String charCountKey(String s) {
    int[] count = new int[26];
    for (char c : s.toCharArray()) count[c - 'a']++;
    return Arrays.toString(count); // e.g., "[1,0,1,0,...,0]"
}
```

## Template E — Longest Consecutive Sequence

```

public int longestConsecutive(int[] nums) {
    Set<Integer> numSet = new HashSet<>();
    for (int num : nums) numSet.add(num);

    int maxLen = 0;

    for (int num : numSet) {
        // Only start a new sequence at the beginning (no num-1 in set)
        if (!numSet.contains(num - 1)) {
            int curr = num;
            int len = 1;

            while (numSet.contains(curr + 1)) {
                curr++;
                len++;
            }

            maxLen = Math.max(maxLen, len);
        }
    }

    return maxLen;
}

```

## Template F — Pair Sum Variants

```

// Count pairs with sum = k
public int countPairsWithSum(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    int count = 0;

    for (int num : nums) {
        int complement = k - num;
        count += freq.getOrDefault(complement, 0);
        freq.merge(num, 1, Integer::sum);
    }

    return count;
}

// 4Sum II: count tuples (a,b,c,d) where a+b+c+d = 0
public int fourSumCount(int[] a, int[] b, int[] c, int[] d) {
    Map<Integer, Integer> sumAB = new HashMap<>();
    for (int x : a) for (int y : b) sumAB.merge(x + y, 1, Integer::sum);

    int count = 0;
    for (int x : c) for (int y : d) count += sumAB.getOrDefault(-(x + y), 0);
    return count;
}

```

## Template G — Isomorphic / Pattern Matching

```

public boolean isIsomorphic(String s, String t) {
    Map<Character, Character> sToT = new HashMap<>();
    Map<Character, Character> tToS = new HashMap<>();

    for (int i = 0; i < s.length(); i++) {
        char cs = s.charAt(i), ct = t.charAt(i);

        if (sToT.containsKey(cs) && sToT.get(cs) != ct) return false;
        if (tToS.containsKey(ct) && tToS.get(ct) != cs) return false;

        sToT.put(cs, ct);
        tToS.put(ct, cs);
    }
    return true;
}

```

## 4. Complexity Cheatsheet

| Operation                 | Time                 | Space    |
|---------------------------|----------------------|----------|
| HashMap get/put           | O(1) avg, O(n) worst | O(n)     |
| HashSet add/contains      | O(1) avg             | O(n)     |
| Group anagrams (sort key) | O(n × k log k)       | O(n × k) |
| Longest consecutive       | O(n)                 | O(n)     |
| Two Sum                   | O(n)                 | O(n)     |

HashMap worst case O(n) per operation is theoretical (hash collision). In practice, treated as O(1).

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                      | Difficulty | Why Important            |
|------|------------------------------|------------|--------------------------|
| 1    | Two Sum                      | Easy       | Core complement lookup   |
| 217  | Contains Duplicate           | Easy       | HashSet existence        |
| 49   | Group Anagrams               | Medium     | HashMap grouping         |
| 128  | Longest Consecutive Sequence | Medium     | HashSet + sequence start |
| 347  | Top K Frequent Elements      | Medium     | Freq map + bucket sort   |

### Advanced Variations

| LC # | Problem               | Difficulty | Why Important        |
|------|-----------------------|------------|----------------------|
| 454  | 4Sum II               | Medium     | Two-pass HashMap     |
| 560  | Subarray Sum Equals K | Medium     | Prefix sum + HashMap |

| LC # | Problem                  | Difficulty | Why Important             |
|------|--------------------------|------------|---------------------------|
| 205  | Isomorphic Strings       | Easy       | Bidirectional mapping     |
| 290  | Word Pattern             | Easy       | Pattern matching          |
| 76   | Minimum Window Substring | Hard       | Sliding window + freq map |

## 6. Solve Step-by-Step — LC 128: Longest Consecutive Sequence

**Problem:** Find length of the longest consecutive sequence.  $O(n)$  required.

### Why Naive $O(n^2)$ Fails

For each number, check if  $num+1$ ,  $num+2...$  exist — repeated HashSet lookups per number.

### $O(n)$ Insight: Only Start at Sequence Beginnings

A number  $n$  is a sequence start if and only if  $n-1$  is NOT in the set.

This ensures each number is only part of one sequence walk.

```
public int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int num : nums) set.add(num);

    int maxLen = 0;

    for (int num : set) {
        if (!set.contains(num - 1)) { // sequence start
            int curr = num, len = 1;
            while (set.contains(curr + 1)) { curr++; len++; }
            maxLen = Math.max(maxLen, len);
        }
    }
    return maxLen;
}
```

**Dry Run:** [100, 4, 200, 1, 3, 2]

set = {100,4,200,1,3,2}

num=100: 99 not in set → start sequence  
101 not in set → len=1, maxLen=1

num=4: 3 is in set → NOT a start, skip

num=200: 199 not in set → start sequence  
201 not in set → len=1

num=1: 0 not in set → start sequence  
2 in set → len=2  
3 in set → len=3  
4 in set → len=4  
5 not in set → done, maxLen=4

Answer: 4 ([1,2,3,4]) ✓

---

## 7. Pattern Variations

| Problem                 | HashMap Key          | HashMap Value           |
|-------------------------|----------------------|-------------------------|
| Two Sum                 | number               | index                   |
| Subarray sum = k        | prefix sum           | count of occurrences    |
| Group anagrams          | sorted string        | list of anagrams        |
| Contains near duplicate | number               | last seen index         |
| Isomorphic strings      | char (bidirectional) | mapped char             |
| Top K frequent          | number               | frequency               |
| Longest consecutive     | —                    | just HashSet membership |

---

## 8. Common Interview Mistakes

- 1. Two Sum: putting current element before checking** — would match element with itself; check first, then put
- 2. Group anagrams: using char array as key** — arrays don't have value-based `.equals()` ; use `String` or `Arrays.toString`
- 3. Longest consecutive: iterating over array** instead of set — duplicates cause redundant work; iterate over set
- 4. Not handling null in `getOrDefault`** — use `getOrDefault(key, 0)` not `get(key)` without null check
- 5. Freq map with `merge`** — cleaner than `getOrDefault + put` ; know both styles

---

## 9. Interview Cheat Sheet

## HASHING — MENTAL CHECKLIST

=====

- Pair/complement lookup? → HashMap: value→index
- Existence check (no index)? → HashSet
- Frequency/count? → HashMap with merge(key, 1, Integer::sum)
- Group by property? → HashMap<Key, List<Value>>
- Deduplicate? → HashSet
- Subarray sum = k? → Prefix sum + HashMap (see Pattern 02)

## COMMON PATTERNS

=====

Two Sum: map.get(target - nums[i]) → check, then put

Freq count: map.merge(num, 1, Integer::sum)

Anagram key: new String(sorted char array)

Near duplicate: map.get(num) and check distance

## JAVA QUICK REF

=====

map.getOrDefault(key, 0)

map.merge(key, 1, Integer::sum)

map.computeIfAbsent(key, k -> new ArrayList<>()).add(val)

set.add(x) // returns false if already present

!set.add(x) // true = duplicate found

## TRICKS

=====

- Longest consecutive: only start at sequence beginnings → O(n)
- Group anagrams: sort key OR character count array as string key
- 4Sum II: split into two pairs, hash first pair sums
- Bucket sort by frequency → O(n) top-k without heap

---

## 10. Practice Roadmap

### Beginner:

- LC 1 — Two Sum
- LC 217 — Contains Duplicate
- LC 242 — Valid Anagram
- LC 205 — Isomorphic Strings

### Intermediate:

- LC 49 — Group Anagrams
- LC 128 — Longest Consecutive Sequence
- LC 347 — Top K Frequent Elements
- LC 219 — Contains Duplicate II

### Hard:

- LC 560 — Subarray Sum Equals K
- LC 454 — 4Sum II
- LC 76 — Minimum Window Substring
- LC 30 — Substring with Concatenation of All Words

# Pattern 22 — Subarray Count Problems

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "number of subarrays with sum/product exactly k"
- "count subarrays where..."
- "at most k distinct", "exactly k distinct"
- "number of nice subarrays" (odd count = k)
- "binary subarrays with sum"

**The Three Approaches:**

1. **Prefix Sum + HashMap** — when condition is about sums (including negatives)
2. **Sliding Window (AtMost trick)** — when condition is about constraints (counts, distinct values) and values are non-negative
3. **Two Pointers** — when shrinking is safe

**Signal phrase:** \_"Count subarrays with sum equal to k"\_ → Prefix + HashMap

**Signal phrase:** \_"Count subarrays with at most k distinct characters"\_ → Sliding Window

**Signal phrase:** \_"Count subarrays with exactly k"\_ →  $\text{atMost}(k) - \text{atMost}(k-1)$

## 2. Core Intuition

### AtMost Trick (Key Insight)

Counting subarrays with **exactly** k of some property is hard.

But counting subarrays with **at most** k is easy (sliding window).

$$\text{exactly}(k) = \text{atMost}(k) - \text{atMost}(k-1)$$

This works when the property is monotonic: adding elements only increases (never decreases) the count of the property.

### Why prefix + HashMap for sums:

Count subarrays with  $\text{sum} = k$   
= Count pairs  $(i, j)$  where  $\text{prefix}[j] - \text{prefix}[i] = k$   
= Count pairs where  $\text{prefix}[i] = \text{prefix}[j] - k$   
= For each j, look up how many times  $(\text{prefix}[j] - k)$  appeared before

This is exactly what the HashMap tracks.

## 3. Generic Java Templates

### Template A — Subarray Sum Equals K (Prefix + HashMap)

```
// Works with negative numbers
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1); // empty prefix
    int prefixSum = 0, count = 0;

    for (int num : nums) {
        prefixSum += num;
        count += prefixCount.getOrDefault(prefixSum - k, 0);
        prefixCount.merge(prefixSum, 1, Integer::sum);
    }

    return count;
}
```

## Template B — AtMost K Distinct (Sliding Window Count)

```
// Count subarrays with AT MOST k distinct elements
// Each iteration: right - left + 1 = number of valid subarrays ending at right
public int atMostK(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    int left = 0, count = 0;

    for (int right = 0; right < nums.length; right++) {
        freq.merge(nums[right], 1, Integer::sum);

        while (freq.size() > k) {
            freq.merge(nums[left], -1, Integer::sum);
            if (freq.get(nums[left]) == 0) freq.remove(nums[left]);
            left++;
        }

        count += right - left + 1; // all subarrays [left..right], [left+1..right], ...
    }

    return count;
}

// Count subarrays with EXACTLY k distinct elements
public int exactlyK(int[] nums, int k) {
    return atMostK(nums, k) - atMostK(nums, k - 1);
}
```

## Template C — Binary Subarrays with Sum (AtMost on Binary Array)

```
// Count subarrays with sum = goal (binary array – only 0s and 1s)
public int numSubarraysWithSum(int[] nums, int goal) {
    return atMostBinary(nums, goal) - atMostBinary(nums, goal - 1);
}

private int atMostBinary(int[] nums, int goal) {
    if (goal < 0) return 0;
    int left = 0, sum = 0, count = 0;

    for (int right = 0; right < nums.length; right++) {
        sum += nums[right];
        while (sum > goal) sum -= nums[left++];
        count += right - left + 1;
    }

    return count;
}
```

### Template D — Count Subarrays with Product Less Than K

```
public int numSubarrayProductLessThanK(int[] nums, int k) {
    if (k <= 1) return 0;
    int left = 0, product = 1, count = 0;

    for (int right = 0; right < nums.length; right++) {
        product *= nums[right];
        while (product >= k) product /= nums[left++];
        count += right - left + 1; // all subarrays ending at right
    }

    return count;
}
```

### Template E — Count Subarrays with Bounded Max (AtMost trick on range)

```
// Count subarrays where max element is in [minK, maxK]
// = atMost(maxK) - atMost(minK - 1)
public int numSubarrayBoundedMax(int[] nums, int left, int right) {
    return atMostMax(nums, right) - atMostMax(nums, left - 1);
}

private int atMostMax(int[] nums, int bound) {
    int count = 0, curr = 0;
    for (int num : nums) {
        curr = (num <= bound) ? curr + 1 : 0;
        count += curr;
    }
    return count;
}
```

### Template F — Count Nice Subarrays (Exactly k Odd Numbers)

```
// Transform: odd=1, even=0, then count subarrays with sum = k
public int numberOfSubarrays(int[] nums, int k) {
    // prefix sum approach on transformed array
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1);
    int prefixSum = 0, count = 0;

    for (int num : nums) {
        prefixSum += num % 2; // 1 if odd, 0 if even
        count += prefixCount.getOrDefault(prefixSum - k, 0);
        prefixCount.merge(prefixSum, 1, Integer::sum);
    }

    return count;
}
```

## 4. Complexity Cheatsheet

| Approach                 | Time   | Space  | When to Use                     |
|--------------------------|--------|--------|---------------------------------|
| Prefix + HashMap         | $O(n)$ | $O(n)$ | Sum-based, allows negatives     |
| AtMost sliding window    | $O(n)$ | $O(k)$ | Count/distinct, positive values |
| AtMost on binary/bounded | $O(n)$ | $O(1)$ | Binary or bounded values        |

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                             | Difficulty | Approach                |
|------|-------------------------------------|------------|-------------------------|
| 560  | Subarray Sum Equals K               | Medium     | Prefix + HashMap        |
| 992  | Subarrays with K Different Integers | Hard       | AtMost(k) - AtMost(k-1) |
| 930  | Binary Subarrays with Sum           | Medium     | AtMost binary           |
| 713  | Subarray Product Less Than K        | Medium     | Sliding window count    |
| 1248 | Count Number of Nice Subarrays      | Medium     | Transform + prefix      |

### Advanced Variations

| LC # | Problem                      | Difficulty | Approach             |
|------|------------------------------|------------|----------------------|
| 974  | Subarray Sums Divisible by K | Medium     | Prefix mod + HashMap |

| LC # | Problem                                          | Difficulty | Approach             |
|------|--------------------------------------------------|------------|----------------------|
| 795  | Number of Subarrays with Bounded Maximum         | Medium     | AtMost range trick   |
| 1358 | Number of Substrings Containing All 3 Characters | Medium     | AtMost / two pointer |
| 2302 | Count Subarrays with Score Less Than K           | Medium     | Sliding window       |
| 2537 | Count the Number of Good Subarrays               | Medium     | HashMap pairs        |

## 6. Solve Step-by-Step — LC 992: Subarrays with K Different Integers

**Problem:** Count subarrays with exactly K distinct integers.

### Why AtMost Works Here

- "Exactly K distinct" = subarrays with at most K distinct - subarrays with at most K-1 distinct
- "At most K" = shrink window when distinct count exceeds K, count all valid subarrays at each step

```
public int subarraysWithKDistinct(int[] nums, int k) {
    return atMostK(nums, k) - atMostK(nums, k - 1);
}

private int atMostK(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    int left = 0, count = 0;

    for (int right = 0; right < nums.length; right++) {
        freq.merge(nums[right], 1, Integer::sum);

        while (freq.size() > k) {
            freq.merge(nums[left], -1, Integer::sum);
            if (freq.get(nums[left]) == 0) freq.remove(nums[left]);
            left++;
        }

        count += right - left + 1;
    }

    return count;
}
```

**Dry Run:** nums=[1,2,1,2,3], k=2

```

atMost(2):
right=0: freq={1:1}, size=1, count+=1=1
right=1: freq={1:1,2:1}, size=2, count+=2=3
right=2: freq={1:2,2:1}, size=2, count+=3=6
right=3: freq={1:2,2:2}, size=2, count+=4=10
right=4: freq={1:2,2:2,3:1}, size=3 > 2
  shrink: remove 1(→1), left=1; still size=3
  shrink: remove 2(→1), left=2; size=2
  count+=3(right-left+1=4-2+1=3)=13

```

```

atMost(1):
right=0: count=1
right=1: {1,2} size=2 > 1, shrink(1→0,remove), left=1; count+=1=2
right=2: {2:1,1:1} size=2>1, shrink(2→0,remove), left=2; count+=1=3
right=3: {1:1,2:1} size=2>1, shrink(1→0,remove), left=3; count+=1=4
right=4: {2:1,3:1} size=2>1, shrink(2→0,remove), left=4; count+=1=5

```

Answer:  $\text{atMost}(2) - \text{atMost}(1) = 13 - 5 = 7 \dots$

Wait, let me recount (actual answer for  $[1,2,1,2,3]$   $k=2$  is 7) ✓

## 7. Choosing the Right Approach

Count subarrays where...

- Sum =  $k$
- ■■■ Array has negatives? → Prefix + HashMap
- ■■■ Array is non-negative? → Sliding window OR prefix + HashMap
- Exactly  $k$  distinct elements →  $\text{atMost}(k) - \text{atMost}(k-1)$
- Sum divisible by  $k$  → Prefix mod  $k$  + HashMap
- Product <  $k$  → Sliding window count
- Max in  $[\text{min}, \text{max}]$  →  $\text{atMost}(\text{max}) - \text{atMost}(\text{min}-1)$
- Exactly  $k$  odds → transform ( $\text{odd}=1, \text{even}=0$ ) + prefix + HashMap

## 8. Common Interview Mistakes

1. **Prefix + HashMap: forgetting** `prefixCount.put(0, 1)` — misses subarrays starting at index 0
2. **AtMost trick: using**  $k \leq 0 \rightarrow \text{return } 0$  — needed when subtracting  $\text{atMost}(k-1)$  with  $k=0$
3. **Sliding window count formula** — it's  $\text{right} - \text{left} + 1$  (all subarrays ending at right with left as minimum start)
4. **Not removing zero-frequency entries from HashMap** — `freq.size()` counts keys including zero-value ones
5. **Confusing prefix sum approaches for product problems** — products don't have the same complement structure; use sliding window

## 9. Interview Cheat Sheet

## SUBARRAY COUNT — DECISION TREE

=====

Has negatives? → Prefix + HashMap  
Exactly k property? →  $\text{atMost}(k) - \text{atMost}(k-1)$   
Sum = k (non-negative)? → Sliding window or prefix  
Product < k? → Sliding window  
Distinct count? → atMost sliding window

## SLIDING WINDOW COUNT FORMULA

=====

while window invalid: shrink left  
 $\text{count} += \text{right} - \text{left} + 1 \leftarrow \text{subarrays } [\text{left}..\text{right}], [\text{left}+1..\text{right}], \dots [\text{right}..\text{right}]$

## PREFIX SUM COUNT FORMULA

=====

```
count += map.getOrDefault(prefixSum - k, 0)
map.merge(prefixSum, 1, Integer::sum)
// Initialize: map.put(0, 1)
```

## ATMOST TRICK

=====

$\text{exactly}(k) = \text{atMost}(k) - \text{atMost}(k-1)$   
Works when: adding an element never DECREASES the "count" metric

---

## 10. Practice Roadmap

### Beginner:

- LC 713 — Subarray Product Less Than K
- LC 560 — Subarray Sum Equals K
- LC 930 — Binary Subarrays with Sum

### Intermediate:

- LC 1248 — Count Number of Nice Subarrays
- LC 974 — Subarray Sums Divisible by K
- LC 795 — Number of Subarrays with Bounded Maximum

### Hard:

- LC 992 — Subarrays with K Different Integers
- LC 1358 — Substrings Containing All 3 Characters
- LC 2302 — Count Subarrays with Score Less Than K

# Pattern 23 — Implicit Graphs (BFS on State Spaces)

---

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "minimum number of operations/steps to reach target"
- "word transformation", "one character change at a time"
- "unlock combination lock", "minimum moves"
- "jump game", "reachable states"
- "sliding puzzle", "8-puzzle", "rubik's cube moves"
- "minimum genetic mutations"

**What makes a graph "implicit"?**

The graph is never given to you explicitly as an adjacency list.

Instead, nodes are **states** and edges are **valid transitions** between states.

You generate neighbors on the fly.

**Signal phrase:** `"Minimum steps to transform X into Y, one change at a time"` → Implicit Graph  
BFS

**Signal phrase:** `"Minimum moves to reach target state"` → BFS on state space

**Why BFS (not DFS)?**

BFS gives shortest path. In implicit graphs with uniform edge weights, BFS level = number of transformations = the answer.

---

## 2. Core Intuition

**Reframe the problem as a graph:**

- **Node** = current state (a word, a lock combination, a board configuration)
- **Edge** = one valid transformation (change one letter, turn one dial, make one move)
- **Source** = starting state
- **Target** = goal state
- **Answer** = BFS level when target is first reached

Word Ladder: "hit" → "hot" → "dot" → "dog" → "cog"

State space (nodes = words in dictionary):

```
"hit" — "hot" — "dot" — "dog" — "cog"
 \ /
  "lot"—"log"
```

BFS from "hit":

Level 0: {hit}

Level 1: {hot} ← change i→o

Level 2: {dot, lot} ← change h→d or h→l

Level 3: {dog, log} ← change t→g

Level 4: {cog} ← change d→c ← TARGET FOUND

Answer: 4 transformations (5 words including start)

### Key optimizations:

1. **Visited set:** mark states visited when enqueued (not when dequeued) to avoid re-processing
  2. **Bidirectional BFS:** search from both source AND target simultaneously — cuts search space from  $O(b^d)$  to  $O(b^{(d/2)})$
  3. **State encoding:** encode states as strings/integers for efficient hashing
- 

## 3. Generic Java Templates

### Template A — Basic Implicit Graph BFS

```

public int minSteps(State start, State target) {
    if (start.equals(target)) return 0;

    Queue<State> queue = new LinkedList<>();
    Set<State> visited = new HashSet<>();

    queue.offer(start);
    visited.add(start);
    int steps = 0;

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        steps++;

        for (int i = 0; i < levelSize; i++) {
            State curr = queue.poll();

            for (State next : getNeighbors(curr)) {
                if (next.equals(target)) return steps;

                if (!visited.contains(next)) {
                    visited.add(next);
                    queue.offer(next);
                }
            }
        }

        return -1; // unreachable
    }

    // Override this per problem
    private List<State> getNeighbors(State curr) {
        // generate all valid one-step transitions from curr
        return new ArrayList<>();
    }
}

```

## Template B — Word Ladder (String State)

```

public int ladderLength(String beginWord, String endWord, List<String> wordList) {
    Set<String> wordSet = new HashSet<>(wordList);
    if (!wordSet.contains(endWord)) return 0;

    Queue<String> queue = new LinkedList<>();
    Set<String> visited = new HashSet<>();
    queue.offer(beginWord);
    visited.add(beginWord);
    int steps = 1;

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        steps++;

        for (int i = 0; i < levelSize; i++) {
            String word = queue.poll();
            char[] chars = word.toCharArray();

            // Generate all one-character variations
            for (int j = 0; j < chars.length; j++) {
                char original = chars[j];

                for (char c = 'a'; c <= 'z'; c++) {
                    if (c == original) continue;
                    chars[j] = c;
                    String next = new String(chars);

                    if (next.equals(endWord)) return steps;

                    if (wordSet.contains(next) && !visited.contains(next)) {
                        visited.add(next);
                        queue.offer(next);
                    }
                }

                chars[j] = original; // restore
            }
        }

        return 0; // no path
    }
}

```

## Template C — Open the Lock (Digit State)

```

public int openLock(String[] deadends, String target) {
    Set<String> dead = new HashSet<>(Arrays.asList(deadends));
    String start = "0000";

    if (dead.contains(start)) return -1;
    if (start.equals(target)) return 0;

    Queue<String> queue = new LinkedList<>();
    Set<String> visited = new HashSet<>();
    queue.offer(start);
    visited.add(start);
    int turns = 0;

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        turns++;

        for (int i = 0; i < levelSize; i++) {
            String curr = queue.poll();

            for (String next : getNeighbors(curr)) {
                if (next.equals(target)) return turns;

                if (!dead.contains(next) && !visited.contains(next)) {
                    visited.add(next);
                    queue.offer(next);
                }
            }
        }

        return -1;
    }

    // Each dial can go +1 or -1 (wrapping: 0↔9)
    private List<String> getNeighbors(String state) {
        List<String> neighbors = new ArrayList<>();
        char[] chars = state.toCharArray();

        for (int i = 0; i < 4; i++) {
            char original = chars[i];

            // Turn forward
            chars[i] = (char)((original - '0' + 1) % 10 + '0');
            neighbors.add(new String(chars));

            // Turn backward
            chars[i] = (char)((original - '0' + 9) % 10 + '0');
            neighbors.add(new String(chars));

            chars[i] = original; // restore
        }

        return neighbors;
    }
}

```

## Template D — Bidirectional BFS (Word Ladder Optimized)

```
public int ladderLengthBiDir(String beginWord, String endWord, List<String> wordList) {
    Set<String> wordSet = new HashSet<>(wordList);
    if (!wordSet.contains(endWord)) return 0;

    Set<String> beginSet = new HashSet<>();
    Set<String> endSet = new HashSet<>();
    Set<String> visited = new HashSet<>();

    beginSet.add(beginWord);
    endSet.add(endWord);
    int steps = 1;

    while (!beginSet.isEmpty() && !endSet.isEmpty()) {
        steps++;

        // Always expand the smaller set
        if (beginSet.size() > endSet.size()) {
            Set<String> temp = beginSet;
            beginSet = endSet;
            endSet = temp;
        }

        Set<String> nextLevel = new HashSet<>();

        for (String word : beginSet) {
            char[] chars = word.toCharArray();

            for (int j = 0; j < chars.length; j++) {
                char original = chars[j];

                for (char c = 'a'; c <= 'z'; c++) {
                    if (c == original) continue;
                    chars[j] = c;
                    String next = new String(chars);

                    if (endSet.contains(next)) return steps; // met in the middle!

                    if (wordSet.contains(next) && !visited.contains(next)) {
                        visited.add(next);
                        nextLevel.add(next);
                    }
                }
            }

            chars[j] = original;
        }

        beginSet = nextLevel;
    }

    return 0;
}
```

## Template E — Sliding Puzzle (Board State as String)

```

public int slidingPuzzle(int[][] board) {
    // Encode board as string: "123450" where 0 = empty
    StringBuilder sb = new StringBuilder();
    for (int[] row : board) for (int val : row) sb.append(val);

    String start = sb.toString();
    String target = "123450";

    if (start.equals(target)) return 0;

    // Valid swaps for each position of 0 in a 2x3 grid
    int[][] neighbors = {
        {1, 3}, // pos 0: can swap with pos 1, 3
        {0, 2, 4}, // pos 1: can swap with pos 0, 2, 4
        {1, 5}, // pos 2: can swap with pos 1, 5
        {0, 4}, // pos 3: can swap with pos 0, 4
        {1, 3, 5}, // pos 4: can swap with pos 1, 3, 5
        {2, 4} // pos 5: can swap with pos 2, 4
    };

    Queue<String> queue = new LinkedList<>();
    Set<String> visited = new HashSet<>();
    queue.offer(start);
    visited.add(start);
    int moves = 0;

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        moves++;

        for (int i = 0; i < levelSize; i++) {
            String curr = queue.poll();
            int zeroPos = curr.indexOf('0');

            for (int neighborPos : neighbors[zeroPos]) {
                char[] next = curr.toCharArray();
                // Swap 0 with neighbor
                next[zeroPos] = next[neighborPos];
                next[neighborPos] = '0';
                String nextStr = new String(next);

                if (nextStr.equals(target)) return moves;

                if (!visited.contains(nextStr)) {
                    visited.add(nextStr);
                    queue.offer(nextStr);
                }
            }
        }

        return -1;
    }
}

```

## Template F — Jump Game with BFS (Min Jumps)

```

public int jump(int[] nums) {
    int n = nums.length;
    if (n == 1) return 0;

    // BFS: each level = one jump
    boolean[] visited = new boolean[n];
    Queue<Integer> queue = new LinkedList<>();
    queue.offer(0);
    visited[0] = true;
    int jumps = 0;

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        jumps++;

        for (int i = 0; i < levelSize; i++) {
            int pos = queue.poll();

            for (int step = 1; step <= nums[pos]; step++) {
                int next = pos + step;
                if (next >= n - 1) return jumps;
                if (!visited[next]) {
                    visited[next] = true;
                    queue.offer(next);
                }
            }
        }
    }

    return -1;
}

```

## 4. Complexity Cheatsheet

| Problem                               | States         | Transitions per State | Total Time                     |
|---------------------------------------|----------------|-----------------------|--------------------------------|
| Word Ladder (L=word len, N=dict size) | N              | $L \times 26$         | $O(N \times L \times 26)$      |
| Open Lock                             | $10^4 = 10000$ | $4 \times 2 = 8$      | $O(10000 \times 8)$            |
| Sliding Puzzle (2×3)                  | $6! = 720$     | $\leq 4$              | $O(720 \times 4)$              |
| Jump Game                             | n              | nums[i]               | $O(n \times \text{max\_jump})$ |

Bidirectional BFS reduces Word Ladder from  $O(N \times L \times 26)$  to  $O(\sqrt{N} \times L \times 26)$  in practice.

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                        | Difficulty | State          | Transitions              |
|------|--------------------------------|------------|----------------|--------------------------|
| 127  | Word Ladder                    | Hard       | string word    | 26 × word_len variations |
| 752  | Open the Lock                  | Medium     | 4-digit string | 8 dial turns             |
| 433  | Minimum Genetic Mutation       | Medium     | gene string    | 4 × gene_len mutations   |
| 1091 | Shortest Path in Binary Matrix | Medium     | (row, col)     | 8 directions             |
| 773  | Sliding Puzzle                 | Hard       | board string   | swap 0 with neighbors    |

## Advanced Variations

| LC # | Problem                       | Difficulty | State               | Key Trick                         |
|------|-------------------------------|------------|---------------------|-----------------------------------|
| 126  | Word Ladder II (all paths)    | Hard       | string              | BFS levels + DFS backtrack        |
| 815  | Bus Routes                    | Hard       | bus stop            | BFS on routes, not stops          |
| 1197 | Minimum Knight Moves          | Medium     | (row, col)          | 8 knight moves, bidirectional     |
| 854  | K-Similar Strings             | Hard       | string              | BFS on permutation states         |
| 864  | Shortest Path to Get All Keys | Hard       | (pos, keys bitmask) | State = position + collected keys |

## 6. Solve Step-by-Step — LC 752: Open the Lock

**Problem:** Start at "0000", avoid deadends, reach target in minimum turns. Each turn rotates one of 4 dials  $\pm 1$  (wrapping:  $0 \leftrightarrow 9$ ).

(Full implementation in Template C — dry run below)

**Dry Run:** deadends=["0201", "0101", "0102", "1212", "2002"], target="0202"

start="0000", target="0202"

Level 1 neighbors of "0000":

dial 0: "1000", "9000"

dial 1: "0100", "0900"

dial 2: "0010", "0090" ← "0010" useful

dial 3: "0001", "0009"

None equal target, none are deadends (except check: "0101" is dead)

Level 2 neighbors include "0102", "0110"...

"0101" is dead → skip

"0102" is dead → skip

Level 3: eventually reaches "0202"

Answer: 6 turns

### Key insight — deadends block paths:

If deadend contains "0000", return -1 immediately. Otherwise skip deadend states during BFS.

## 7. State Design Patterns

| State Type           | Encoding                              | Example                 |
|----------------------|---------------------------------------|-------------------------|
| Grid position        | int[] {row, col} or row * cols + col  | Maze, shortest path     |
| String configuration | String (directly hashable)            | Word Ladder, lock       |
| Board layout         | Arrays.toString() or row-major string | Sliding Puzzle          |
| Position + extras    | String combining position + flags     | Keys + position bitmask |
| Number sequence      | Integer or string representation      | Combinations            |

**State encoding rule:** The state must capture **all information needed to determine valid next moves**. If two positions look the same but have different histories that matter — they're different states.

## 8. Common Interview Mistakes

- 1. Marking visited when dequeuing instead of enqueueing** — same state gets enqueued multiple times → TLE or wrong answer
- 2. Forgetting to check if start == target** — return 0 immediately
- 3. Word Ladder: not checking if endWord is in wordList** — return 0 if not present
- 4. Lock: not handling deadend at "0000"** — check before BFS starts
- 5. Bidirectional BFS: swapping sets incorrectly** — always expand the smaller set
- 6. State size explosion** — for complex states (like all keys bitmask), estimate state count before BFS to ensure feasibility

## 9. Interview Cheat Sheet

### IMPLICIT GRAPH BFS – MENTAL CHECKLIST

=====

- What is the STATE? (encodes full situation)
- What are the TRANSITIONS? (valid one-step moves)
- Can I reach target from start?
- Mark visited when ENQUEUEING (not dequeuing)
- Use Set<State> for O(1) visited lookup
- Bidirectional BFS if search space is large

### TEMPLATE SKELETON

=====

```
queue.offer(start); visited.add(start); steps=0
while queue not empty:
    steps++
    for each node in current level:
        node = queue.poll()
        for each neighbor in getNeighbors(node):
            if neighbor == target: return steps
            if not visited: visited.add, queue.offer
```

### STATE DESIGN

=====

Position only → int[] or encoded int  
String state → directly as String key  
Board → row-major string "123045"  
Position + bitmask → "row,col,mask" string

### COMPLEXITY FORMULA

=====

$O(\text{total\_states} \times \text{transitions\_per\_state})$   
Word Ladder:  $O(N \times L \times 26)$   
Lock:  $O(10^4 \times 8)$   
Always check: is state space small enough for BFS?

### TRICKS

=====

- Bidirectional BFS: expand smaller frontier →  $O(b^{(d/2)})$  vs  $O(b^d)$
- Precompute adjacency lists for word sets (group by pattern)
- Dead state (deadend/wall) → skip but still check
- "All keys" problems → bitmask in state ( $2^k$  states)

---

## 10. Practice Roadmap

### Beginner:

- LC 433 — Minimum Genetic Mutation
- LC 752 — Open the Lock
- LC 1091 — Shortest Path in Binary Matrix

### Intermediate:

- LC 127 — Word Ladder
- LC 773 — Sliding Puzzle

- LC 1197 — Minimum Knight Moves

**Hard:**

- LC 126 — Word Ladder II (all shortest paths)
- LC 864 — Shortest Path to Get All Keys
- LC 815 — Bus Routes
- LC 854 — K-Similar Strings

# Pattern 24 — Heap + Greedy: Minimum Number Problems

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "minimum number of operations/steps to reduce to 1"
- "task scheduler", "reorganize string", "rearrange"
- "last stone weight", "stone game"
- "minimum refueling stops"
- "maximize capital / profit with limited selections"
- "find the optimal next item given a constraint"

**When Heap + Greedy beats pure DP:**

- You always want the locally optimal next choice (max or min available)
- The set of available choices changes dynamically as you process
- Items become "available" as you progress (unlock by reaching a threshold)

**Signal phrase:** \_"Always pick the most/least X that is available at this moment"\_ → Max/Min Heap + Greedy

**Signal phrase:** \_"Rearrange such that no two same elements are adjacent"\_ → Max Heap + cooldown

## 2. Core Intuition

### Greedy with Heap

Greedy algorithms make locally optimal choices at each step. A heap gives you the locally optimal element (max or min) in  $O(\log n)$ .

The classic pattern:

1. **Sort / preprocess** to determine when items become available
2. **Sweep** through time/capacity/positions
3. At each step, **greedily pick** the best available item from the heap
4. **Add newly available items** to the heap as you progress

Minimum Refueling Stops:

Target = 100 miles, fuel = 10

Stations: [(10,60), (20,30), (30,30), (60,40)]

At mile 10: can reach, add station fuel (60) to max-heap

At mile 20: can reach, add (30) to max-heap

At mile 30: can reach, add (30) to max-heap

Ran out of fuel at mile 40 → MUST refuel → take max from heap (60)

Now at mile 40+60=100 → reached! → 1 stop

Key: we delay the greedy choice until we NEED to refuel.

Max-heap always gives us the best past station to pick.

---

### 3. Generic Java Templates

#### Template A — Last Stone Weight (Max Heap Simulation)

```
public int lastStoneWeight(int[] stones) {
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
    for (int stone : stones) maxHeap.offer(stone);

    while (maxHeap.size() > 1) {
        int heaviest = maxHeap.poll();
        int second = maxHeap.poll();

        if (heaviest != second) {
            maxHeap.offer(heaviest - second); // remainder goes back
        }
        // if equal, both destroyed – don't add anything
    }

    return maxHeap.isEmpty() ? 0 : maxHeap.peek();
}
```

#### Template B — Task Scheduler (Max Heap + Cooldown)

```

public int leastInterval(char[] tasks, int n) {
    int[] freq = new int[26];
    for (char task : tasks) freq[task - 'A']++;

    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
    for (int f : freq) if (f > 0) maxHeap.offer(f);

    // Queue to track tasks in cooldown: [remaining_count, ready_time]
    Queue<int[]> cooldown = new LinkedList<>();
    int time = 0;

    while (!maxHeap.isEmpty() || !cooldown.isEmpty()) {
        time++;

        // Release tasks whose cooldown has expired
        if (!cooldown.isEmpty() && cooldown.peek()[1] == time) {
            maxHeap.offer(cooldown.poll()[0]);
        }

        if (!maxHeap.isEmpty()) {
            int count = maxHeap.poll() - 1;
            if (count > 0) {
                cooldown.offer(new int[]{count, time + n + 1}); // ready after cooldown
            }
        }
        // else: CPU idles this cycle
    }

    return time;
}

```

## Template C — Reorganize String (Max Heap + Interleave)

```

public String reorganizeString(String s) {
    int[] freq = new int[26];
    for (char c : s.toCharArray()) freq[c - 'a']++;

    PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) -> b[1] - a[1]);
    for (int i = 0; i < 26; i++) {
        if (freq[i] > 0) maxHeap.offer(new int[]{i, freq[i]});
    }

    StringBuilder sb = new StringBuilder();
    int[] prev = null; // previously used char (in cooldown)

    while (!maxHeap.isEmpty()) {
        int[] curr = maxHeap.poll();
        sb.append((char)('a' + curr[0]));
        curr[1]--;

        // Re-add previous char (it's been one position since last use)
        if (prev != null && prev[1] > 0) maxHeap.offer(prev);

        prev = curr; // current goes into cooldown
    }

    return sb.length() == s.length() ? sb.toString() : "";
}

```

### Template D — IPO / Maximize Capital (Two Heap Unlock Pattern)

```

// Select at most k projects to maximize capital.
// Projects become available when capital >= cost.
public int findMaximizedCapital(int k, int w, int[] profits, int[] capital) {
    int n = profits.length;

    // Min-heap by capital requirement: unlock projects as we gain capital
    PriorityQueue<int[]> locked = new PriorityQueue<>((a, b) -> a[0] - b[0]);
    for (int i = 0; i < n; i++) locked.offer(new int[]{capital[i], profits[i]});

    // Max-heap by profit: always pick the most profitable unlocked project
    PriorityQueue<Integer> available = new PriorityQueue<>(Collections.reverseOrder());

    for (int i = 0; i < k; i++) {
        // Unlock all projects we can now afford
        while (!locked.isEmpty() && locked.peek()[0] <= w) {
            available.offer(locked.poll()[1]);
        }

        if (available.isEmpty()) break; // no affordable projects

        w += available.poll(); // pick most profitable available project
    }

    return w;
}

```

### Template E — Minimum Refueling Stops

```

public int minRefuelStops(int target, int startFuel, int[][] stations) {
    // Max-heap: stores fuel amounts of stations we've passed
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

    int fuel = startFuel;
    int stops = 0;
    int prev = 0; // previous position

    for (int[] station : stations) {
        int position = station[0];
        int stationFuel = station[1];

        fuel -= (position - prev); // fuel consumed to reach this station
        prev = position;

        // While we can't reach this station, refuel from best past station
        while (fuel < 0 && !maxHeap.isEmpty()) {
            fuel += maxHeap.poll();
            stops++;
        }

        if (fuel < 0) return -1; // can't reach even with all past stations

        maxHeap.offer(stationFuel); // this station is now available
    }

    // Check if we can reach target from last station
    fuel -= (target - prev);
    while (fuel < 0 && !maxHeap.isEmpty()) {
        fuel += maxHeap.poll();
        stops++;
    }

    return fuel >= 0 ? stops : -1;
}

```

## Template F — Furthest Building You Can Reach (Greedy Heap)

```
// Use ladders for the largest height differences, bricks for small ones
public int furthestBuilding(int[] heights, int bricks, int ladders) {
    // Min-heap: track the largest jumps where we used a ladder
    PriorityQueue<Integer> ladderJumps = new PriorityQueue<>();

    for (int i = 0; i < heights.length - 1; i++) {
        int diff = heights[i + 1] - heights[i];
        if (diff <= 0) continue; // going down – free

        // Greedily use ladder for this jump
        ladderJumps.offer(diff);

        // If used more ladders than available, replace smallest ladder jump with bricks
        if (ladderJumps.size() > ladders) {
            bricks -= ladderJumps.poll(); // smallest ladder jump → use bricks instead
        }

        if (bricks < 0) return i; // ran out of bricks
    }

    return heights.length - 1;
}
```

## 4. Complexity Cheatsheet

| Problem             | Time                  | Space   | Key Insight                      |
|---------------------|-----------------------|---------|----------------------------------|
| Last Stone Weight   | $O(n \log n)$         | $O(n)$  | Simulate with max-heap           |
| Task Scheduler      | $O(n \log n)$         | $O(1)$  | 26 tasks max                     |
| Reorganize String   | $O(n \log 26) = O(n)$ | $O(26)$ | 26 chars                         |
| IPO                 | $O(n \log n)$         | $O(n)$  | Two heaps: locked + available    |
| Min Refueling Stops | $O(n \log n)$         | $O(n)$  | Retroactive greedy               |
| Furthest Building   | $O(n \log k)$         | $O(k)$  | Swap ladders for bricks greedily |

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem           | Difficulty | Pattern                 |
|------|-------------------|------------|-------------------------|
| 1046 | Last Stone Weight | Easy       | Max-heap simulation     |
| 621  | Task Scheduler    | Medium     | Max-heap + cooldown     |
| 767  | Reorganize String | Medium     | Max-heap + interleave   |
| 502  | IPO               | Hard       | Two heaps (lock/unlock) |

| LC # | Problem                           | Difficulty | Pattern                     |
|------|-----------------------------------|------------|-----------------------------|
| 871  | Minimum Number of Refueling Stops | Hard       | Retroactive max-heap greedy |

## Advanced Variations

| LC # | Problem                         | Difficulty | Pattern                     |
|------|---------------------------------|------------|-----------------------------|
| 1642 | Furthest Building You Can Reach | Medium     | Min-heap swap greedy        |
| 630  | Course Schedule III             | Hard       | Max-heap + greedy swap      |
| 1834 | Single-Threaded CPU             | Medium     | Min-heap by enqueue time    |
| 2402 | Meeting Rooms III               | Hard       | Two heaps (free + occupied) |
| 1353 | Maximum Number of Events        | Medium     | Greedy + min-heap by end    |

## 6. Solve Step-by-Step — LC 871: Minimum Number of Refueling Stops

**Problem:** Car starts at position 0 with `startFuel` . Stations at `[position, fuel]` . Minimum stops to reach `target` .

### Why Retroactive Greedy Works

We don't decide when to refuel in advance. Instead:

- Drive as far as possible
- When we run out of fuel, retroactively pick the best station we passed (max-heap)
- This is optimal: we always pick the highest-fuel past station when needed

(Full implementation in Template E)

### Dry Run

```

target=100, startFuel=10
stations=[(10,60),(20,30),(30,30),(60,40)]

fuel=10, prev=0

station (10,60): fuel -= 10-0 = 0. heap=[60]
station (20,30): fuel -= 20-10 = -10 < 0
    → refuel from heap: fuel += 60 = 50, stops=1
    → fuel -= 0 (already at 20), heap=[30]
station (30,30): fuel -= 30-20 = 40. heap=[30,30]
station (60,40): fuel -= 60-30 = 10. heap=[40,30,30]

Final: fuel -= 100-60 = -30 < 0
    → refuel: fuel += 40 = 10, stops=2
    → still -30+40=10 >= 0? No: 10-40=... wait

Actually: fuel=10, need 40 more miles
fuel -= 40 → fuel = -30
refuel with max(40): fuel = -30+40 = 10 ≥ 0? No

Hmm—correct answer for this input is 2 stops.
(10→60 refuel at 10 gets us to 70,
then refuel at 60 gets to 100: 2 stops) ✓

```

---

## 7. The Two-Heap Unlock Pattern (IPO)

This is a recurring pattern worth memorizing separately:

PATTERN: Items unlock when you reach a threshold.  
Always pick the best unlocked item.

Two heaps:

1. Min-heap by unlock threshold → sorted by "when available"
2. Max-heap by value → gives best available greedily

Algorithm:

```

while k rounds remain:
    move all newly affordable items from min-heap to max-heap
    pick top of max-heap (best available)
    update resource (capital, time, etc.)

```

Problems using this exact pattern: IPO (502), Course Schedule III (630), Maximum Events (1353).

---

## 8. Common Interview Mistakes

- 1. Task Scheduler: using math formula only** — the heap simulation is more generalizable and interviewers prefer it
- 2. Reorganize String: not checking impossibility** — if max frequency > (n+1)/2, return ""
- 3. Refueling: adding station to heap before checking if reachable** — add AFTER confirming we reached it; we already consumed fuel to get there
- 4. IPO: wrong heap direction** — locked = min-heap (cheapest first), available = max-heap (most profitable first)

**5. Furthest Building: heap stores jumps not positions** — we track height differences, not building indices

---

## 9. Interview Cheat Sheet

### HEAP + GREEDY — MENTAL CHECKLIST

=====

- "Always pick best available" → Max/Min Heap
- Items unlock over time/resource → Two Heap pattern
- Rearrange with spacing constraint → Max-heap + prev cooldown
- Retroactive best choice → add to heap as passed, pull when needed
- Swap resource types (ladders/bricks) → Min-heap of used resource

### PATTERNS

=====

Simulation: max-heap, repeatedly merge/pick, check termination

Cooldown: max-heap + cooldown queue with (count, ready\_time)

Interleave: max-heap + prev variable (one-step cooldown)

Unlock: min-heap (by cost) + max-heap (by value)

Retroactive: collect into heap while passing, pull when resource < 0

### JAVA HEAP SYNTAX (REMINDER)

=====

Max-heap: `new PriorityQueue<>(Collections.reverseOrder())`

Min-heap: `new PriorityQueue<>()`

Custom: `new PriorityQueue<>((a,b) -> a[0]-b[0])`

### GREEDY CORRECTNESS ARGUMENT

=====

Always justify: "Picking the locally best option at each step is globally optimal because..."

- If we must refuel, best past station maximizes remaining range
- If we must use bricks, smallest ladder jump was least valuable
- If we must idle CPU, pick most frequent task to minimize future idles

---

## 10. Practice Roadmap

### Beginner:

- LC 1046 — Last Stone Weight
- LC 703 — Kth Largest Element in a Stream

### Intermediate:

- LC 767 — Reorganize String
- LC 621 — Task Scheduler
- LC 1642 — Furthest Building You Can Reach
- LC 1353 — Maximum Number of Events That Can Be Attended

### Hard:

- LC 871 — Minimum Number of Refueling Stops
- LC 502 — IPO
- LC 630 — Course Schedule III

- LC 2402 — Meeting Rooms III

# Pattern 25 — Tree Maze & Grid Backtracking (Path Finding)

## 1. Pattern Recognition Guide

**Keywords to look for:**

- "find all paths from root to leaf"
- "path exists in maze / grid"
- "all paths that sum to target"
- "explore all routes", "find path avoiding obstacles"
- "robot can move in 4 directions", "reach destination"
- "unique paths with obstacles"

**Tree Maze vs Word Search (Pattern 19):**

- **Word Search** = match a specific sequence of characters — must follow the word
- **Tree/Grid Maze** = find any valid path satisfying a structural condition (reach destination, hit target sum, collect all items)

**Signal phrase:** \_"Find all root-to-leaf paths with sum = target"\_ → Tree DFS + backtrack

**Signal phrase:** \_"Can the robot reach the exit?"\_ → Grid DFS/BFS + visited marking

## 2. Core Intuition

**Tree Maze = DFS with explicit path tracking + backtracking.**

The difference from plain DFS is that you need to:

1. **Record the path** as you go (not just the answer)
2. **Undo the recording** when you backtrack
3. **Collect/check** the full path at leaf/destination nodes

**Grid Maze adds:**

- 4-directional movement
- Bounds checking
- Visited marking (to avoid cycles)
- Restore visited mark after backtracking (if you need ALL paths)

Tree path sum = 22:

```
5
 / \
4   8
 / / \
11 13 4
 / \ \
7  2 1
```

Path: 5→4→11→2 = 22 ✓

Path: 5→8→4→1 = 18 ✗

Backtracking trace:

```
visit 5 (path=[5], rem=17)
  visit 4 (path=[5,4], rem=13)
    visit 11 (path=[5,4,11], rem=2)
      visit 7 (path=[5,4,11,7], rem=-5) → leaf, not 0 → backtrack
      visit 2 (path=[5,4,11,2], rem=0) → leaf, rem==0 → ADD ✓ → backtrack
    backtrack from 11 (path=[5,4])
  backtrack from 4 (path=[5])
visit 8 ...
```

---

## 3. Generic Java Templates

### Template A — All Root-to-Leaf Paths with Target Sum (Tree)

```
public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(root, targetSum, new ArrayList<>(), result);
    return result;
}

private void backtrack(TreeNode node, int remaining,
    List<Integer> path, List<List<Integer>> result) {
    if (node == null) return;

    path.add(node.val); // choose
    remaining -= node.val;

    // Check at leaf node
    if (node.left == null && node.right == null && remaining == 0) {
        result.add(new ArrayList<>(path)); // copy – important!
    } else {
        backtrack(node.left, remaining, path, result);
        backtrack(node.right, remaining, path, result);
    }

    path.remove(path.size() - 1); // unchoose (backtrack)
}
```

### Template B — All Paths from Root to Leaf (No Condition)

```

public List<String> binaryTreePaths(TreeNode root) {
    List<String> result = new ArrayList<>();
    if (root == null) return result;

    dfs(root, String.valueOf(root.val), result);
    return result;
}

private void dfs(TreeNode node, String path, List<String> result) {
    if (node.left == null && node.right == null) {
        result.add(path);
        return;
    }
    if (node.left != null) dfs(node.left, path + "->" + node.left.val, result);
    if (node.right != null) dfs(node.right, path + "->" + node.right.val, result);
}

```

### Template C — Grid Maze: Find If Path Exists (DFS)

```

public boolean hasPath(int[][] grid, int[] start, int[] end) {
    int rows = grid.length, cols = grid[0].length;
    boolean[][] visited = new boolean[rows][cols];
    return dfs(grid, start[0], start[1], end[0], end[1], visited);
}

private boolean dfs(int[][] grid, int r, int c, int er, int ec, boolean[][] visited) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return false;
    if (grid[r][c] == 1) return false; // obstacle
    if (visited[r][c]) return false; // already visited
    if (r == er && c == ec) return true; // reached destination

    visited[r][c] = true; // mark visited (no need to unmark for path existence)

    return dfs(grid, r+1, c, er, ec, visited) ||
        dfs(grid, r-1, c, er, ec, visited) ||
        dfs(grid, r, c+1, er, ec, visited) ||
        dfs(grid, r, c-1, er, ec, visited);
}

```

### Template D — Grid Maze: ALL Paths (Backtracking with Restore)

```

public List<List<int[]>> allPaths(int[][] grid, int[] start, int[] end) {
    List<List<int[]>> result = new ArrayList<>();
    boolean[][] visited = new boolean[grid.length][grid[0].length];
    List<int[]> path = new ArrayList<>();

    path.add(start);
    visited[start[0]][start[1]] = true;

    dfsAllPaths(grid, start[0], start[1], end[0], end[1], visited, path, result);
    return result;
}

private void dfsAllPaths(int[][] grid, int r, int c, int er, int ec,
    boolean[][] visited, List<int[]> path,
    List<List<int[]>> result) {
    if (r == er && c == ec) {
        result.add(new ArrayList<>(path));
        return;
    }

    int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};
    for (int[] d : dirs) {
        int nr = r + d[0], nc = c + d[1];
        if (nr < 0 || nr >= grid.length || nc < 0 || nc >= grid[0].length) continue;
        if (grid[nr][nc] == 1 || visited[nr][nc]) continue;

        visited[nr][nc] = true; // choose
        path.add(new int[]{nr, nc});
        dfsAllPaths(grid, nr, nc, er, ec, visited, path, result);
        path.remove(path.size() - 1); // unchoose
        visited[nr][nc] = false; // restore – CRITICAL for all-paths
    }
}

```

## Template E — Unique Paths with Obstacles (DP, not backtracking)

```

// Count distinct paths from top-left to bottom-right (only right/down moves)
public int uniquePathsWithObstacles(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    if (grid[0][0] == 1 || grid[rows-1][cols-1] == 1) return 0;

    int[] dp = new int[cols];
    dp[0] = 1;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 1) {
                dp[c] = 0; // obstacle blocks path
            } else if (c > 0) {
                dp[c] += dp[c - 1]; // paths from left + paths from above
            }
        }
    }

    return dp[cols - 1];
}

```

## Template F — Maze with Sliding (Ball Rolls Until Wall) — BFS

```
// LC 490: Ball rolls in a direction until hitting a wall; find path to hole
public boolean hasPath(int[][] maze, int[] start, int[] destination) {
    int rows = maze.length, cols = maze[0].length;
    boolean[][] visited = new boolean[rows][cols];
    Queue<int[]> queue = new LinkedList<>();

    queue.offer(start);
    visited[start[0]][start[1]] = true;

    int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};

    while (!queue.isEmpty()) {
        int[] curr = queue.poll();

        if (curr[0] == destination[0] && curr[1] == destination[1]) return true;

        for (int[] d : dirs) {
            int r = curr[0], c = curr[1];

            // Roll until hitting a wall
            while (r + d[0] >= 0 && r + d[0] < rows &&
                   c + d[1] >= 0 && c + d[1] < cols &&
                   maze[r + d[0]][c + d[1]] == 0) {
                r += d[0];
                c += d[1];
            }

            if (!visited[r][c]) {
                visited[r][c] = true;
                queue.offer(new int[]{r, c});
            }
        }
    }

    return false;
}
```

## Template G — Path Sum III (Count Paths in Tree, Any Node to Any Node)

```
// Count paths with sum = targetSum, starting from any node going downward
public int pathSum(TreeNode root, int targetSum) {
    Map<Long, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0L, 1);
    return dfs(root, 0L, targetSum, prefixCount);
}

private int dfs(TreeNode node, long currSum, int target, Map<Long, Integer> prefixCount) {
    if (node == null) return 0;

    currSum += node.val;
    int count = prefixCount.getOrDefault(currSum - target, 0);

    prefixCount.merge(currSum, 1, Integer::sum);

    count += dfs(node.left, currSum, target, prefixCount);
    count += dfs(node.right, currSum, target, prefixCount);

    // Backtrack: remove current node's prefix sum
    prefixCount.merge(currSum, -1, Integer::sum);

    return count;
}
```

## 4. Complexity Cheatsheet

| Problem                | Time                        | Space           | Notes                           |
|------------------------|-----------------------------|-----------------|---------------------------------|
| All root-to-leaf paths | $O(n \times h)$             | $O(h)$          | $h$ = height, copy path at leaf |
| Grid maze existence    | $O(m \times n)$             | $O(m \times n)$ | Each cell visited once          |
| Grid maze all paths    | $O(4^{(m \times n)})$ worst | $O(m \times n)$ | Exponential — only small grids  |
| Unique paths (DP)      | $O(m \times n)$             | $O(n)$          | Rolling 1D DP                   |
| Path Sum III           | $O(n)$                      | $O(n)$          | Prefix sum + HashMap            |

**Key:** If problem asks for EXISTENCE or OPTIMAL path → use BFS/DP. If it asks for ALL paths → backtracking. Never use backtracking for "count distinct paths" — use DP.

## 5. Canonical LC Problems

### Core (Must Solve)

| LC # | Problem                     | Difficulty | Template        |
|------|-----------------------------|------------|-----------------|
| 113  | Path Sum II (all paths)     | Medium     | Template A      |
| 257  | Binary Tree Paths           | Easy       | Template B      |
| 63   | Unique Paths II (obstacles) | Medium     | Template E (DP) |
| 437  | Path Sum III (any node)     | Medium     | Template G      |

| LC # | Problem     | Difficulty | Template          |
|------|-------------|------------|-------------------|
| 79   | Word Search | Medium     | Grid backtracking |

## Advanced Variations

| LC # | Problem                           | Difficulty | Template                  |
|------|-----------------------------------|------------|---------------------------|
| 980  | Unique Paths III (visit all)      | Hard       | Grid backtracking + count |
| 490  | The Maze                          | Medium     | Template F (BFS rolling)  |
| 505  | The Maze II (shortest)            | Medium     | Dijkstra rolling          |
| 1219 | Path with Maximum Gold            | Medium     | Grid DFS + backtrack      |
| 329  | Longest Increasing Path in Matrix | Hard       | DFS + memoization         |

## 6. Solve Step-by-Step — LC 980: Unique Paths III

**Problem:** Count paths from start to end that visit every non-obstacle cell exactly once.

### Why Backtracking (not DP)

We must visit ALL empty cells — this is a Hamiltonian path problem. No overlapping subproblems in the classical sense; backtracking with pruning is the way.

```

public int uniquePathsIII(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    int startR = 0, startC = 0, totalEmpty = 0;

    // Find start position and count empty cells
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] != -1) totalEmpty++; // count non-obstacles
            if (grid[r][c] == 1) { startR = r; startC = c; }
        }
    }

    return dfs(grid, startR, startC, totalEmpty);
}

private int dfs(int[][] grid, int r, int c, int remaining) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return 0;
    if (grid[r][c] == -1) return 0; // obstacle

    if (grid[r][c] == 2) { // reached end
        return remaining == 1 ? 1 : 0; // valid only if all cells visited
    }

    int temp = grid[r][c];
    grid[r][c] = -1; // mark as visited (temporarily obstacle)

    int paths = dfs(grid, r+1, c, remaining-1) +
        dfs(grid, r-1, c, remaining-1) +
        dfs(grid, r, c+1, remaining-1) +
        dfs(grid, r, c-1, remaining-1);

    grid[r][c] = temp; // restore
    return paths;
}

```

## Dry Run (2x2 grid)

Grid: 1 0  
0 2

Start=(0,0), empty=4

```

dfs(0,0,4): mark (0,0) as -1
dfs(1,0,3): mark (1,0) as -1
dfs(1,1,2): this is 2(end), remaining=2 ≠ 1 → 0 (didn't visit all)
dfs(0,0,2): already -1 → 0
... → 0
dfs(0,1,3): mark (0,1) as -1
dfs(1,1,2): end, remaining=2≠1 → 0
dfs(0,0,2): -1 → 0
... → 0

```

Total with 2x2 having only one valid path: [expected answer = 2 for 3x3 case]

## 7. Tree vs Grid vs Implicit — Comparison

| Aspect            | Tree Maze                       | Grid Maze                    | Implicit Graph                     |
|-------------------|---------------------------------|------------------------------|------------------------------------|
| Structure         | Binary/N-ary tree               | 2D grid                      | Generated on the fly               |
| Movement          | left/right child                | 4 directions                 | Problem-defined transitions        |
| Visited tracking  | Not needed (tree, no cycles)    | <code>visited[][]</code>     | <code>Set&lt;State&gt;</code>      |
| Backtrack restore | <code>Path list remove()</code> | <code>visited = false</code> | <code>visited.remove(state)</code> |
| When to stop      | At leaf or null                 | At destination or bounds     | At target state                    |

## 8. Common Interview Mistakes

- 1. Not copying path when adding to result:** `result.add(path)` adds a reference — use `new ArrayList<>(path)`
- 2. Not restoring** `visited[r][c] = false` when finding ALL paths (for existence only, no need)
- 3. Counting paths with DP when backtracking is needed:** unique paths (no constraint) → DP; must-visit-all → backtracking
- 4. Path Sum III: not backtracking the prefix map** — `prefixCount.merge(currSum, -1, ...)` after recursion
- 5. Grid bounds check order:** check bounds BEFORE accessing `grid[r][c]` — short-circuit prevents index out of bounds
- 6. Confusing** `remaining` **semantics:** in Unique Paths III, decrement before passing to child (not at child entry)

## 9. Interview Cheat Sheet

## TREE / GRID MAZE BACKTRACKING — MENTAL CHECKLIST

=====

- ALL paths? → Backtracking with path list + copy at destination
- PATH EXISTS? → DFS/BFS, no restore needed
- COUNT paths (no must-visit-all)? → DP (not backtracking!)
- MUST visit all cells? → Backtracking with remaining counter
- Tree? → No visited needed (trees are acyclic)
- Grid? → visited[][] needed, restore if finding ALL paths

## TREE PATH TEMPLATE (MEMORIZE)

=====

```
void dfs(node, remaining, path, result):
    if null: return
    path.add(node.val)
    remaining -= node.val
    if leaf and remaining == 0:
        result.add(new ArrayList<>(path))
    else:
        dfs(left, ...) ; dfs(right, ...)
    path.remove(last) ← BACKTRACK
```

## GRID PATH TEMPLATE (MEMORIZE)

=====

```
void dfs(grid, r, c, ...):
    if out of bounds or obstacle or visited: return
    if destination: record answer
    visited[r][c] = true ← mark
    for each direction: dfs(next)
    visited[r][c] = false ← restore (if finding ALL paths)
```

## PATH SUM III TRICK

=====

Use prefix sum + HashMap (like subarray sum = k)  
Backtrack: remove current prefix from map after recursion  
Use Long for sum to avoid integer overflow

---

## 10. Practice Roadmap

### Beginner:

- LC 257 — Binary Tree Paths
- LC 112 — Path Sum
- LC 113 — Path Sum II

### Intermediate:

- LC 437 — Path Sum III
- LC 63 — Unique Paths II
- LC 1219 — Path with Maximum Gold
- LC 490 — The Maze

### Hard:

- LC 980 — Unique Paths III (visit all cells)
- LC 329 — Longest Increasing Path in Matrix

- LC 505 — The Maze II (shortest path with rolling)
- LC 126 — Word Ladder II (all shortest transformation paths)